Manual técnico

Gestión de Inventario de Componentes Electrónicos

Módulos, código fuente y estructura de la base de datos

Dato	Valor
Versión del manual	1.1
Fecha	22 de octubre de 2026
Corresponde a	proyecto_completo_PyQt5.zip entregado en esa fecha (aplicación PyQt5 refactorizada por capas, con almacén de documentos por hash SHA-256)
Documentos relacionados	Manual_Inventario (usuario), Manual_Instalacion, Manual_Tecnico

Índice

Apéndice A. Módulos de la aplicación: descripción 3
app_inventario.py 3
config.py 3
catalogo.py 3
db.py 3
almacenamiento.py 3
utilidades.py 3
ocr.py 3
capital.py 3
migrar_a_hash.py 3
interfaz__init__.py 8
interfaz\widgets.py 8
interfaz\arbol_parametros.py 8
interfaz\ventana_principal.py 9
interfaz\dialogo_componente.py 9
interfaz\dialogo_parametros.py 10
interfaz\dialogo_catalogos.py 10
interfaz\dialogo_capital.py 11
interfaz\dialogo_etiquetas.py 11
interfaz\editor_imagen.py 11
interfaz\visor_pdf.py 11
Apéndice B. Código fuente completo 13
app_inventario.py 13
config.py 14
catalogo.py 15
db.py 18
almacenamiento.py 29
utilidades.py 33
ocr.py 34
capital.py 36
migrar_a_hash.py 41
interfaz__init__.py 44
interfaz\widgets.py 45
interfaz\arbol_parametros.py 48
interfaz\ventana_principal.py 50
interfaz\dialogo_componente.py 58
interfaz\dialogo_parametros.py 72
interfaz\dialogo_catalogos.py 75
interfaz\dialogo_capital.py 81
interfaz\dialogo_etiquetas.py 84
interfaz\editor_imagen.py 88
interfaz\visor_pdf.py 91
Apéndice C. Base de datos: estructura en árbol 97
C.1 Vista general 97
C.2 Árbol completo de tablas y columnas 97
C.3 Relaciones 98
C.4 Contenido de la tabla listas_valores 98
C.5 Contenido inicial de una base nueva 98

Este manual reúne la documentación técnica de la aplicación para quien la mantiene o la amplía: descripción de cada módulo, su código fuente completo y la estructura de la base de datos. Para instalar y poner en marcha la aplicación vea el Manual de instalación; para usarla, el Manual del usuario.

Apéndice A. Módulos de la aplicación: descripción

La aplicación está organizada en capas. Cada capa solo usa a las que están debajo, y la interfaz no contiene consultas a la base de datos: todo el SQL está en db.py. Este apéndice describe cada módulo; el siguiente (Apéndice B) reproduce su código fuente completo.

Capa	Módulos	Responsabilidad
Arranque	app_inventario.py	Prepara carpetas y base de datos y abre la ventana principal.
Configuración y datos de referencia	config.py, catalogo.py	Rutas portátiles; taxonomía, listas y parámetros iniciales (solo constantes).
Acceso a datos	db.py	Toda la lógica SQLite: consultas, altas, bajas, validaciones.
Servicios	almacenamiento.py, ocr.py, utilidades.py	Documentos por hash SHA-256, reconocimiento de texto, funciones de texto.
Lógica de negocio	capital.py	Valorización del inventario y planilla Excel (sin PyQt).
Herramientas	migrar_a_hash.py	Migración única de adjuntos antiguos al almacén por hash.
Interfaz (PyQt5)	carpeta interfaz	Una ventana o diálogo por módulo.

Orden de dependencia: interfaz → (capital, almacenamiento, ocr, utilidades) → db → (catalogo, config).

app_inventario.py

Capa: Arranque. Líneas: 33.

Usa módulos del proyecto: almacenamiento, config, db, interfaz.ventana_principal

> *Punto de entrada de la aplicación de inventario de componentes electrónicos.*
>
> *Solo prepara carpetas y base de datos y abre la ventana principal.*

Este módulo no define funciones ni clases propias: solo contiene constantes o instrucciones de arranque.

config.py

Capa: Configuración. Líneas: 30.

Usa módulos del proyecto: ninguno.

> *Rutas de la aplicación.*
>
> *Importar este módulo NO toca el disco. Las carpetas se crean al llamar a asegurar_directorios(), que la aplicación invoca una sola vez al arrancar.*

Elemento	Tipo	Qué hace
asegurar_directorios()	Función	Crea las carpetas que la aplicación espera encontrar (idempotente).

catalogo.py

Capa: Datos de referencia. Líneas: 196.

Usa módulos del proyecto: ninguno.

> *Datos de referencia iniciales: taxonomía, listas de valores y parámetros por clasificación.*
>
> *Solo contiene constantes (sin lógica ni dependencias). La taxonomía y las listas son la SEMILLA de las tablas catalogo_clasificacion, listas_valores y plantillas_parametros: se copian a la base una sola vez; después se administran desde la aplicación.*

Este módulo no define funciones ni clases propias: solo contiene constantes o instrucciones de arranque.

db.py

Capa: Acceso a datos. Líneas: 811.

Usa módulos del proyecto: catalogo, config, utilidades

Capa de acceso a datos (SQLite).

Todas las consultas de la aplicación viven aquí: la interfaz no abre conexiones propias, llama a las funciones de este módulo.

Importar este módulo no toca el disco. La base se crea al llamar a inicializar_base_de_datos().

Convención de filtros: un filtro en None (o vacío) significa "sin filtrar". La interfaz es quien traduce sus textos comodín ("-- Todos los Tipos --", etc.) a None antes de llamar.

Elemento	Tipo	Qué hace
conexion()	Función	Abre una conexión con las claves foráneas activadas y la cierra al salir.
inicializar_base_de_datos()	Función	Crea las tablas si no existen, aplica las adaptaciones de versiones anteriores y siembra el catálogo y las listas iniciales.
_ruta(valores)	Función	Normaliza un camino de hasta 4 niveles a una tupla de 4 textos ('' = nivel sin definir).
_insertar_ruta(conn, ruta)	Función	Agrega un camino al catálogo (si no existe) y quita los marcadores de nivel que ya tienen hijos.
_valores_de(texto, multiple)	Función	Valores sueltos de un campo de componente (los de selección múltiple se guardan como 'A, B, C').
_agregar_valor(conn, lista, valor)	Función	Inserta un valor en una lista de desplegable (si no existe) al final del orden.
_asegurar_sin_dato(conn)	Función	«N/A» encabeza las listas que lo llevan (también en bases creadas antes de que existiera en ellas).
_sembrar(conn)	Función	Carga el contenido inicial. Cada tabla se siembra solo si está vacía, así lo que el usuario borre o renombre después no vuelve a aparecer. También incorpora lo que ya hay en los componentes.
_listar_nivel(nivel, previos)	Función	Nombres distintos del nivel `nivel` (0..3). `previos`: valores de los niveles anteriores; None/'' = cualquiera.
listar_tipos()	Función	Nombres de los tipos de componente (primer nivel de la taxonomía).
listar_categorias(tipo)	Función	Categorías del catálogo; con `tipo` solo las de ese tipo.
listar_subcategorias(tipo, cat)	Función	Subcategorías del catálogo; cada filtro presente se aplica.
listar_items(tipo, cat, subcat)	Función	Ítems del catálogo; cada filtro presente se aplica.
_validar_nombre(nombre)	Función	Valida un nombre del catálogo: no vacío y sin empezar con «--».
_condicion_ruta(ruta, columnas)	Función	('c1=? AND c2=?', params) para los niveles dados en `ruta`.
listar_catalogo()	Función	Todos los caminos del catálogo: lista de (tipo, categoría, subcategoría, ítem); '' = nivel sin definir.
agregar_catalogo(ruta)	Función	Agrega un nodo al catálogo. `ruta`: nombres desde el tipo hasta el nivel nuevo (1 a 4). Devuelve la ruta tal como quedó (el tipo sin número de prefijo). Lanza ValueError si no es válido.
contar_componentes_catalogo(ruta)	Función	Cuántos componentes están clasificados bajo ese nodo.
renombrar_catalogo(ruta, nuevo)	Función	Cambia el nombre del último nivel de `ruta`, en el catálogo y en los componentes que lo usan.
eliminar_catalogo(ruta)	Función	Quita el nodo (y todo lo que cuelga de él, con sus parámetros) del catálogo. Los componentes conservan su clasificación escrita.
ruta_de_clasificacion(tipo, cat, subcat, item)	Función	Camino de un componente como tupla de 0 a 4 nombres (se corta en el primer nivel vacío o comodín «-- … --»).
_nodo(ruta)	Función	Ruta de 1 a 4 nombres -> tupla de 4 con '' en los niveles que faltan.
listar_plantillas()	Función	Todos los parámetros definidos: lista de (ruta del nodo, parámetro), en el orden de cada nodo.
listar_parametros_plantilla(ruta)	Función	Parámetros propios de ese nodo (sin los heredados).
parametros_heredados(ruta)	Función	Parámetros que corresponden a un componente de esa clasificación: [(parámetro, ruta del nodo que lo define)].
agregar_parametro_plantilla(ruta, parametro)	Función	Agrega un parámetro al nodo y devuelve el nombre tal como quedó (si ya existía, el existente).
renombrar_parametro_plantilla(ruta, viejo, nuevo)	Función	Cambia el nombre de un parámetro de ese nodo. No toca los valores ya guardados en los componentes.
eliminar_parametro_plantilla(ruta, parametro)	Función	Quita un parámetro de ese nodo. Los valores ya guardados en los componentes se conservan.
listar_valores(lista)	Función	Valores de la lista, en el orden en que se cargaron.
_validar_valor(lista, valor)	Función	Valida un valor de lista: nombre válido, distinto de «N/A» y sin comas donde se eligen varias opciones.
agregar_valor(lista, valor)	Función	Agrega un valor a la lista y devuelve el texto tal como quedó guardado (si ya existía con otras mayúsculas, devuelve el existente). Lanza ValueError si no es válido.
_usos(conn, lista, valor)	Función	[(id_componente, columna, texto)] de los componentes que tienen ese valor en la lista.

Elemento	Tipo	Qué hace
contar_uso_valor(lista, valor)	Función	Cuántos componentes usan ese valor.
renombrar_valor(lista, viejo, nuevo)	Función	Cambia un valor de la lista y también en los componentes que lo usan. Lanza ValueError si no es válido.
eliminar_valor(lista, valor)	Función	Quita el valor de la lista. Los componentes que ya lo tienen conservan el texto.
buscar_componentes(tipo, cat, subcat, item, texto)	Función	Filas para la grilla principal (19 columnas, el orden importa a la interfaz).
obtener_componente(comp_id)	Función	Todas las columnas del componente como diccionario, o None si no existe.
obtener_datos_etiqueta(item_id)	Función	(chip_principal, item_especifico, descripcion, ubicacion, fabricante, stock) o None.
_valores_componente(d)	Función	Valores de la fila en el orden exacto que esperan el UPDATE y el INSERT.
guardar_componente(comp_id, datos, archivos, parametros)	Función	Inserta (comp_id=None) o actualiza un componente con sus archivos y parámetros.
eliminar_componente(comp_id)	Función	Borra el componente. Con foreign_keys activo, el CASCADE borra sus filas de archivos y parámetros.
listar_inventario()	Función	Todos los componentes con lo necesario para valorizar el stock: lista de diccionarios.
listar_cotizaciones()	Función	[(moneda, unidades por 1 USD o None si falta, fecha o None)]: USD primero, luego las monedas de la lista y las que usan los componentes.
guardar_cotizacion(moneda, por_usd, fecha)	Función	Guarda cuántas unidades de `moneda` equivalen a 1 USD (None = borrar la cotización). La fecha por defecto es hoy. Lanza ValueError si el valor no es válido.
listar_archivos(comp_id)	Función	Lista de (tipo_archivo, nombre_amigable, ruta_archivo).
listar_archivos_con_id(comp_id)	Función	Lista de (id, tipo_archivo, ruta_archivo), para verificar que existan en disco.
listar_rutas_archivos()	Función	Todas las rutas de archivos adjuntos registradas (de todos los componentes), sin repetir.
listar_todos_los_archivos()	Función	Lista de (id, ruta_archivo) de todos los adjuntos (usado por la migración al almacén por hash).
actualizar_rutas_archivos(cambios)	Función	Cambia la ruta de varios adjuntos en una sola transacción. `cambios`: lista de (id, ruta_nueva).
eliminar_archivos_por_id(ids)	Función	Borra filas de archivos_componentes por su id.
listar_parametros(comp_id)	Función	Lista de (parametro_clave, parametro_valor).

almacenamiento.py

Capa: Almacenamiento de documentos. Líneas: 275.

Usa módulos del proyecto: config

Almacenamiento de documentos por contenido (hash SHA-256).

Punto único por el que la aplicación guarda, comprueba y retira archivos adjuntos. La interfaz no toca el disco por su cuenta: llama a este módulo.

Cómo se guardan los documentos (etapa 7)

** Cada documento se guarda UNA sola vez, con el nombre de su huella SHA-256:*
adjuntos/almacen/<2 primeros caracteres>/<huella><extensión>
Dos documentos con el mismo contenido (aunque tengan distinto nombre o se suban a distintos componentes) ocupan un único archivo.
** La base de datos guarda la ruta RELATIVA a la carpeta `adjuntos` (por ejemplo "almacen/ab/ab12...ef.pdf"), con barras "/". Así la carpeta de la aplicación se puede mover o copiar a otro equipo sin romper nada. El nombre que ve el usuario ("LM358.pdf") queda en la columna `nombre_amigable`, no en el disco.*
** La escritura es atómica: se copia a un archivo temporal dentro del almacén, se calcula la huella en la misma pasada y recién entonces se renombra al nombre definitivo. Un corte a mitad de copia nunca deja un documento a medias.*
** Quitar un documento de un componente NO borra el archivo (otro componente puede estar usándolo). Los archivos que ya nadie referencia se mueven a `adjuntos/papelera` al arrancar la aplicación (limpiar_huerfanos); nada se borra de forma definitiva.*
** La cantidad de usos (conteo de referencias) no se guarda aparte: se obtiene de la base de datos, que es la fuente de verdad.*

Las rutas que maneja la interfaz en memoria son absolutas; `resolver` convierte la ruta guardada en la base de datos y `para_guardar` hace el camino inverso.

Importar este módulo no toca el disco.

Elemento	Tipo	Qué hace
nombre_archivo(chip, resto)	Función	'<chip saneado>_<resto>': el chip solo conserva letras y números (lo demás pasa a '_').
ruta_temporal(nombre)	Función	Ruta de un archivo de trabajo en adjuntos/tmp.
ruta_pdf_etiquetas()	Función	Ruta fija (carpeta `Etiquetas`) del PDF de etiquetas generado; crea la carpeta si falta.
resolver(ruta)	Función	Ruta guardada en la base de datos -> ruta absoluta en disco.
resolver_archivos(filas)	Función	Aplica `resolver` a la ruta de cada fila (tipo, nombre, ruta) leída de la base.
esta_en_almacen(ruta)	Función	True si la ruta (absoluta o relativa) apunta dentro del almacén por hash.
_relativa(ruta_absoluta)	Función	Convierte una ruta absoluta en la ruta relativa a la carpeta adjuntos, con «/» como separador.
calcular_hash(ruta)	Función	Huella SHA-256 (hexadecimal) del contenido de un archivo.
_extension(origen)	Función	Extensión del archivo de origen en minúsculas, si es válida; si no, cadena vacía.
_buscar_existente(carpeta, huella)	Función	Archivo ya guardado con esa huella (con cualquier extensión), o None.
incorporar(origen)	Función	Guarda el archivo `origen` en el almacén. Devuelve (ruta_relativa, huella).
preparar_adjunto(origen)	Función	Registra un archivo elegido por el usuario. Devuelve (ruta, nombre_original).
para_guardar(ruta)	Función	Ruta en memoria -> ruta que se guarda en la base de datos.
guardar_imagen(imagen, destino)	Función	Guarda una imagen en la ruta que eligió el usuario, creando la carpeta si falta.
guardar_temporal(imagen, nombre, formato)	Función	Guarda una imagen en adjuntos/tmp y devuelve la ruta. `formato` es opcional (p. ej. 'PNG').
existe(ruta)	Función	True si el archivo existe en disco (acepta ruta absoluta o relativa a `adjuntos`).
listar_almacen()	Función	Rutas absolutas de todos los documentos del almacén (sin las copias en curso).
verificar_almacen()	Función	Recalcula la huella de cada documento. Devuelve [(ruta, problema)] (vacía si todo está bien).
huerfanos(rutas_en_uso)	Función	Documentos del almacén que ninguna fila de la base referencia.
limpiar_huerfanos(rutas_en_uso, antiguedad_minima_s)	Función	Mueve a `adjuntos/papelera` los documentos sin uso. Devuelve las rutas movidas.

utilidades.py

Capa: Utilidades. Líneas: 16.

Usa módulos del proyecto: ninguno.

Funciones de texto sin dependencias de la interfaz.

Elemento	Tipo	Qué hace
limpiar_prefijo(texto)	Función	Quita el prefijo numérico de un nombre (por ejemplo «01. Resistencias» → «Resistencias»).
sin_comodin(valor)	Función	Devuelve None si el valor está vacío o es un texto comodín ("-- Todos --"); si no, el valor.

ocr.py

Capa: Servicios (OCR). Líneas: 116.

Usa módulos del proyecto: config

Reconocimiento de texto (OCR) con Tesseract, para páginas de PDF cuyo texto es parte de una imagen.

Requiere DOS cosas (la biblioteca de Python es solo el puente hacia el programa):
1. *La biblioteca de Python: pip install pytesseract*
2. *El programa Tesseract:*
   - *Windows: instalador de UB Mannheim (https://github.com/UB-Mannheim/tesseract/wiki). Se instala en C:\Program Files\Tesseract-OCR. En el instalador conviene tildar el idioma "Spanish" (Additional language data).*
   - *Si Tesseract está en otra carpeta, defina la variable de entorno TESSERACT_CMD con la ruta de tesseract.exe.*
   - *Versión portátil: una carpeta "Tesseract-OCR" junto a la aplicación (ver config.BASE_DIR) se usa antes que cualquier otra; así el OCR viaja con el pendrive.*

Este módulo no depende de la interfaz: recibe la ruta de una imagen y devuelve el texto.
Importarlo no ejecuta nada (pytesseract se importa al usarlo, así la aplicación arranca aunque falte).

Elemento	Tipo	Qué hace
OcrError(Exception)	Clase	El OCR no pudo ejecutarse (falta algo, falló o tardó demasiado). El mensaje se puede mostrar al usuario.
_ruta_portatil()	Función	tesseract.exe dentro de <carpeta de la aplicación>\Tesseract-OCR, si existe (instalación portátil).
buscar_tesseract()	Función	Ruta del programa Tesseract, o None si no está instalado.
_preparar()	Función	Devuelve el módulo pytesseract configurado con la ruta del programa.
_idioma(pytesseract, exe)	Función	Mejor combinación de idiomas instalada (inglés + español si están; si no, lo que haya).
_limpiar(texto)	Función	Limpia el texto reconocido: quita saltos de página, espacios repetidos y líneas vacías.
reconocer_texto(ruta_imagen)	Función	Texto reconocido en la imagen (cadena vacía si no encontró nada). Lanza OcrError si no se pudo ejecutar.

capital.py

Capa: Lógica de negocio (capital). Líneas: 314.

Usa módulos del proyecto: db

Capital en existencia: valorización del inventario y planilla Excel.

Criterio (decidido con el usuario):
** Moneda base: USD. Cada precio se convierte con la cotización cargada en la tabla `cotizaciones` (unidades de la moneda por 1 USD). Nunca se inventa una cotización: si falta, el componente queda «Sin cotización» y no suma.*
** Precio de cada componente: el local si existe (> 0); si no, el del exterior.*
** Capital de un componente = precio unitario en USD × stock.*

La planilla se escribe con FÓRMULAS, así que al cambiar una cotización dentro de Excel se recalcula todo. `valorizar` repite en Python la misma regla para poder mostrar un resumen antes de generar el archivo y para verificar que las fórmulas son correctas.

Este módulo no usa la interfaz. `openpyxl` se importa recién al exportar, así la aplicación arranca aunque no esté instalado.

Elemento	Tipo	Qué hace
ExportError(Exception)	Clase	No se pudo generar la planilla; el mensaje explica el motivo.
_numero(valor)	Función	Convierte un valor a número; devuelve 0 si está vacío o no es numérico.
valorizar(componente, cotizaciones)	Función	Valora un componente. `cotizaciones`: {moneda en minúscula: unidades por 1 USD} (solo las cargadas).
calcular()	Función	Valoriza todo el inventario con las cotizaciones cargadas.
formato_monto(valor)	Función	1234567.5 -> '1.234.567,50' (como se escribe en Uruguay).
_grupos(filas)	Función	Tipos, categorías y subcategorías que realmente tienen componentes, ordenados sin distinguir mayúsculas.
subtotales(filas)	Función	Subtotales por clasificación (para mostrar en pantalla).
exportar_xlsx(destino, datos)	Función	Escribe la planilla en `destino` (.xlsx). `datos`: resultado de calcular() (si no se da, se calcula).

migrar_a_hash.py

Capa: Herramienta de migración. Líneas: 165.

Usa módulos del proyecto: almacenamiento, config, db

Migra los documentos existentes al almacén por hash SHA-256 (etapa 7).

Uso (desde la carpeta de la aplicación):

```
python migrar_a_hash.py                 # SOLO INFORMA qué haría; no cambia nada
python migrar_a_hash.py --aplicar       # migra (antes hace una copia de la base de datos)
python migrar_a_hash.py --aplicar --archivar-originales
                                        # además mueve los archivos viejos a adjuntos/originales_migrados
```

Qué hace, para cada documento registrado en la base de datos:
1. Localiza el archivo (ruta absoluta antigua o ruta relativa a `adjuntos`).
2. Lo copia al almacén con el nombre de su huella SHA-256. Si otro documento tiene exactamente el mismo contenido, comparten un único archivo.

3. Cambia la ruta guardada en la base de datos por la ruta relativa del almacén.

Garantías:
** Por defecto no modifica nada (modo informe).*
** Con --aplicar, primero copia la base de datos a un archivo .bak con fecha y hora.*
** Los archivos originales NO se borran. Con --archivar-originales se mueven (no se eliminan) a una carpeta aparte, y solo si ya están a salvo en el almacén.*
** Los documentos que no se encuentran en el disco quedan como estaban y se listan.*
** Se puede ejecutar varias veces: lo ya migrado se reconoce y no se repite.*

Elemento	Tipo	Qué hace
_respaldar_base()	Función	Copia la base de datos a un archivo .bak con fecha y hora, sin pisar copias anteriores.
migrar(aplicar, archivar_originales, salida)	Función	Devuelve un dict con el resumen. Con aplicar=False no escribe nada (ni en disco ni en la base).
_mb(n)	Función	Formatea un tamaño en bytes como megabytes.
main(argv)	Función	Lee las opciones de la línea de comandos (--aplicar, --archivar-originales) y ejecuta la migración.

interfaz__init__.py

Capa: Interfaz. Líneas: 1.

Usa módulos del proyecto: ninguno.

Interfaz gráfica (PyQt5): un módulo por ventana o diálogo.

Este módulo no define funciones ni clases propias: solo contiene constantes o instrucciones de arranque.

interfaz\widgets.py

Capa: Interfaz. Líneas: 161.

Usa módulos del proyecto: ninguno.

Widgets propios reutilizables.

Elemento	Tipo	Qué hace
aplicar_franjas(widget)	Función	Pinta las filas de una grilla (tabla, árbol o lista) en colores alternados, con selección azul.
LineEditSeguro(QLineEdit)	Clase	Campo de texto con estilo fijo que ignora Enter, para que no cierre el diálogo por accidente.
(otros métodos)	Métodos	__init__, keyPressEvent
ComboBoxMultiCheckbox(QComboBox)	Clase	Desplegable con casillas. Si se da `on_nueva`, el menú ofrece «Agregar nueva opción…».
.agregar_opcion()	Método	Agrega una opción al menú (si no estaba) y, con `marcar`, la deja elegida.
(otros métodos)	Métodos	__init__, _crear_casilla, _pedir_nueva_opcion, showPopup, actualizar_texto_seleccionado, text, set_seleccionadas

interfaz\arbol_parametros.py

Capa: Interfaz. Líneas: 89.

Usa módulos del proyecto: db, interfaz.widgets

Árbol de la taxonomía con los parámetros propios de cada nodo.

Lo usan la ventana de parámetros del componente (consulta) y la ventana de administración (edición). No tiene consultas propias: lee de db.py.

Elemento	Tipo	Qué hace
ArbolParametros(QTreeWidget)	Clase	Árbol de la taxonomía con los parámetros propios de cada nodo (ver la descripción del módulo).
.refrescar()	Método	Reconstruye el árbol conservando los nodos abiertos.
.nodo_actual()	Método	Camino del nodo elegido (si hay un parámetro elegido, el de su nodo), o None.
.parametro_actual()	Método	(camino, nombre) si lo elegido es un parámetro; si no, None.
(otros métodos)	Métodos	__init__, _rutas_abiertas

interfaz\ventana_principal.py

Capa: Interfaz. Líneas: 553.

Usa módulos del proyecto: almacenamiento, db, interfaz.dialogo_capital, interfaz.dialogo_catalogos, interfaz.dialogo_componente, interfaz.dialogo_etiquetas, interfaz.dialogo_parametros, interfaz.visor_pdf, interfaz.widgets, utilidades

Ventana principal: búsqueda, listado y acciones sobre componentes.

Elemento	Tipo	Qué hace
AppInventario(QMainWindow)	Clase	Ventana principal de la aplicación: búsqueda, listado y botones de acción.
.abrir_nuevo_dialogo()	Método	Flujo exclusivo y limpio para crear un componente nuevo.
.abrir_administrador_catalogos()	Método	Ventana para agregar, renombrar y quitar entradas del catálogo y de las listas de los desplegables.
.abrir_capital()	Método	Muestra el capital en existencia en una grilla (con opción de exportarlo a Excel).
.abrir_dialogo_parametros_avanzados()	Método	Consulta los parámetros guardados del componente seleccionado, junto al árbol de la taxonomía.
.abrir_editar_dialogo()	Método	Flujo exclusivo para editar un componente existente de la grilla.
(otros métodos)	Métodos	__init__, init_ui, cargar_comboboxes_cascada, on_tipo_changed, on_cat_changed, on_subcat_changed, filtrar_tabla, limpiar_y_verificar_archivos, on_row_selected, abrir_archivo_asociado, eliminar_registro, abrir_impresion

interfaz\dialogo_componente.py

Capa: Interfaz. Líneas: 1063.

Usa módulos del proyecto: almacenamiento, db, interfaz.dialogo_parametros, interfaz.editor_imagen, interfaz.visor_pdf, interfaz.widgets, utilidades

Diálogo de alta y edición de un componente.

Elemento	Tipo	Qué hace
DialogoNuevoComponente(QDialog)	Clase	Ventana de alta y edición de un componente (según reciba o no los datos de uno existente).
._construir_cabecera()	Método	Miniatura, título del componente y vista previa del QR.
._construir_checkbox_edicion()	Método	Casilla para habilitar la edición completa (solo al editar).
._construir_clasificacion()	Método	Clasificación, descripción y datos técnicos en cuadrícula.
._crear_combo_proveedor()	Método	Desplegable editable de proveedores (encabezado «N/A»); se puede escribir uno nuevo.
._construir_proveedores()	Método	Fila de proveedores (exterior y local).
._construir_precios()	Método	Fila de precios y monedas.
._construir_obsolescencia()	Método	Botones de opción de obsolescencia.
._construir_boton_parametros()	Método	Botón de parámetros eléctricos y técnicos avanzados.
._construir_archivos()	Método	Sección de archivos múltiples (imágenes, datasheets, tutoriales, esquemas).
._construir_boton_guardar()	Método	Botón de guardar o actualizar.
.abrir_dialogo_parametros_avanzados()	Método	Abre la ventana de parámetros ubicada en la clasificación que tiene el formulario en este momento.
._campos_dependientes()	Método	Todos los campos del formulario salvo los 4 combos de clasificación.
._poner_habilitado()	Método	Habilita o deshabilita un campo y lo atenúa (sus estilos propios no lo harían solos).
._combo_definido()	Método	True si el combo tiene una selección real (no vacío ni '-- Todas las ... --').
.actualizar_habilitacion_campos()	Método	Componente nuevo, en tres pasos: 1) solo los 4 combos de clasificación; 2) con los 4 definidos, se habilita el Chip Principal (con el marco parpadeando); 3) con el chip escrito, se habilita el resto de los campos.
._a_mayusculas()	Método	Convierte a mayúsculas lo que el usuario escribe en un QLineEdit (no lo que se carga por programa).
._a_mayusculas_texto()	Método	Igual que _a_mayusculas, para un QTextEdit. Solo actúa mientras el usuario escribe (el campo tiene el foco).
._activar_mayusculas()	Método	Chip, ubicación, proveedores, fabricante y descripción. (Tipo, categoría, subcategoría e ítem se eligen de listas.)
._valor_pendiente()	Método	True si el dato aún no se ingresó: vacío, 0, o 'N/A' (lo que guardan los combos de fabricante, interfaz, pines y encapsulado cuando no se eligió nada).
._sincronizar_monedas()	Método	La moneda se puede cambiar cuando se puede cambiar su precio.
._poner_texto_combo()	Método	Muestra `valor` en un combo editable (vacío = «N/A»); si no está en la lista, lo deja escrito.
._texto_clasificacion()	Método	Texto del combo de clasificación. Al crear, si coincide (sin distinguir mayúsculas) con una entrada del catálogo se usa la ortografía del catálogo, para no crear «arduino uno» junto a «Arduino Uno».

Elemento	Tipo	Qué hace
._leer_campos_formulario()	Método	Lee los widgets del formulario y devuelve el dict de datos del componente (sin las monedas).
._archivos_para_guardar()	Método	Archivos múltiples (imágenes, datasheets, tutoriales) como tuplas (tipo, nombre, ruta en el almacén).
._parametros_para_guardar()	Método	Parámetros eléctricos / especificaciones como lista de (clave, valor).
(otros métodos)	Métodos	__init__, init_ui, recibir_parametros_avanzados, generar_texto_qr_actual, actualizar_qr_en_vivo, formatear_descripcion_en_vivo, create_seccion_multianexo, actualizar_titulo_componente, _combos_clasificacion, _parpadeo_chip, _alternar_marco_chip, bloquear_campos_completos, toggle_habilitar_edicion, toggle_habilitar_quitar, agregar_imagen_desde_temporal, agregar_archivo_multianexo, ver_archivo_multianexo, quitar_archivo_multianexo, refrescar_combos_multianexo, on_imagen_seleccionada, actualizar_miniatura_superior, cargar_combos_enlazados, on_tipo_changed, on_cat_changed, on_subcat_changed, cargar_datos_existentes, guardar_todo_unificado

interfaz\dialogo_parametros.py

Capa: Interfaz. Líneas: 232.

Usa módulos del proyecto: db, interfaz.arbol_parametros, interfaz.widgets

Diálogo de parámetros eléctricos y técnicos de un componente.

A la izquierda, el árbol de la taxonomía con los parámetros disponibles en cada nodo;
a la derecha, la tabla de valores del componente. Un componente hereda los parámetros
de todos los niveles de su clasificación (ver db.parametros_heredados).

Elemento	Tipo	Qué hace
DialogoParametrosElectricos(QDialog)	Clase	Ventana de parámetros eléctricos y técnicos de un componente (edición o consulta).
.__init__()	Método	`ruta`: clasificación del componente (hasta 4 nombres). `solo_lectura`: consulta sin edición.
.mostrar_heredados()	Método	Debajo del árbol: todos los parámetros que corresponden al nodo elegido, con el nodo que los define.
.agregar_desde_nodo()	Método	Suma a la tabla los parámetros del nodo elegido que todavía no están (no toca los que ya tienen valor).
.obtener_plantilla_por_contexto()	Método	Nombres de los parámetros que corresponden a la clasificación del componente.
(otros métodos)	Métodos	_construir_panel_arbol, _construir_panel_tabla, agregar_fila, eliminar_fila, cargar_datos_iniciales, aceptar_parametros

interfaz\dialogo_catalogos.py

Capa: Interfaz. Líneas: 437.

Usa módulos del proyecto: db, interfaz.arbol_parametros, interfaz.widgets

Ventana para administrar el catálogo de clasificación y las listas de los desplegables.

No contiene consultas: todo el acceso a datos está en db.py.

Elemento	Tipo	Qué hace
_boton(texto, color, accion)	Función	Crea un botón con el estilo común de la ventana y su acción asociada.
_ayuda(texto)	Función	Crea una etiqueta de ayuda con texto gris y ajuste de línea.
_PestanaClasificacion(QWidget)	Clase	Árbol Tipo > Categoría > Subcategoría > Ítem.
.refrescar()	Método	Reconstruye el árbol conservando los nodos abiertos; `abrir` es una ruta que se despliega y selecciona.
(otros métodos)	Métodos	__init__, _rutas_abiertas, _ruta_elegida, agregar, renombrar, eliminar
_PestanaLista(QWidget)	Clase	Una lista de valores (fabricantes, interfaces, pines, encapsulados, monedas).
(otros métodos)	Métodos	__init__, refrescar, _valor_elegido, agregar, renombrar, eliminar
_PestanaParametros(QWidget)	Clase	Parámetros disponibles en cada nodo de la taxonomía (se heredan hacia abajo).
.refrescar()	Método	Se vuelve a leer todo (el catálogo pudo cambiar en la otra pestaña).
(otros métodos)	Métodos	__init__, mostrar_heredados, agregar, renombrar, quitar
_PestanaCotizaciones(QWidget)	Clase	Cotizaciones de las monedas respecto del USD (base del cálculo de capital).
.refrescar()	Método	Vuelve a leer las cotizaciones (las monedas pudieron cambiar en otra pestaña); con `conservar`, sin perder lo que se está escribiendo.

Elemento	Tipo	Qué hace
.guardar()	Método	Valida todo antes de guardar nada; solo escribe (y fecha) las cotizaciones que cambiaron.
(otros métodos)	Métodos	__init__, _texto
DialogoAdministrarCatalogos(QDialog)	Clase	Ventana «Listas y Catálogo»: reúne las pestañas de clasificación, listas, parámetros y cotizaciones.
(otros métodos)	Métodos	__init__

interfaz\dialogo_capital.py

Capa: Interfaz. Líneas: 157.

Usa módulos del proyecto: capital, interfaz.widgets

Ventana con el capital en existencia: subtotales por clasificación, detalle y exportación a Excel.

Elemento	Tipo	Qué hace
_celda(texto, derecha, alerta)	Función	Crea una celda de tabla no editable, con alineación a la derecha y color de alerta opcionales.
DialogoCapital(QDialog)	Clase	Ventana «Capital en existencia»: total, subtotales por clasificación, detalle y exportación a Excel.
(otros métodos)	Métodos	__init__, cargar, _llenar_arbol, _llenar_detalle, exportar

interfaz\dialogo_etiquetas.py

Capa: Interfaz. Líneas: 277.

Usa módulos del proyecto: almacenamiento, db, interfaz.visor_pdf

Diálogo de impresión de etiquetas y generación de su PDF.

Elemento	Tipo	Qué hace
DialogoImpresionEtiquetas(QDialog)	Clase	Ventana para configurar la hoja de etiquetas y generar su PDF.
._dibujar_guias_de_corte()	Método	Líneas a rayas por el punto medio de cada separación (y a media separación de los bordes exteriores) para indicar por dónde troquelar. Con separación 0 coinciden con el borde de las etiquetas.
(otros métodos)	Métodos	__init__, init_ui, generar_pdf

interfaz\editor_imagen.py

Capa: Interfaz. Líneas: 227.

Usa módulos del proyecto: almacenamiento

Editor y visor de imágenes (recorte y guardado).

Elemento	Tipo	Qué hace
pedir_destino_y_guardar(parent, imagen, sufijo, titulo_dialogo, chip)	Función	Pide al usuario la carpeta y el nombre, guarda `imagen` ahí y avisa. Devuelve la ruta, o None si canceló o falló. Es el único camino de guardado de recortes e imágenes (visor de PDF y visor de imagen).
DialogoEditorImagen(QDialog)	Clase	Visor de imagen con recorte. Guardar (región o entera) hace UNA sola cosa: pide la carpeta y el nombre, y escribe el archivo ahí. No registra la imagen en ningún componente: eso lo decide el usuario después. `dialogo_componente` (opcional) solo sirve para proponer un nombre de archivo con el chip.
._guardar()	Método	Guarda con `pedir_destino_y_guardar`; si se guardó, cierra el visor (si canceló, queda abierto).
(otros métodos)	Métodos	__init__, actualizar_pixmap_escalado, aumentar_zoom, reducir_zoom, reset_zoom, canvas_paint_event, canvas_mouse_press, canvas_mouse_move, canvas_mouse_release, guardar_recorte, guardar_completa

interfaz\visor_pdf.py

Capa: Interfaz. Líneas: 420.

Usa módulos del proyecto: almacenamiento, interfaz.editor_imagen, ocr

Visor de PDF con selección y recorte interactivo.

Elemento	Tipo	Qué hace
hacer_links_clicables_en_pdf(doc_pdf)	Función	Detecta direcciones web en el texto de un PDF y las convierte en vínculos clicables.
WidgetVisorPDFInteractivo(QWidget)	Clase	Widget que dibuja una página de PDF y permite seleccionar texto o recortar una región con el ratón.
._texto_por_ocr()	Método	Reconoce con Tesseract el texto de una región que es imagen. Cadena vacía si no hay o no hay OCR (en ese caso `aviso_ocr` explica por qué).
._texto_en_region()	Método	Texto de la región. Primero el texto contenido en el marco; si no hay, las palabras que el marco toca (un marco que corta una letra no debe dejar la selección vacía).
._renderizar()	Método	Imagen de la página (o de una región de ella), por defecto a doble resolución. Copia propia: no depende del buffer de PyMuPDF.
.imagen_recorte()	Método	Imagen de la región marcada en modo recorte, o None si no hay una región válida.
.imagen_pagina()	Método	Imagen de toda la página que muestra el visor.
(otros métodos)	Métodos	__init__, set_modo, actualizar_renderizado, paintEvent, mousePressEvent, mouseMoveEvent, mouseReleaseEvent, extraer_texto_seleccionado, contextMenuEvent
DialogoVisorPDF(QDialog)	Clase	Ventana del visor de PDF con zoom, páginas, Modo Texto y Modo Recorte.
(otros métodos)	Métodos	__init__, init_ui, cambiar_modo_visor, _chip_actual, guardar_recorte, guardar_imagen_entera, actualizar_visor_pagina, zoom_mas, zoom_menos, pagina_anterior, pagina_siguiente

Apéndice B. Código fuente completo

Se reproduce el texto íntegro de cada archivo, en el orden del Apéndice A. Es el mismo contenido que el archivo proyecto_completo_PyQt5.zip. Si hay diferencia, vale el del zip más reciente: este apéndice es una copia de lectura y no conviene copiar el código desde el manual (la copia desde un PDF o Word puede alterar sangrías y comillas).

app_inventario.py

```
"""Punto de entrada de la aplicación de inventario de componentes electrónicos.

Solo prepara carpetas y base de datos y abre la ventana principal.
"""
import sys

from PyQt5.QtWidgets import QApplication

import almacenamiento
import db
from config import asegurar_directorios
from interfaz.ventana_principal import AppInventario

# Crear las carpetas necesarias y luego la base de datos (mismo orden que antes)
asegurar_directorios()
db.inicializar_base_de_datos()

# Mover a la papelera los documentos del almacén que ningún componente usa (no borra nada).
# Un fallo aquí no debe impedir abrir la aplicación.
try:
    almacenamiento.limpiar_huerfanos(db.listar_rutas_archivos())
except Exception as e:
    print(f"No se pudo revisar el almacén de documentos: {e}")


if __name__ == "__main__":
    app = QApplication(sys.argv)
    # ---> APLICAR ESTILO GLOBAL DE NEGRITA <---
    app.setStyleSheet("QWidget { font-weight: bold; }")

    window = AppInventario()
    window.show()
    sys.exit(app.exec_())
```

config.py

```
"""Rutas de la aplicación.

Importar este módulo NO toca el disco. Las carpetas se crean al llamar a
asegurar_directorios(), que la aplicación invoca una sola vez al arrancar.
"""
import os
import sys

# Detectar si estamos ejecutando como un archivo compilado (.exe) o como script de Python
if getattr(sys, 'frozen', False):
    BASE_DIR = os.path.dirname(sys.executable)
else:
    BASE_DIR = os.path.dirname(os.path.abspath(__file__))

# La base de datos se guarda dentro de la misma carpeta de la aplicación
DB_DIR = os.path.join(BASE_DIR, "Base_Inventario")
DB_PATH = os.path.join(DB_DIR, "inventario_electronica.sqlite3")

ATTACHMENTS_DIR = os.path.join(BASE_DIR, "adjuntos")
TMP_DIR = os.path.join(ATTACHMENTS_DIR, "tmp")

# Aquí se escribe el PDF de etiquetas generado (siempre el mismo lugar, sin importar desde dónde se ejecute la
aplicación)
ETIQUETAS_DIR = os.path.join(BASE_DIR, "Etiquetas")


def asegurar_directorios():
    """Crea las carpetas que la aplicación espera encontrar (idempotente)."""
    os.makedirs(DB_DIR, exist_ok=True)
    os.makedirs(TMP_DIR, exist_ok=True)
    os.makedirs(ETIQUETAS_DIR, exist_ok=True)
```

catalogo.py

```
"""Datos de referencia iniciales: taxonomía, listas de valores y parámetros por clasificación.

Solo contiene constantes (sin lógica ni dependencias). La taxonomía y las listas
son la SEMILLA de las tablas catalogo_clasificacion, listas_valores y plantillas_parametros: se copian a
la base una sola vez; después se administran desde la aplicación.
"""

# ============================================================
# CATÁLOGO PREDEFINIDO CON TAXONOMÍA EXPERTA DE SISTEMAS EMBEBIDOS
# ============================================================
CATALOGO_PREDEFINIDO_4N = {
    "Sistemas Embebidos": {
        "Microcontroladores y Plataformas de Prototipado (MCU Boards)": {
            "Microcontroladores de Propósito General": ["Arduino Uno", "Arduino Nano", "Arduino Mega"],
            "SoCs de Conectividad / IoT": ["ESP32", "ESP8266", "Raspberry Pi Pico"],
        },
        "Computadoras de Placa Única (SBC - Single Board Computers)": {
            "Computadoras de Propósito General": ["Raspberry Pi 4", "Raspberry Pi 5", "Raspberry Pi 3 Model B+"],
            "Módulos y Accesorios SBC": ["Módulos de expansión", "Placas de interfaz para SBC"],
        },
    },
    "Componentes Pasivos": {
        "Resistivos": {
            "Resistencias Fijas": ["Película de Carbón", "Película Metálica", "Alambre Bobinado (Potencia)"],
            "Resistencias Variables": ["Potenciómetros rotativos", "Potenciómetros deslizantes", "Trimmers (Ajuste
fino)"],
            "Sensores Resistivos": ["Termistores NTC / PTC", "Fotorresistencias (LDR)", "FSR (Fuerza)"],
        },
        "Capacitivos (Condensadores)": {
            "Cerámicos": ["Cerámicos de disco", "Cerámicos multicapa (MLCC)"],
            "Electrolíticos": ["Electrolíticos de Aluminio", "Condensadores de Tántalo"],
            "Película Plástica": ["Poliéster (MKT)", "Polipropileno (MKP)"],
        },
        "Inductivos y Magnéticos": {
            "Inductores Fijos": ["Bobinas de núcleo de aire", "Inductores apantallados SMD"],
            "Transformadores": ["Transformadores reductores", "Transformadores toroidales"],
        },
    },
    "Componentes Activos": {
        "Semiconductores Discretos": {
            "Diodos": ["Diodos Rectificadores", "Diodos Zener", "Diodos Schottky", "Diodos LED", "Diodos DIAC",
"Diodos TVS"],
            "Transistores Bipolares (BJT)": ["NPN de uso general", "PNP de uso general", "Darlington"],
            "Transistores de Efecto de Campo": ["MOSFET Canal N", "MOSFET Canal P", "JFET"],
            "Tiristores": ["SCR", "TRIAC"],
        },
        "Circuitos Integrados (ICs)": {
            "Analógicos / Lineales": ["Amplificadores Operacionales", "Reguladores de Tensión Lineales",
"Comparadores"],
            "Digitales": ["Lógica Estándar", "Microcontroladores", "Memorias RAM/ROM"],
            "Mixtos y de Potencia": ["Conversores A/D y D/A", "Drivers para Motores", "Gestión de Energía"],
        },
    },
    "Electromecánicos y Conexión": {
        "Dispositivos de Conmutación y Protección": {
            "Interruptores y Pulsadores": ["Pulsadores táctiles", "Interruptores deslizantes/palanca"],
            "Relés": ["Relés electromecánicos", "Relés de estado sólido (SSR)"],
            "Protecciones": ["Fusibles de vidrio/cerámica", "Termistores PTC de protección"],
        },
        "Conectores y Bornes": {
            "Bornes y Pines": ["Headers de pines macho/hembra", "Borneras de tornillo"],
            "Conectores de Interfaz": ["Conectores USB", "Conectores Jack DC", "Conectores RJ45"],
        },
    },
    "Optoelectrónica y Sensores": {
        "Displays y Visualización": {
            "Displays Numéricos": ["Displays de 7 segmentos", "Matrices LED"],
            "Pantallas": ["Pantallas LCD I2C", "Pantallas OLED gráficas"],
        },
        "Sensores no resistivos": {
            "Magnéticos y Posición": ["Sensores de efecto Hall", "Encoders rotativos"],
            "Físicos y Entorno": ["Micrófonos", "Sensores ultrasónicos", "IMUs", "Sensores ToF"],
        },
    },
}
# TAXONOMIA_GENERAL_INVENTARIO

# ============================================================
# LISTAS DE VALORES (semilla inicial). Al crear la base se copian a la tabla
```

```
# listas_valores; desde ahí se administran desde la aplicación.
# ====================================================================
LISTAS_PREDEFINIDAS = {
    "fabricante": [
        "N/A", "Samsung Electronics", "Intel Corporation", "Texas Instruments (TI)", "STMicroelectronics",
        "Infineon Technologies", "NXP Semiconductors", "Analog Devices (ADI)", "Microchip Technology",
        "ON Semiconductor (onsemi)", "Renesas Electronics", "Broadcom", "Qualcomm", "MediaTek",
        "TSMC", "GlobalFoundries", "NVIDIA", "AMD", "Raspberry Pi Foundation", "Arduino", "Espressif Systems",
    ],
    "interfaz": [
        "N/A", "I2C", "SPI", "UART / USART", "USB", "Ethernet (RJ45)", "CAN Bus", "1-Wire",
        "RS-232", "RS-485", "Wi-Fi", "Bluetooth / BLE", "Zigbee", "LoRa", "PCIe", "HDMI", "MIPI DSI/CSI",
    ],
    "pines": [
        "N/A", "VCC", "VDD", "+5V", "+3.3V", "GND", "VSS", "VEE", "VREF",
        "SDA", "SCL", "MOSI", "MISO", "SCLK / SCK", "CS / SS", "TX", "RX", "CANH", "CANL",
        "D+ (DP)", "D- (DM)", "EN / CE", "RST / RESET", "INT / IRQ", "CLK / OSC", "MODE / AD0",
        "AIN / VIN", "AOUT / VOUT", "PWM", "GPIO", "Colector (C)", "Emisor (E)", "Base (B)",
        "Drenador (D)", "Surtidor (S)", "Puerta / Compuerta (G)", "Ánodo (A)", "Cátodo (K)", "IN", "OUT",
    ],
    "encapsulado": [
        "N/A", "DIP-8", "DIP-14", "DIP-16", "DIP-18", "DIP-20", "DIP-24", "DIP-28", "DIP-40",
        "SIP-3", "SIP-5", "SIP-7", "SIP-9", "TO-92", "TO-126", "TO-220", "TO-247",
        "SOIC-8", "SOIC-14", "SOIC-16", "SOIC-20", "SOIC-24", "SOT-23", "SOT-89", "SOT-223",
        "SSOP-16", "SSOP-20", "TSSOP-8", "TSSOP-16", "TSSOP-20", "TQFP-32", "TQFP-44",
        "TQFP-64", "LQFP-100", "QFN-16", "QFN-24", "QFN-32", "QFN-48", "BGA-144", "BGA-256",
        "BGA-400", "AXIAL-0.3", "AXIAL-0.4", "AXIAL-0.6", "RADIAL-2.5", "RADIAL-5.0",
        "RADIAL-7.5", "SMD-0402", "SMD-0603", "SMD-0805", "SMD-1206", "SMD-1210", "LED-3mm", "LED-5mm", "PLCC-2",
    ],
    "moneda": ["USD", "UYU", "BRL", "ARS"],
    "proveedor_local": ["N/A"],
    "proveedor_exterior": ["N/A"],
}

# ====================================================================
# PARÁMETROS POR CLASIFICACIÓN (semilla inicial de la tabla plantillas_parametros)
#
# Cada clave es un camino de la taxonomía: (Tipo,), (Tipo, Categoría), (Tipo, Categoría, Subcategoría)
# o hasta el Ítem. Un componente HEREDA los parámetros de todos los niveles por los que pasa su
# clasificación y suma los de su propio nivel: lo común se define arriba, lo particular abajo.
# ====================================================================
_EMB = "Sistemas Embebidos"
_MCU = (_EMB, "Microcontroladores y Plataformas de Prototipado (MCU Boards)")
_SBC = (_EMB, "Computadoras de Placa Única (SBC - Single Board Computers)")
_PAS = "Componentes Pasivos"
_RES = (_PAS, "Resistivos")
_CAP = (_PAS, "Capacitivos (Condensadores)")
_IND = (_PAS, "Inductivos y Magnéticos")
_ACT = "Componentes Activos"
_DIS = (_ACT, "Semiconductores Discretos")
_CIS = (_ACT, "Circuitos Integrados (ICs)")
_ELM = "Electromecánicos y Conexión"
_CON = (_ELM, "Dispositivos de Conmutación y Protección")
_CNX = (_ELM, "Conectores y Bornes")
_OPT = "Optoelectrónica y Sensores"
_DSP = (_OPT, "Displays y Visualización")
_SEN = (_OPT, "Sensores no resistivos")

PARAMETROS_PREDEFINIDOS = {
    # --- Sistemas Embebidos ---
    (_EMB,): ["Tensión de Operación (V)", "Corriente de Consumo (mA)", "Temperatura de Operación (°C)"],
    _MCU: ["Microcontrolador / SoC", "Frecuencia de Reloj (MHz)", "Memoria Flash", "Memoria RAM",
           "Tensión Lógica de E/S (V)", "Corriente por GPIO (mA)", "Cantidad de Pines GPIO", "Interfaces de
Comunicación"],
    _MCU + ("Microcontroladores de Propósito General",): ["Entradas Analógicas (ADC)", "Salidas PWM", "Conector de
Programación / USB"],
    _MCU + ("SoCs de Conectividad / IoT",): ["Conectividad Inalámbrica (Wi-Fi/BLE)", "Entradas Analógicas (ADC)",
"Salidas PWM", "Consumo en Reposo (µA)"],
    _SBC: ["Procesador (CPU)", "Frecuencia de CPU (GHz)", "Memoria RAM (GB)", "Almacenamiento (SD/eMMC)",
           "Alimentación de Entrada (V/A)", "Tensión de GPIO (V)", "Interfaces de Red (Wi-Fi/Ethernet)", "Puertos
USB / Video"],
    _SBC + ("Computadoras de Propósito General",): ["Sistema Operativo Soportado", "Consumo Máximo (W)"],
    _SBC + ("Módulos y Accesorios SBC",): ["Compatibilidad (Modelos de SBC)", "Interfaz de Conexión
(GPIO/CSI/DSI/USB)"],

    # --- Componentes Pasivos ---
    (_PAS,): ["Tolerancia (%)", "Temperatura de Operación (°C)"],
    _RES: ["Valor Nominal (Ω)", "Potencia Nominal (W)"],
    _RES + ("Resistencias Fijas",): ["Tecnología / Material", "Coeficiente de Temperatura (ppm/°C)", "Tensión
Máxima de Trabajo (V)"],
    _RES + ("Resistencias Variables",): ["Curva (Lineal/Logarítmica)", "Cantidad de Vueltas", "Tipo de Eje /
Montaje"],
```

```
    _RES + ("Sensores Resistivos",): ["Rango de Medición", "Valor Beta / Sensibilidad", "Tiempo de Respuesta"],
    _CAP: ["Capacitancia (F/µF/nF/pF)", "Tensión Máxima Vdc (V)"],
    _CAP + ("Cerámicos",): ["Dieléctrico (C0G/X7R/Y5V)", "Tamaño / Paso"],
    _CAP + ("Electrolíticos",): ["ESR (Ω)", "Corriente de Ripple (mA)", "Vida Útil (horas)", "Polaridad"],
    _CAP + ("Película Plástica",): ["Dieléctrico (MKT/MKP)", "Tensión AC Máxima (V)", "Paso (mm)"],
    _IND + ("Inductores Fijos",): ["Inductancia (H/mH/µH)", "Corriente de Saturación Isat (A)", "Corriente Nominal
(A)",
                                   "Resistencia DCR (Ω)", "Frecuencia de Resonancia (MHz)"],
    _IND + ("Transformadores",): ["Tensión Primaria (V)", "Tensión Secundaria (V)", "Potencia (VA)", "Relación de
Transformación", "Frecuencia (Hz)"],

    # --- Componentes Activos ---
    (_ACT,): ["Temperatura de Operación (°C)"],
    _DIS: ["Potencia Máxima Pd (W)"],
    _DIS + ("Diodos",): ["Corriente Directa If (A)", "Tensión Inversa Vrrm (V)", "Caída Directa Vf (V)", "Tensión
Zener Vz (V)",
                         "Tiempo de Recuperación trr (ns)", "Color / Longitud de Onda (LED)"],
    _DIS + ("Transistores Bipolares (BJT)",): ["Tensión Vceo (V)", "Corriente de Colector Ic (A)", "Ganancia hFE
(β)",
                                               "Frecuencia de Transición ft (MHz)", "Polaridad (NPN/PNP)"],
    _DIS + ("Transistores de Efecto de Campo",): ["Tensión Vds (V)", "Corriente de Drenador Id (A)", "Resistencia
Rds(on) (Ω)",
                                                  "Tensión Umbral Vgs(th) (V)", "Capacitancia de Entrada Ciss
(pF)", "Canal (N/P)"],
    _DIS + ("Tiristores",): ["Tensión de Bloqueo Vdrm (V)", "Corriente Nominal It (A)", "Corriente de Disparo Igt
(mA)", "Corriente de Mantenimiento Ih (mA)"],
    _CIS: ["Tensión de Alimentación Vcc (V)", "Corriente de Reposo Icc (mA)"],
    _CIS + ("Analógicos / Lineales",): ["Ancho de Banda (MHz)", "Slew Rate (V/µs)", "Número de Canales", "Tensión
de Salida (V)", "Corriente de Salida Máxima (mA)"],
    _CIS + ("Digitales",): ["Familia Lógica (TTL/CMOS)", "Frecuencia Máxima (MHz)", "Retardo de Propagación (ns)",
"Capacidad de Memoria"],
    _CIS + ("Mixtos y de Potencia",): ["Resolución (bits)", "Tasa de Muestreo (SPS)", "Corriente de Salida Máxima
(A)",
                                       "Frecuencia de Conmutación (kHz)", "Eficiencia (%)"],

    # --- Electromecánicos y Conexión ---
    (_ELM,): ["Temperatura de Operación (°C)", "Vida Útil Estimada (Ciclos)"],
    _CON: ["Tensión Máxima (VAC/VDC)", "Corriente Máxima (A)"],
    _CON + ("Interruptores y Pulsadores",): ["Configuración de Contactos (SPST/SPDT)", "Tipo de Accionamiento",
"Resistencia de Contacto (mΩ)"],
    _CON + ("Relés",): ["Tensión de Bobina Vcoil (V)", "Resistencia de Bobina (Ω)", "Tipo de Contacto
(SPDT/DPDT)",
                        "Tensión de Control (Entrada VDC)", "Tipo de Conmutación (Zero-Cross/Random)"],
    _CON + ("Protecciones",): ["Capacidad de Ruptura (A)", "Tiempo de Respuesta (Lento/Rápido)"],
    _CNX: ["Cantidad de Pines / Vías", "Paso (mm)", "Corriente por Contacto (A)", "Tensión Nominal (V)"],
    _CNX + ("Bornes y Pines",): ["Género (Macho/Hembra)", "Tipo de Montaje (THT/SMD)"],
    _CNX + ("Conectores de Interfaz",): ["Tipo de Conector (USB-A/B/C, Jack, RJ45)", "Tipo de Montaje (THT/SMD)",
"Diámetro / Polaridad (Jack DC)"],

    # --- Optoelectrónica y Sensores ---
    (_OPT,): ["Tensión de Alimentación Vcc (V)", "Temperatura de Operación (°C)"],
    _DSP: ["Consumo de Corriente (mA)"],
    _DSP + ("Displays Numéricos",): ["Cantidad de Dígitos", "Tipo (Ánodo/Cátodo Común)", "Color", "Tamaño del
Dígito (mm)"],
    _DSP + ("Pantallas",): ["Tamaño (pulgadas)", "Resolución (px)", "Controlador (SSD1306/HD44780)", "Interfaz
(I2C/SPI/Paralelo)", "Color / Retroiluminación"],
    _SEN: ["Interfaz (I2C/SPI/Analógica)", "Rango de Medición", "Resolución / Precisión", "Consumo de Corriente
(mA)"],
    _SEN + ("Magnéticos y Posición",): ["Tipo de Salida (Digital/Analógica)", "Pulsos por Vuelta (Encoders)",
"Sensibilidad (mV/mT)"],
    _SEN + ("Físicos y Entorno",): ["Magnitud Medida", "Tiempo de Respuesta", "Frecuencia de Muestreo (Hz)"],
}
```

db.py

```
"""Capa de acceso a datos (SQLite).

Todas las consultas de la aplicación viven aquí: la interfaz no abre
conexiones propias, llama a las funciones de este módulo.

Importar este módulo no toca el disco. La base se crea al llamar a
inicializar_base_de_datos().

Convención de filtros: un filtro en None (o vacío) significa "sin filtrar".
La interfaz es quien traduce sus textos comodín ("-- Todos los Tipos --", etc.)
a None antes de llamar.
"""
import math
import sqlite3
from contextlib import contextmanager
from datetime import date

import catalogo
from config import DB_PATH
from utilidades import limpiar_prefijo


@contextmanager
def conexion():
    """Abre una conexión con las claves foráneas activadas y la cierra al salir.

    PRAGMA foreign_keys se aplica por conexión en SQLite; sin esta línea el
    ON DELETE CASCADE declarado en el esquema no se ejecuta.
    """
    conn = sqlite3.connect(DB_PATH)
    try:
        conn.execute("PRAGMA foreign_keys = ON")
        yield conn
    finally:
        conn.close()


# ---------------------------------------------------------------------
# Esquema
# ---------------------------------------------------------------------
def inicializar_base_de_datos():
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS productos_fabricantes (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            nombre TEXT,
            producto_id TEXT,
            tipo_componente TEXT,
            categoria TEXT,
            subcategoria TEXT,
            item_especifico TEXT,
            descripcion TEXT,
            fabricante TEXT,
            chip_principal TEXT,
            interfaz_comunicacion TEXT,
            pines_relevantes TEXT,
            encapsulado TEXT,
            stock INTEGER,
            precio REAL,
            ubicacion TEXT,
            obsolescencia TEXT,
            proveedor_exterior TEXT,
            precio_exterior REAL,
            moneda_exterior TEXT,   -- Asegúrate de añadir esta
            proveedor_local TEXT,
            precio_local REAL,
            moneda_local TEXT       -- Y esta
        )
    """)
    conn.commit()

    cursor.execute("PRAGMA table_info(productos_fabricantes)")
    columnas = [col[1] for col in cursor.fetchall()]
    if "qr_path" not in columnas:
        cursor.execute("ALTER TABLE productos_fabricantes ADD COLUMN qr_path TEXT")
    if "precio_exterior" not in columnas:
        cursor.execute("ALTER TABLE productos_fabricantes ADD COLUMN precio_exterior REAL DEFAULT 0.0")
```

```
    if "proveedor_exterior" not in columnas:
        cursor.execute("ALTER TABLE productos_fabricantes ADD COLUMN proveedor_exterior TEXT")
    if "precio_local" not in columnas:
        cursor.execute("ALTER TABLE productos_fabricantes ADD COLUMN precio_local REAL DEFAULT 0.0")
    if "proveedor_local" not in columnas:
        cursor.execute("ALTER TABLE productos_fabricantes ADD COLUMN proveedor_local TEXT")

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS archivos_componentes (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            componente_id INTEGER NOT NULL,
            tipo_archivo TEXT NOT NULL,
            nombre_amigable TEXT NOT NULL,
            ruta_archivo TEXT NOT NULL,
            FOREIGN KEY (componente_id) REFERENCES productos_fabricantes (id) ON DELETE CASCADE
        )
    """)
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS parametros_electricos (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            componente_id INTEGER NOT NULL,
            parametro_clave TEXT NOT NULL,
            parametro_valor TEXT NOT NULL,
            FOREIGN KEY (componente_id) REFERENCES productos_fabricantes (id) ON DELETE CASCADE
        )
    """)
    # Catálogo de clasificación: una fila por camino tipo > categoría > subcategoría > ítem.
    # Un nivel sin definir se guarda como '' (así un tipo recién creado existe aunque no tenga hijos).
    # COLLATE NOCASE: "Arduino Uno" y "ARDUINO UNO" son la misma entrada.
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS catalogo_clasificacion (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            tipo TEXT NOT NULL COLLATE NOCASE,
            categoria TEXT NOT NULL DEFAULT '' COLLATE NOCASE,
            subcategoria TEXT NOT NULL DEFAULT '' COLLATE NOCASE,
            item TEXT NOT NULL DEFAULT '' COLLATE NOCASE,
            UNIQUE (tipo, categoria, subcategoria, item)
        )
    """)
    # Listas de los desplegables (fabricante, interfaz, pines, encapsulado, moneda).
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS listas_valores (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            lista TEXT NOT NULL,
            valor TEXT NOT NULL COLLATE NOCASE,
            orden INTEGER NOT NULL DEFAULT 0,
            UNIQUE (lista, valor)
        )
    """)
    # Parámetros disponibles por nodo de la taxonomía (se heredan hacia abajo).
    # El nodo es el camino tipo > categoría > subcategoría > ítem, con '' en los niveles que no aplican.
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS plantillas_parametros (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            tipo TEXT NOT NULL COLLATE NOCASE,
            categoria TEXT NOT NULL DEFAULT '' COLLATE NOCASE,
            subcategoria TEXT NOT NULL DEFAULT '' COLLATE NOCASE,
            item TEXT NOT NULL DEFAULT '' COLLATE NOCASE,
            parametro TEXT NOT NULL COLLATE NOCASE,
            orden INTEGER NOT NULL DEFAULT 0,
            UNIQUE (tipo, categoria, subcategoria, item, parametro)
        )
    """)
    # Cotizaciones: cuántas unidades de cada moneda equivalen a 1 USD (la moneda base). Las carga el usuario.
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS cotizaciones (
            moneda TEXT PRIMARY KEY COLLATE NOCASE,
            por_usd REAL,
            fecha TEXT
        )
    """)
    conn.commit()
    _sembrar(conn)
    conn.commit()
    conn.close()


# ---------------------------------------------------------------------
# Siembra inicial de las tablas de catálogo y listas
# ---------------------------------------------------------------------
NIVELES = ("tipo", "categoria", "subcategoria", "item")                     # columnas del catálogo
```

```
_COL_COMPONENTE = ("tipo_componente", "categoria", "subcategoria", "item_especifico")  # mismas, en
productos_fabricantes

# lista -> (columnas de productos_fabricantes que la usan, ¿guarda varios valores separados por ", "?)
LISTAS = {
    "fabricante": (("fabricante",), False),
    "interfaz": (("interfaz_comunicacion",), True),
    "pines": (("pines_relevantes",), True),
    "encapsulado": (("encapsulado",), True),
    "moneda": (("moneda_exterior", "moneda_local"), False),
    "proveedor_local": (("proveedor_local",), False),
    "proveedor_exterior": (("proveedor_exterior",), False),
}
SIN_DATO = "N/A"
MONEDA_BASE = "USD"   # las cotizaciones se expresan como unidades de cada moneda por 1 USD
# listas que siempre empiezan con «N/A» (sin dato)
LISTAS_CON_SIN_DATO = ("fabricante", "interfaz", "pines", "encapsulado", "proveedor_local", "proveedor_exterior")


def _ruta(valores):
    """Normaliza un camino de hasta 4 niveles a una tupla de 4 textos ('' = nivel sin definir).

    Devuelve None si no hay tipo. Un nivel vacío o comodín ("-- Todos --") corta el camino.
    """
    ruta = []
    for i in range(4):
        v = str(valores[i]).strip() if i < len(valores) and valores[i] is not None else ""
        if v.startswith("--"):
            v = ""
        if i == 0 and v:
            v = limpiar_prefijo(v)
        if not v:
            break
        ruta.append(v)
    if not ruta:
        return None
    return tuple(ruta) + ("",) * (4 - len(ruta))


def _insertar_ruta(conn, ruta):
    """Agrega un camino al catálogo (si no existe) y quita los marcadores de nivel que ya tienen hijos."""
    n = sum(1 for x in ruta if x)
    if n < 4:   # un camino corto solo hace falta si todavía no tiene nada debajo
        cond = " AND ".join(f"{c}=?" for c in NIVELES[:n]) + f" AND {NIVELES[n]}<>''"
        if conn.execute(f"SELECT 1 FROM catalogo_clasificacion WHERE {cond}", ruta[:n]).fetchone():
            return
    conn.execute("INSERT OR IGNORE INTO catalogo_clasificacion (tipo, categoria, subcategoria, item) VALUES
(?,?,?,?)", ruta)
    for k in range(1, n):
        ancestro = ruta[:k] + ("",) * (4 - k)
        conn.execute("DELETE FROM catalogo_clasificacion WHERE tipo=? AND categoria=? AND subcategoria=? AND
item=?", ancestro)


def _valores_de(texto, multiple):
    """Valores sueltos de un campo de componente (los de selección múltiple se guardan como 'A, B, C')."""
    texto = str(texto or "").strip()
    partes = [p.strip() for p in texto.split(",")] if multiple else [texto]
    return [p for p in partes if p and p.upper() != SIN_DATO and not p.startswith("--")]


def _agregar_valor(conn, lista, valor):
    conn.execute("""INSERT OR IGNORE INTO listas_valores (lista, valor, orden)
                    VALUES (?, ?, COALESCE((SELECT MAX(orden) FROM listas_valores WHERE lista=?), 0) + 1)""",
                 (lista, valor, lista))


def _asegurar_sin_dato(conn):
    """«N/A» encabeza las listas que lo llevan (también en bases creadas antes de que existiera en ellas)."""
    for lista in LISTAS_CON_SIN_DATO:
        conn.execute("INSERT OR IGNORE INTO listas_valores (lista, valor, orden) VALUES (?, ?, 0)", (lista,
SIN_DATO))


def _sembrar(conn):
    """Carga el contenido inicial. Cada tabla se siembra solo si está vacía, así lo que el usuario
    borre o renombre después no vuelve a aparecer. También incorpora lo que ya hay en los componentes."""
    if conn.execute("SELECT COUNT(*) FROM catalogo_clasificacion").fetchone()[0] == 0:
        rutas = []
        for tipo, cats in catalogo.CATALOGO_PREDEFINIDO_4N.items():
            for cat, subs in cats.items():
                for sub, items in subs.items():
```

```
                    for item in items:
                        rutas.append((tipo, cat, sub, item))
        for fila in conn.execute("SELECT DISTINCT tipo_componente, categoria, subcategoria, item_especifico FROM
productos_fabricantes"):
            rutas.append(fila)
        for r in rutas:
            ruta = _ruta(r)
            if ruta:
                _insertar_ruta(conn, ruta)

    for lista, (columnas, multiple) in LISTAS.items():
        if conn.execute("SELECT COUNT(*) FROM listas_valores WHERE lista=?", (lista,)).fetchone()[0] == 0:
            for v in catalogo.LISTAS_PREDEFINIDAS.get(lista, []):
                _agregar_valor(conn, lista, v)
            for col in columnas:
                for (texto,) in conn.execute(f"SELECT DISTINCT {col} FROM productos_fabricantes").fetchall():
                    for v in _valores_de(texto, multiple):
                        _agregar_valor(conn, lista, v)
    _asegurar_sin_dato(conn)
    conn.execute("INSERT OR IGNORE INTO cotizaciones (moneda, por_usd, fecha) VALUES (?, 1.0, NULL)",
(MONEDA_BASE,))

    if conn.execute("SELECT COUNT(*) FROM plantillas_parametros").fetchone()[0] == 0:
        for ruta, parametros in catalogo.PARAMETROS_PREDEFINIDOS.items():
            nodo = _ruta(ruta)
            for orden, parametro in enumerate(parametros, start=1):
                conn.execute("INSERT OR IGNORE INTO plantillas_parametros (tipo, categoria, subcategoria, item,
parametro, orden) VALUES (?,?,?,?,?,?)",
                             nodo + (parametro, orden))


# ---------------------------------------------------------------------
# Consultas para los desplegables en cascada (leen el catálogo)
# ---------------------------------------------------------------------
def _listar_nivel(nivel, previos):
    """Nombres distintos del nivel `nivel` (0..3). `previos`: valores de los niveles anteriores; None/'' =
cualquiera."""
    col = NIVELES[nivel]
    donde, params = [f"{col}<>''"], []
    for c, v in zip(NIVELES[:nivel], previos):
        if v:
            donde.append(f"{c}=?")
            params.append(v)
    with conexion() as conn:
        cursor = conn.execute(f"SELECT DISTINCT {col} FROM catalogo_clasificacion WHERE {' AND '.join(donde)}
ORDER BY {col}", params)
        return [r[0] for r in cursor.fetchall()]


def listar_tipos():
    return _listar_nivel(0, ())


def listar_categorias(tipo=None):
    """Categorías del catálogo; con `tipo` solo las de ese tipo."""
    return _listar_nivel(1, (tipo,))


def listar_subcategorias(tipo=None, cat=None):
    """Subcategorías del catálogo; cada filtro presente se aplica."""
    return _listar_nivel(2, (tipo, cat))


def listar_items(tipo=None, cat=None, subcat=None):
    """Ítems del catálogo; cada filtro presente se aplica."""
    return _listar_nivel(3, (tipo, cat, subcat))


# ---------------------------------------------------------------------
# Administración del catálogo de clasificación
# ---------------------------------------------------------------------
def _validar_nombre(nombre):
    nombre = str(nombre or "").strip()
    if not nombre:
        raise ValueError("El nombre no puede estar vacío.")
    if nombre.startswith("--"):
        raise ValueError("El nombre no puede empezar con «--».")
    return nombre


def _condicion_ruta(ruta, columnas):
    """('c1=? AND c2=?', params) para los niveles dados en `ruta`."""
```

```
    return " AND ".join(f"{c}=?" for c in columnas[:len(ruta)]), list(ruta)


def listar_catalogo():
    """Todos los caminos del catálogo: lista de (tipo, categoría, subcategoría, ítem); '' = nivel sin definir."""
    with conexion() as conn:
        return conn.execute("SELECT tipo, categoria, subcategoria, item FROM catalogo_clasificacion ORDER BY tipo,
categoria, subcategoria, item").fetchall()


def agregar_catalogo(ruta):
    """Agrega un nodo al catálogo. `ruta`: nombres desde el tipo hasta el nivel nuevo (1 a 4).
    Devuelve la ruta tal como quedó (el tipo sin número de prefijo). Lanza ValueError si no es válido."""
    ruta = tuple(_validar_nombre(x) for x in ruta)
    if not 1 <= len(ruta) <= 4:
        raise ValueError("La clasificación tiene como máximo 4 niveles.")
    completa = _ruta(ruta)
    with conexion() as conn:
        try:
            _insertar_ruta(conn, completa)
            conn.commit()
        except Exception:
            conn.rollback()
            raise
    return tuple(x for x in completa if x)


def contar_componentes_catalogo(ruta):
    """Cuántos componentes están clasificados bajo ese nodo."""
    cond, params = _condicion_ruta(ruta, _COL_COMPONENTE)
    with conexion() as conn:
        return conn.execute(f"SELECT COUNT(*) FROM productos_fabricantes WHERE {cond}", params).fetchone()[0]


def renombrar_catalogo(ruta, nuevo):
    """Cambia el nombre del último nivel de `ruta`, en el catálogo y en los componentes que lo usan.

    Si ya existe un nodo con ese nombre bajo el mismo padre, los dos se unen.
    Devuelve la ruta nueva del nodo.
    """
    ruta = tuple(ruta)
    n = len(ruta) - 1
    nuevo = _validar_nombre(limpiar_prefijo(nuevo) if n == 0 else nuevo)
    viejo = ruta[n]
    cond_cat, params = _condicion_ruta(ruta, NIVELES)
    cond_comp, _ = _condicion_ruta(ruta, _COL_COMPONENTE)
    with conexion() as conn:
        try:
            if nuevo.lower() == viejo.lower():
                # Solo cambia mayúsculas/minúsculas: no puede haber choque, es la misma entrada.
                conn.execute(f"UPDATE catalogo_clasificacion SET {NIVELES[n]}=? WHERE {cond_cat}", [nuevo] +
params)
            else:
                conn.execute(f"UPDATE OR IGNORE catalogo_clasificacion SET {NIVELES[n]}=? WHERE {cond_cat}",
[nuevo] + params)
                # lo que no pudo cambiar chocó con una entrada existente: ya está representado por ella
                conn.execute(f"DELETE FROM catalogo_clasificacion WHERE {cond_cat}", params)
            # los parámetros del nodo (y de los que cuelgan de él) lo acompañan
            if nuevo.lower() == viejo.lower():
                conn.execute(f"UPDATE plantillas_parametros SET {NIVELES[n]}=? WHERE {cond_cat}", [nuevo] +
params)
            else:
                conn.execute(f"UPDATE OR IGNORE plantillas_parametros SET {NIVELES[n]}=? WHERE {cond_cat}",
[nuevo] + params)
                conn.execute(f"DELETE FROM plantillas_parametros WHERE {cond_cat}", params)
            if n >= 1:   # 'nombre' repite el último nivel definido; se mantiene al día
                conn.execute(f"UPDATE productos_fabricantes SET nombre=?, producto_id=? WHERE nombre=? AND
{cond_comp}",
                             [nuevo, nuevo, viejo] + params)
            conn.execute(f"UPDATE productos_fabricantes SET {_COL_COMPONENTE[n]}=? WHERE {cond_comp}", [nuevo] +
params)
            conn.commit()
        except Exception:
            conn.rollback()
            raise
    return ruta[:-1] + (nuevo,)


def eliminar_catalogo(ruta):
    """Quita el nodo (y todo lo que cuelga de él, con sus parámetros) del catálogo. Los componentes conservan su
clasificación escrita."""
    ruta = tuple(ruta)
```

```
    cond, params = _condicion_ruta(ruta, NIVELES)
    with conexion() as conn:
        try:
            conn.execute(f"DELETE FROM catalogo_clasificacion WHERE {cond}", params)
            conn.execute(f"DELETE FROM plantillas_parametros WHERE {cond}", params)
            if len(ruta) > 1:   # si el padre se quedó sin hijos, sigue existiendo como nodo
                _insertar_ruta(conn, _ruta(ruta[:-1]))
            conn.commit()
        except Exception:
            conn.rollback()
            raise


# ---------------------------------------------------------------------
# Parámetros por clasificación (con herencia)
# ---------------------------------------------------------------------
def ruta_de_clasificacion(tipo, cat, subcat, item):
    """Camino de un componente como tupla de 0 a 4 nombres (se corta en el primer nivel vacío o comodín «-- …
--»)."""
    ruta = _ruta((tipo, cat, subcat, item))
    return tuple(x for x in ruta if x) if ruta else ()


def _nodo(ruta):
    """Ruta de 1 a 4 nombres -> tupla de 4 con '' en los niveles que faltan."""
    ruta = tuple(str(x).strip() for x in ruta)
    if not 1 <= len(ruta) <= 4 or not all(ruta):
        raise ValueError("Elija primero un nodo de la clasificación.")
    return ruta + ("",) * (4 - len(ruta))


def listar_plantillas():
    """Todos los parámetros definidos: lista de (ruta del nodo, parámetro), en el orden de cada nodo."""
    with conexion() as conn:
        filas = conn.execute("SELECT tipo, categoria, subcategoria, item, parametro FROM plantillas_parametros
ORDER BY tipo, categoria, subcategoria, item, orden, id").fetchall()
    return [(tuple(x for x in f[:4] if x), f[4]) for f in filas]


def listar_parametros_plantilla(ruta):
    """Parámetros propios de ese nodo (sin los heredados)."""
    with conexion() as conn:
        return [r[0] for r in conn.execute(
            "SELECT parametro FROM plantillas_parametros WHERE tipo=? AND categoria=? AND subcategoria=? AND
item=? ORDER BY orden, id", _nodo(ruta))]


def parametros_heredados(ruta):
    """Parámetros que corresponden a un componente de esa clasificación: [(parámetro, ruta del nodo que lo
define)].

    Van de lo general a lo particular (primero los del tipo, al final los del último nivel) y un
    nombre repetido se muestra una sola vez, con el nodo más general que lo define.
    """
    ruta = tuple(ruta)
    resultado, vistos = [], set()
    for k in range(1, len(ruta) + 1):
        for parametro in listar_parametros_plantilla(ruta[:k]):
            if parametro.lower() not in vistos:
                vistos.add(parametro.lower())
                resultado.append((parametro, ruta[:k]))
    return resultado


def agregar_parametro_plantilla(ruta, parametro):
    """Agrega un parámetro al nodo y devuelve el nombre tal como quedó (si ya existía, el existente)."""
    nodo = _nodo(ruta)
    parametro = _validar_nombre(parametro)
    with conexion() as conn:
        conn.execute("""INSERT OR IGNORE INTO plantillas_parametros (tipo, categoria, subcategoria, item,
parametro, orden)
                        VALUES (?,?,?,?,?, COALESCE((SELECT MAX(orden) FROM plantillas_parametros WHERE tipo=? AND
categoria=? AND subcategoria=? AND item=?), 0) + 1)""",
                     nodo + (parametro,) + nodo)
        conn.commit()
        return conn.execute("SELECT parametro FROM plantillas_parametros WHERE tipo=? AND categoria=? AND
subcategoria=? AND item=? AND parametro=?", nodo + (parametro,)).fetchone()[0]


def renombrar_parametro_plantilla(ruta, viejo, nuevo):
    """Cambia el nombre de un parámetro de ese nodo. No toca los valores ya guardados en los componentes."""
    nodo = _nodo(ruta)
```

```
    nuevo = _validar_nombre(nuevo)
    with conexion() as conn:
        try:
            conn.execute("UPDATE plantillas_parametros SET parametro=? WHERE tipo=? AND categoria=? AND
subcategoria=? AND item=? AND parametro=?", (nuevo,) + nodo + (viejo,))
            conn.commit()
        except sqlite3.IntegrityError:
            conn.rollback()
            raise ValueError(f"«{nuevo}» ya existe en ese nodo.")


def eliminar_parametro_plantilla(ruta, parametro):
    """Quita un parámetro de ese nodo. Los valores ya guardados en los componentes se conservan."""
    nodo = _nodo(ruta)
    with conexion() as conn:
        conn.execute("DELETE FROM plantillas_parametros WHERE tipo=? AND categoria=? AND subcategoria=? AND item=?
AND parametro=?", nodo + (parametro,))
        conn.commit()


# ---------------------------------------------------------------------
# Listas de valores de los desplegables (fabricante, interfaz, pines, encapsulado, moneda)
# ---------------------------------------------------------------------
def listar_valores(lista):
    """Valores de la lista, en el orden en que se cargaron."""
    with conexion() as conn:
        cursor = conn.execute("SELECT valor FROM listas_valores WHERE lista=? ORDER BY orden, id", (lista,))
        return [r[0] for r in cursor.fetchall()]


def _validar_valor(lista, valor):
    valor = _validar_nombre(valor)
    if valor.upper() == SIN_DATO:
        raise ValueError(f"«{SIN_DATO}» ya existe y no se puede modificar.")
    if LISTAS[lista][1] and "," in valor:
        raise ValueError("En esta lista no se permiten comas (se usan para separar las opciones elegidas).")
    return valor


def agregar_valor(lista, valor):
    """Agrega un valor a la lista y devuelve el texto tal como quedó guardado
    (si ya existía con otras mayúsculas, devuelve el existente). Lanza ValueError si no es válido."""
    valor = _validar_valor(lista, valor)
    with conexion() as conn:
        _agregar_valor(conn, lista, valor)
        conn.commit()
        return conn.execute("SELECT valor FROM listas_valores WHERE lista=? AND valor=?", (lista,
valor)).fetchone()[0]


def _usos(conn, lista, valor):
    """[(id_componente, columna, texto)] de los componentes que tienen ese valor en la lista."""
    columnas, multiple = LISTAS[lista]
    usos = []
    for col in columnas:
        if multiple:
            filas = conn.execute(f"SELECT id, {col} FROM productos_fabricantes WHERE {col} LIKE ?", (f"%{valor}
%",)).fetchall()
            usos += [(i, col, t) for i, t in filas if valor in [p.strip() for p in str(t).split(",")]]
        else:
            usos += [(i, col, valor) for (i,) in conn.execute(f"SELECT id FROM productos_fabricantes WHERE
{col}=?", (valor,)).fetchall()]
    return usos


def contar_uso_valor(lista, valor):
    """Cuántos componentes usan ese valor."""
    with conexion() as conn:
        return len({i for i, _, _ in _usos(conn, lista, valor)})


def renombrar_valor(lista, viejo, nuevo):
    """Cambia un valor de la lista y también en los componentes que lo usan. Lanza ValueError si no es válido."""
    if str(viejo).upper() == SIN_DATO:
        raise ValueError(f"«{SIN_DATO}» no se puede modificar.")
    nuevo = _validar_valor(lista, nuevo)
    columnas, multiple = LISTAS[lista]
    with conexion() as conn:
        try:
            usos = _usos(conn, lista, viejo)     # antes de tocar nada
            try:
                conn.execute("UPDATE listas_valores SET valor=? WHERE lista=? AND valor=?", (nuevo, lista, viejo))
```

```
            except sqlite3.IntegrityError:
                raise ValueError(f"«{nuevo}» ya existe en la lista.")
            if lista == "moneda":
                conn.execute("UPDATE OR IGNORE cotizaciones SET moneda=? WHERE moneda=?", (nuevo, viejo))
            for i, col, texto in usos:
                if multiple:
                    texto = ", ".join(nuevo if p.strip() == viejo else p.strip() for p in str(texto).split(","))
                    conn.execute(f"UPDATE productos_fabricantes SET {col}=? WHERE id=?", (texto, i))
                else:
                    conn.execute(f"UPDATE productos_fabricantes SET {col}=? WHERE id=?", (nuevo, i))
            conn.commit()
        except Exception:
            conn.rollback()
            raise


def eliminar_valor(lista, valor):
    """Quita el valor de la lista. Los componentes que ya lo tienen conservan el texto."""
    if str(valor).upper() == SIN_DATO:
        raise ValueError(f"«{SIN_DATO}» no se puede eliminar.")
    with conexion() as conn:
        conn.execute("DELETE FROM listas_valores WHERE lista=? AND valor=?", (lista, valor))
        conn.commit()


# ---------------------------------------------------------------------
# Componentes
# ---------------------------------------------------------------------
def buscar_componentes(tipo=None, cat=None, subcat=None, item=None, texto=None):
    """Filas para la grilla principal (19 columnas, el orden importa a la interfaz)."""
    query = """
            SELECT id, nombre, chip_principal, tipo_componente, categoria, subcategoria, item_especifico,
                   descripcion, fabricante, interfaz_comunicacion, pines_relevantes, stock, precio, ubicacion,
qr_path,
                   precio_exterior, proveedor_exterior, precio_local, proveedor_local
            FROM productos_fabricantes WHERE 1=1
        """
    params = []

    if tipo:
        query += " AND tipo_componente=?"
        params.append(tipo)
    if cat:
        query += " AND categoria=?"
        params.append(cat)
    if subcat:
        query += " AND subcategoria=?"
        params.append(subcat)
    if item:
        query += " AND item_especifico=?"
        params.append(item)

    if texto:
        query += " AND (nombre LIKE ? OR descripcion LIKE ? OR fabricante LIKE ? OR chip_principal LIKE ? OR
proveedor_exterior LIKE ? OR proveedor_local LIKE ?)"
        p = f"%{texto}%"
        params.extend([p, p, p, p, p, p])

    with conexion() as conn:
        cursor = conn.execute(query, params)
        return cursor.fetchall()


def obtener_componente(comp_id):
    """Todas las columnas del componente como diccionario, o None si no existe."""
    with conexion() as conn:
        cursor = conn.execute("SELECT * FROM productos_fabricantes WHERE id=?", (comp_id,))
        col_names = [description[0] for description in cursor.description]
        row_data = cursor.fetchone()
    return dict(zip(col_names, row_data)) if row_data else None


def obtener_datos_etiqueta(item_id):
    """(chip_principal, item_especifico, descripcion, ubicacion, fabricante, stock) o None."""
    with conexion() as conn:
        cursor = conn.execute("SELECT chip_principal, item_especifico, descripcion, ubicacion, fabricante, stock
FROM productos_fabricantes WHERE id=?", (item_id,))
        return cursor.fetchone()


def _valores_componente(d):
    """Valores de la fila en el orden exacto que esperan el UPDATE y el INSERT."""
```

```
    return (d['nombre'], d['nombre'], d['tipo_componente'], d['categoria'], d['subcategoria'],
d['item_especifico'],
            d['descripcion'], d['fabricante'], d['chip_principal'],
            d['interfaz_comunicacion'], d['pines_relevantes'], d['encapsulado'],
            d['stock'], d['precio'], d['ubicacion'],
            d['precio_exterior'], d['moneda_exterior'], d['proveedor_exterior'],
            d['precio_local'], d['moneda_local'], d['proveedor_local'], d['obsolescencia'])


def guardar_componente(comp_id, datos, archivos, parametros):
    """Inserta (comp_id=None) o actualiza un componente con sus archivos y parámetros.

    Todo ocurre en una sola transacción: si algo falla se revierte y se vuelve
    a lanzar la excepción. Devuelve el id del componente.

    datos:      dict con las columnas del componente (producto_id se toma de 'nombre').
    archivos:   lista de (tipo_archivo, nombre_amigable, ruta_archivo).
    parametros: lista de (clave, valor); las claves vacías se ignoran.
    """
    with conexion() as conn:
        cursor = conn.cursor()
        try:
            valores = _valores_componente(datos)
            if comp_id is not None:
                cursor.execute("""
                    UPDATE productos_fabricantes
                    SET nombre=?, producto_id=?, tipo_componente=?, categoria=?, subcategoria=?,
item_especifico=?,
                        descripcion=?, fabricante=?, chip_principal=?, interfaz_comunicacion=?,
pines_relevantes=?,
                        encapsulado=?, stock=?, precio=?, ubicacion=?,
                        precio_exterior=?, moneda_exterior=?, proveedor_exterior=?,
                        precio_local=?, moneda_local=?, proveedor_local=?, obsolescencia=?
                    WHERE id=?
                """, valores + (comp_id,))
            else:
                cursor.execute("""
                    INSERT INTO productos_fabricantes
                    (nombre, producto_id, tipo_componente, categoria, subcategoria, item_especifico,
                     descripcion, fabricante, chip_principal, interfaz_comunicacion, pines_relevantes,
                     encapsulado, stock, precio, ubicacion,
                     precio_exterior, moneda_exterior, proveedor_exterior,
                     precio_local, moneda_local, proveedor_local, obsolescencia)
                    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
                """, valores)
                comp_id = cursor.lastrowid

            # Archivos múltiples (imágenes, datasheets, tutoriales)
            cursor.execute("DELETE FROM archivos_componentes WHERE componente_id=?", (comp_id,))
            for tipo, nombre, ruta in archivos:
                cursor.execute("""
                    INSERT INTO archivos_componentes (componente_id, tipo_archivo, nombre_amigable, ruta_archivo)
                    VALUES (?, ?, ?, ?)
                """, (comp_id, tipo, nombre, ruta))

            # Parámetros eléctricos / especificaciones
            cursor.execute("DELETE FROM parametros_electricos WHERE componente_id=?", (comp_id,))
            for clave, valor in parametros:
                if clave.strip():
                    cursor.execute("""
                        INSERT INTO parametros_electricos (componente_id, parametro_clave, parametro_valor)
                        VALUES (?, ?, ?)
                    """, (comp_id, clave.strip(), valor.strip()))

            # Lo escrito a mano pasa a formar parte del catálogo y de las listas (crece según la necesidad)
            ruta = _ruta((datos.get('tipo_componente'), datos.get('categoria'), datos.get('subcategoria'),
datos.get('item_especifico')))
            if ruta:
                _insertar_ruta(conn, ruta)
            for lista, (columnas, multiple) in LISTAS.items():
                for col in columnas:
                    for v in _valores_de(datos.get(col), multiple):
                        _agregar_valor(conn, lista, v)

            conn.commit()
        except Exception:
            conn.rollback()
            raise
    return comp_id


def eliminar_componente(comp_id):
```

```
    """Borra el componente. Con foreign_keys activo, el CASCADE borra sus filas de archivos y parámetros."""
    with conexion() as conn:
        conn.execute("DELETE FROM productos_fabricantes WHERE id=?", (comp_id,))
        conn.commit()


# ---------------------------------------------------------------------
# Inventario valorizable y cotizaciones
# ---------------------------------------------------------------------
def listar_inventario():
    """Todos los componentes con lo necesario para valorizar el stock: lista de diccionarios."""
    columnas = ("id", "tipo_componente", "categoria", "subcategoria", "item_especifico", "chip_principal",
"descripcion",
                "fabricante", "stock", "precio_local", "moneda_local", "precio_exterior", "moneda_exterior")
    with conexion() as conn:
        filas = conn.execute(f"SELECT {', '.join(columnas)} FROM productos_fabricantes "
                             "ORDER BY tipo_componente, categoria, subcategoria, item_especifico, chip_principal,
id").fetchall()
    return [dict(zip(columnas, f)) for f in filas]


def listar_cotizaciones():
    """[(moneda, unidades por 1 USD o None si falta, fecha o None)]: USD primero, luego las monedas de la lista
    y las que usan los componentes."""
    with conexion() as conn:
        guardadas = {m.lower(): (por_usd, fecha) for m, por_usd, fecha in conn.execute("SELECT moneda, por_usd,
fecha FROM cotizaciones")}
        usadas = [m for (m,) in conn.execute(
            "SELECT moneda_local FROM productos_fabricantes UNION SELECT moneda_exterior FROM
productos_fabricantes ORDER BY 1")]
        listadas = [v for (v,) in conn.execute("SELECT valor FROM listas_valores WHERE lista='moneda' ORDER BY
orden, id")]
    monedas = [MONEDA_BASE]
    for m in listadas + usadas:
        m = str(m or "").strip()
        if m and m.lower() not in {x.lower() for x in monedas}:
            monedas.append(m)
    return [(m,) + guardadas.get(m.lower(), (None, None)) for m in monedas]


def guardar_cotizacion(moneda, por_usd, fecha=None):
    """Guarda cuántas unidades de `moneda` equivalen a 1 USD (None = borrar la cotización). La fecha por defecto
es hoy.
    Lanza ValueError si el valor no es válido."""
    moneda = _validar_nombre(moneda)
    if moneda.upper() == MONEDA_BASE:
        raise ValueError(f"{MONEDA_BASE} es la moneda base: vale siempre 1.")
    if por_usd is not None:
        try:
            por_usd = float(por_usd)
        except (TypeError, ValueError):
            raise ValueError("La cotización debe ser un número.")
        if not math.isfinite(por_usd) or por_usd <= 0:
            raise ValueError("La cotización debe ser un número mayor que cero.")
    fecha = fecha or date.today().isoformat()
    with conexion() as conn:
        conn.execute("""INSERT INTO cotizaciones (moneda, por_usd, fecha) VALUES (?, ?, ?)
                        ON CONFLICT(moneda) DO UPDATE SET por_usd=excluded.por_usd, fecha=excluded.fecha""",
(moneda, por_usd, fecha))
        conn.commit()


# ---------------------------------------------------------------------
# Archivos adjuntos (solo filas de la base; los archivos físicos son de otro módulo)
# ---------------------------------------------------------------------
def listar_archivos(comp_id):
    """Lista de (tipo_archivo, nombre_amigable, ruta_archivo)."""
    with conexion() as conn:
        cursor = conn.execute("SELECT tipo_archivo, nombre_amigable, ruta_archivo FROM archivos_componentes WHERE
componente_id=? ORDER BY id", (comp_id,))
        return cursor.fetchall()


def listar_archivos_con_id(comp_id):
    """Lista de (id, tipo_archivo, ruta_archivo), para verificar que existan en disco."""
    with conexion() as conn:
        cursor = conn.execute("SELECT id, tipo_archivo, ruta_archivo FROM archivos_componentes WHERE componente_id
= ? ORDER BY id", (comp_id,))
        return cursor.fetchall()


def listar_rutas_archivos():
```

```
    """Todas las rutas de archivos adjuntos registradas (de todos los componentes), sin repetir."""
    with conexion() as conn:
        cursor = conn.execute("SELECT DISTINCT ruta_archivo FROM archivos_componentes WHERE ruta_archivo IS NOT
NULL AND ruta_archivo <> ''")
        return [fila[0] for fila in cursor.fetchall()]


def listar_todos_los_archivos():
    """Lista de (id, ruta_archivo) de todos los adjuntos (usado por la migración al almacén por hash)."""
    with conexion() as conn:
        cursor = conn.execute("SELECT id, ruta_archivo FROM archivos_componentes ORDER BY id")
        return cursor.fetchall()


def actualizar_rutas_archivos(cambios):
    """Cambia la ruta de varios adjuntos en una sola transacción. `cambios`: lista de (id, ruta_nueva)."""
    with conexion() as conn:
        try:
            conn.executemany("UPDATE archivos_componentes SET ruta_archivo = ? WHERE id = ?",
                             [(ruta, i) for i, ruta in cambios])
            conn.commit()
        except Exception:
            conn.rollback()
            raise


def eliminar_archivos_por_id(ids):
    """Borra filas de archivos_componentes por su id."""
    with conexion() as conn:
        conn.executemany("DELETE FROM archivos_componentes WHERE id = ?", [(i,) for i in ids])
        conn.commit()


# ---------------------------------------------------------------------
# Parámetros eléctricos
# ---------------------------------------------------------------------
def listar_parametros(comp_id):
    """Lista de (parametro_clave, parametro_valor)."""
    with conexion() as conn:
        cursor = conn.execute("SELECT parametro_clave, parametro_valor FROM parametros_electricos WHERE
componente_id=?", (comp_id,))
        return cursor.fetchall()
```

almacenamiento.py

```
"""Almacenamiento de documentos por contenido (hash SHA-256).

Punto único por el que la aplicación guarda, comprueba y retira archivos
adjuntos. La interfaz no toca el disco por su cuenta: llama a este módulo.

Cómo se guardan los documentos (etapa 7)
----------------------------------------
* Cada documento se guarda UNA sola vez, con el nombre de su huella SHA-256:
      adjuntos/almacen/<2 primeros caracteres>/<huella><extensión>
  Dos documentos con el mismo contenido (aunque tengan distinto nombre o se
  suban a distintos componentes) ocupan un único archivo.
* La base de datos guarda la ruta RELATIVA a la carpeta `adjuntos`
  (por ejemplo "almacen/ab/ab12...ef.pdf"), con barras "/". Así la carpeta de
  la aplicación se puede mover o copiar a otro equipo sin romper nada.
  El nombre que ve el usuario ("LM358.pdf") queda en la columna
  `nombre_amigable`, no en el disco.
* La escritura es atómica: se copia a un archivo temporal dentro del almacén,
  se calcula la huella en la misma pasada y recién entonces se renombra al
  nombre definitivo. Un corte a mitad de copia nunca deja un documento a medias.
* Quitar un documento de un componente NO borra el archivo (otro componente
  puede estar usándolo). Los archivos que ya nadie referencia se mueven a
  `adjuntos/papelera` al arrancar la aplicación (limpiar_huerfanos); nada se
  borra de forma definitiva.
* La cantidad de usos (conteo de referencias) no se guarda aparte: se obtiene
  de la base de datos, que es la fuente de verdad.

Las rutas que maneja la interfaz en memoria son absolutas; `resolver` convierte
la ruta guardada en la base de datos y `para_guardar` hace el camino inverso.

Importar este módulo no toca el disco.
"""
import hashlib
import os
import re
import shutil
import time
import uuid

from config import ATTACHMENTS_DIR, ETIQUETAS_DIR, TMP_DIR

NOMBRE_ALMACEN = "almacen"
ALMACEN_DIR = os.path.join(ATTACHMENTS_DIR, NOMBRE_ALMACEN)
PAPELERA_DIR = os.path.join(ATTACHMENTS_DIR, "papelera")
_SUBCARPETA_TEMPORAL = "_tmp"          # dentro del almacén, para copias en curso
_TAM_BLOQUE = 1024 * 1024
_EXTENSION_VALIDA = re.compile(r"^\.[a-z0-9]{1,10}$")


# ---------------------------------------------------------------------
# Nombres y rutas
# ---------------------------------------------------------------------
def nombre_archivo(chip, resto):
    """'<chip saneado>_<resto>': el chip solo conserva letras y números (lo demás pasa a '_')."""
    return f"{re.sub(r'[^a-zA-Z0-9]', '_', chip)}_{resto}"


def ruta_temporal(nombre):
    """Ruta de un archivo de trabajo en adjuntos/tmp."""
    return os.path.join(TMP_DIR, nombre)


def ruta_pdf_etiquetas():
    """Ruta fija (carpeta `Etiquetas`) del PDF de etiquetas generado; crea la carpeta si falta."""
    os.makedirs(ETIQUETAS_DIR, exist_ok=True)
    return os.path.join(ETIQUETAS_DIR, "Etiquetas_Inventario.pdf")


def resolver(ruta):
    """Ruta guardada en la base de datos -> ruta absoluta en disco.

    Las rutas absolutas (documentos anteriores a la etapa 7) se devuelven tal cual;
    las relativas se interpretan respecto de la carpeta `adjuntos`.
    """
    if not ruta or os.path.isabs(ruta):
        return ruta
    return os.path.join(ATTACHMENTS_DIR, *re.split(r"[\\/]", ruta))


def resolver_archivos(filas):
```

```
    """Aplica `resolver` a la ruta de cada fila (tipo, nombre, ruta) leída de la base."""
    return [(tipo, nombre, resolver(ruta)) for tipo, nombre, ruta in filas]


def esta_en_almacen(ruta):
    """True si la ruta (absoluta o relativa) apunta dentro del almacén por hash."""
    if not ruta:
        return False
    absoluta = os.path.normcase(os.path.abspath(resolver(ruta)))
    almacen = os.path.normcase(os.path.abspath(ALMACEN_DIR))
    return absoluta.startswith(almacen + os.sep)


def _relativa(ruta_absoluta):
    return os.path.relpath(ruta_absoluta, ATTACHMENTS_DIR).replace(os.sep, "/")


# ---------------------------------------------------------------------
# Huella y escritura en el almacén
# ---------------------------------------------------------------------
def calcular_hash(ruta):
    """Huella SHA-256 (hexadecimal) del contenido de un archivo."""
    h = hashlib.sha256()
    with open(ruta, "rb") as f:
        for bloque in iter(lambda: f.read(_TAM_BLOQUE), b""):
            h.update(bloque)
    return h.hexdigest()


def _extension(origen):
    ext = os.path.splitext(origen)[1].lower()
    return ext if _EXTENSION_VALIDA.match(ext) else ""


def _buscar_existente(carpeta, huella):
    """Archivo ya guardado con esa huella (con cualquier extensión), o None."""
    if not os.path.isdir(carpeta):
        return None
    for nombre in os.listdir(carpeta):
        if nombre == huella or (nombre.startswith(huella) and nombre[len(huella)] == "."):
            return os.path.join(carpeta, nombre)
    return None


def incorporar(origen):
    """Guarda el archivo `origen` en el almacén. Devuelve (ruta_relativa, huella).

    Si ya existe un documento con el mismo contenido no se escribe nada y se
    devuelve el existente. La copia es atómica (ver el encabezado del módulo).
    """
    carpeta_tmp = os.path.join(ALMACEN_DIR, _SUBCARPETA_TEMPORAL)
    os.makedirs(carpeta_tmp, exist_ok=True)
    temporal = os.path.join(carpeta_tmp, uuid.uuid4().hex + ".part")
    h = hashlib.sha256()
    try:
        with open(origen, "rb") as entrada, open(temporal, "wb") as salida:
            for bloque in iter(lambda: entrada.read(_TAM_BLOQUE), b""):
                h.update(bloque)
                salida.write(bloque)
            salida.flush()
            os.fsync(salida.fileno())
        huella = h.hexdigest()
        carpeta = os.path.join(ALMACEN_DIR, huella[:2])
        os.makedirs(carpeta, exist_ok=True)

        existente = _buscar_existente(carpeta, huella)
        if existente and os.path.getsize(existente) == os.path.getsize(temporal):
            os.remove(temporal)
            destino = existente
        else:
            # No existía, o el existente está incompleto/dañado: se reemplaza de forma atómica.
            destino = existente or os.path.join(carpeta, huella + _extension(origen))
            os.replace(temporal, destino)
    except BaseException:
        if os.path.exists(temporal):
            os.remove(temporal)
        raise
    return _relativa(destino), huella


def preparar_adjunto(origen):
    """Registra un archivo elegido por el usuario. Devuelve (ruta, nombre_original).
```

```
    No copia nada todavía: el documento entra al almacén cuando se guarda el
    componente (`para_guardar`). Así, si el usuario cancela, no queda basura.
    """
    return origen, os.path.basename(origen)


def para_guardar(ruta):
    """Ruta en memoria -> ruta que se guarda en la base de datos.

    * Ya está en el almacén: se devuelve su ruta relativa.
    * Es un archivo existente fuera del almacén: se incorpora y se devuelve la relativa.
    * No existe (referencia rota): se devuelve sin cambios, para no perder el registro.
    """
    if not ruta:
        return ruta
    absoluta = resolver(ruta)
    if esta_en_almacen(absoluta):
        return _relativa(os.path.abspath(absoluta))
    if os.path.isfile(absoluta):
        return incorporar(absoluta)[0]
    return ruta


# ---------------------------------------------------------------------
# Imágenes de trabajo
# ---------------------------------------------------------------------
def guardar_imagen(imagen, destino):
    """Guarda una imagen en la ruta que eligió el usuario, creando la carpeta si falta."""
    os.makedirs(os.path.dirname(destino), exist_ok=True)
    if imagen.save(destino) is False:      # Qt devuelve False (sin excepción) si no pudo escribir
        raise OSError("no se pudo escribir el archivo (¿extensión o permisos?)")


def guardar_temporal(imagen, nombre, formato=None):
    """Guarda una imagen en adjuntos/tmp y devuelve la ruta. `formato` es opcional (p. ej. 'PNG')."""
    ruta = ruta_temporal(nombre)
    if formato:
        imagen.save(ruta, formato)
    else:
        imagen.save(ruta)
    return ruta


# ---------------------------------------------------------------------
# Consulta
# ---------------------------------------------------------------------
def existe(ruta):
    """True si el archivo existe en disco (acepta ruta absoluta o relativa a `adjuntos`)."""
    return os.path.exists(resolver(ruta))


def listar_almacen():
    """Rutas absolutas de todos los documentos del almacén (sin las copias en curso)."""
    encontrados = []
    if not os.path.isdir(ALMACEN_DIR):
        return encontrados
    for carpeta, subcarpetas, archivos in os.walk(ALMACEN_DIR):
        subcarpetas[:] = [d for d in subcarpetas if d != _SUBCARPETA_TEMPORAL]
        encontrados += [os.path.join(carpeta, a) for a in archivos]
    return sorted(encontrados)


def verificar_almacen():
    """Recalcula la huella de cada documento. Devuelve [(ruta, problema)] (vacía si todo está bien)."""
    problemas = []
    for ruta in listar_almacen():
        esperado = os.path.splitext(os.path.basename(ruta))[0]
        if calcular_hash(ruta) != esperado:
            problemas.append((ruta, "el contenido no coincide con su huella"))
    return problemas


# ---------------------------------------------------------------------
# Mantenimiento
# ---------------------------------------------------------------------
def huerfanos(rutas_en_uso):
    """Documentos del almacén que ninguna fila de la base referencia."""
    usadas = {os.path.normcase(os.path.abspath(resolver(r))) for r in (rutas_en_uso or []) if r}
    return [r for r in listar_almacen() if os.path.normcase(os.path.abspath(r)) not in usadas]
```

```
def limpiar_huerfanos(rutas_en_uso, antiguedad_minima_s=3600):
    """Mueve a `adjuntos/papelera` los documentos sin uso. Devuelve las rutas movidas.

    * No borra nada de forma definitiva: la papelera se vacía a mano.
    * Respeta los archivos recientes (por defecto menos de 1 hora) por si otra
      instancia de la aplicación los acaba de subir y todavía no los guardó.
    * Retira también copias temporales abandonadas (más de 1 día).
    """
    ahora = time.time()
    movidos = []
    for ruta in huerfanos(rutas_en_uso):
        if ahora - os.path.getmtime(ruta) < antiguedad_minima_s:
            continue
        os.makedirs(PAPELERA_DIR, exist_ok=True)
        destino = os.path.join(PAPELERA_DIR, os.path.basename(ruta))
        if os.path.exists(destino):
            base, ext = os.path.splitext(destino)
            destino = f"{base}.{int(ahora)}{ext}"
        shutil.move(ruta, destino)
        movidos.append(ruta)
        print(f"Documento sin uso movido a la papelera: {ruta}")
        try:
            os.rmdir(os.path.dirname(ruta))      # solo si quedó vacía
        except OSError:
            pass
    carpeta_tmp = os.path.join(ALMACEN_DIR, _SUBCARPETA_TEMPORAL)
    if os.path.isdir(carpeta_tmp):
        for nombre in os.listdir(carpeta_tmp):
            ruta = os.path.join(carpeta_tmp, nombre)
            if ahora - os.path.getmtime(ruta) > 86400:
                os.remove(ruta)
    return movidos
```

utilidades.py

```
"""Funciones de texto sin dependencias de la interfaz."""
import re


def limpiar_prefijo(texto):
    if not texto:
        return ""
    texto_limpio = re.sub(r'^\d+[\s\.\-_]*', '', str(texto)).strip()
    return texto_limpio if texto_limpio else str(texto).strip()


def sin_comodin(valor, *comodines):
    """Devuelve None si el valor está vacío o es un texto comodín ("-- Todos --"); si no, el valor."""
    if not valor or valor in comodines:
        return None
    return valor
```

ocr.py

```
"""Reconocimiento de texto (OCR) con Tesseract, para páginas de PDF cuyo texto es parte de una imagen.

Requiere DOS cosas (la biblioteca de Python es solo el puente hacia el programa):
  1. La biblioteca de Python:   pip install pytesseract
  2. El programa Tesseract:
       * Windows: instalador de UB Mannheim (https://github.com/UB-Mannheim/tesseract/wiki). Se instala en
         C:\\Program Files\\Tesseract-OCR. En el instalador conviene tildar el idioma "Spanish" (Additional
language data).
       * Si Tesseract está en otra carpeta, defina la variable de entorno TESSERACT_CMD con la ruta de
tesseract.exe.
       * Versión portátil: una carpeta "Tesseract-OCR" junto a la aplicación (ver config.BASE_DIR) se usa antes
que
         cualquier otra; así el OCR viaja con el pendrive.

Este módulo no depende de la interfaz: recibe la ruta de una imagen y devuelve el texto.
Importarlo no ejecuta nada (pytesseract se importa al usarlo, así la aplicación arranca aunque falte).
"""
import os
import shutil
import sys

INSTRUCCIONES = ("OCR no disponible: instale el programa Tesseract "
                 "(https://github.com/UB-Mannheim/tesseract/wiki) y reinicie la aplicación.")
INSTRUCCIONES_PIP = "OCR no disponible: falta la biblioteca de Python. Ejecute:  pip install pytesseract"

_RUTAS_WINDOWS = (
    r"C:\Program Files\Tesseract-OCR\tesseract.exe",
    r"C:\Program Files (x86)\Tesseract-OCR\tesseract.exe",
    os.path.join(os.environ.get("LOCALAPPDATA", ""), "Programs", "Tesseract-OCR", "tesseract.exe"),
)
_PREFERENCIA_IDIOMAS = ("eng+spa", "spa", "eng")
_TIMEOUT_S = 60
_cache_idioma = {}          # ejecutable -> idioma elegido


class OcrError(Exception):
    """El OCR no pudo ejecutarse (falta algo, falló o tardó demasiado). El mensaje se puede mostrar al usuario."""


def _ruta_portatil():
    """tesseract.exe dentro de <carpeta de la aplicación>\\Tesseract-OCR, si existe (instalación portátil)."""
    try:
        from config import BASE_DIR
    except ImportError:
        return None
    return os.path.join(BASE_DIR, "Tesseract-OCR", "tesseract.exe" if sys.platform.startswith("win") else
"tesseract")


def buscar_tesseract():
    """Ruta del programa Tesseract, o None si no está instalado."""
    candidatos = [os.environ.get("TESSERACT_CMD"), _ruta_portatil(), shutil.which("tesseract")]
    if sys.platform.startswith("win"):
        candidatos += list(_RUTAS_WINDOWS)
    for ruta in candidatos:
        if ruta and os.path.isfile(ruta):
            return ruta
    return None


def _preparar():
    """Devuelve el módulo pytesseract configurado con la ruta del programa."""
    try:
        import pytesseract
    except ImportError:
        raise OcrError(INSTRUCCIONES_PIP)
    exe = buscar_tesseract()
    if not exe:
        raise OcrError(INSTRUCCIONES)
    pytesseract.pytesseract.tesseract_cmd = exe
    return pytesseract, exe


def _idioma(pytesseract, exe):
    """Mejor combinación de idiomas instalada (inglés + español si están; si no, lo que haya)."""
    if exe not in _cache_idioma:
        elegido = None
        try:
            instalados = set(pytesseract.get_languages(config=""))
```

```
            for opcion in _PREFERENCIA_IDIOMAS:
                if set(opcion.split("+")) <= instalados:
                    elegido = opcion
                    break
            if elegido is None and instalados - {"osd"}:
                elegido = sorted(instalados - {"osd"})[0]
        except Exception:
            pass                          # sin lista de idiomas se usa el predeterminado de Tesseract
        _cache_idioma[exe] = elegido
    return _cache_idioma[exe]


def _limpiar(texto):
    lineas = [" ".join(l.split()) for l in texto.replace("\x0c", "").splitlines()]
    return "\n".join(l for l in lineas if l).strip()


def reconocer_texto(ruta_imagen):
    """Texto reconocido en la imagen (cadena vacía si no encontró nada). Lanza OcrError si no se pudo ejecutar."""
    pytesseract, exe = _preparar()
    idioma = _idioma(pytesseract, exe)
    from PIL import Image
    try:
        with Image.open(ruta_imagen) as imagen:
            imagen.load()
            # psm 6: bloque de texto; si no encuentra nada, psm 11: texto suelto (rótulos dispersos en un
diagrama)
            for psm in ("6", "11"):
                try:
                    bruto = pytesseract.image_to_string(imagen, lang=idioma, config=f"--psm {psm}",
timeout=_TIMEOUT_S)
                except pytesseract.TesseractNotFoundError:
                    raise OcrError(INSTRUCCIONES)
                except pytesseract.TesseractError as e:
                    raise OcrError(f"Tesseract devolvió un error: {str(e)[:200]}")
                except RuntimeError:                      # pytesseract lanza RuntimeError al agotar el tiempo
                    raise OcrError("El OCR tardó demasiado y se canceló.")
                texto = _limpiar(bruto)
                if texto:
                    return texto
    except OSError as e:
        raise OcrError(f"No se pudo abrir la imagen para el OCR: {e}")
    return ""
```

capital.py

```
"""Capital en existencia: valorización del inventario y planilla Excel.

Criterio (decidido con el usuario):
* Moneda base: USD. Cada precio se convierte con la cotización cargada en la tabla
  `cotizaciones` (unidades de la moneda por 1 USD). Nunca se inventa una cotización:
  si falta, el componente queda «Sin cotización» y no suma.
* Precio de cada componente: el local si existe (> 0); si no, el del exterior.
* Capital de un componente = precio unitario en USD × stock.

La planilla se escribe con FÓRMULAS, así que al cambiar una cotización dentro de Excel
se recalcula todo. `valorizar` repite en Python la misma regla para poder mostrar un
resumen antes de generar el archivo y para verificar que las fórmulas son correctas.

Este módulo no usa la interfaz. `openpyxl` se importa recién al exportar, así la
aplicación arranca aunque no esté instalado.
"""
from datetime import datetime

import db

SIN_DEFINIR = "(sin definir)"
ESTADO_OK = "OK"
ESTADO_SIN_PRECIO = "Sin precio"
ESTADO_SIN_COTIZACION = "Sin cotización"

INSTRUCCIONES_PIP = "Instale la biblioteca con:  pip install openpyxl"


class ExportError(Exception):
    """No se pudo generar la planilla; el mensaje explica el motivo."""


# ---------------------------------------------------------------------
# Valorización
# ---------------------------------------------------------------------
def _numero(valor):
    try:
        return float(valor)
    except (TypeError, ValueError):
        return 0.0


def valorizar(componente, cotizaciones):
    """Valora un componente. `cotizaciones`: {moneda en minúscula: unidades por 1 USD} (solo las cargadas).

    Devuelve un diccionario con: precio, moneda, origen ('Local'/'Exterior'/'—'), unitario_usd (o None),
    total_usd y estado.
    """
    local, exterior = _numero(componente.get("precio_local")), _numero(componente.get("precio_exterior"))
    if local > 0:
        precio, moneda, origen = local, str(componente.get("moneda_local") or "").strip(), "Local"
    elif exterior > 0:
        precio, moneda, origen = exterior, str(componente.get("moneda_exterior") or "").strip(), "Exterior"
    else:
        return dict(precio=0.0, moneda="", origen="—", unitario_usd=None, total_usd=0.0, estado=ESTADO_SIN_PRECIO)

    tasa = cotizaciones.get(moneda.lower())
    if not tasa or tasa <= 0:
        return dict(precio=precio, moneda=moneda, origen=origen, unitario_usd=None, total_usd=0.0,
estado=ESTADO_SIN_COTIZACION)
    unitario = precio / tasa
    stock = int(_numero(componente.get("stock")))
    return dict(precio=precio, moneda=moneda, origen=origen, unitario_usd=unitario, total_usd=unitario * stock,
estado=ESTADO_OK)


def calcular():
    """Valoriza todo el inventario con las cotizaciones cargadas.

    Devuelve: filas (componente + 'valor'), cotizaciones [(moneda, por_usd|None, fecha|None)], total_usd,
    unidades, y los conteos sin_precio / sin_cotizacion, con la lista de monedas sin cotización (que se usan).
    """
    cotizaciones = db.listar_cotizaciones()
    tasas = {m.lower(): p for m, p, _ in cotizaciones if p}
    filas = []
    for comp in db.listar_inventario():
        fila = dict(comp)
        fila["valor"] = valorizar(comp, tasas)
        filas.append(fila)
```

```
    sin_cotizacion = [f for f in filas if f["valor"]["estado"] == ESTADO_SIN_COTIZACION]
    return dict(
        filas=filas,
        cotizaciones=cotizaciones,
        total_usd=sum(f["valor"]["total_usd"] for f in filas),
        unidades=sum(int(_numero(f.get("stock"))) for f in filas),
        sin_precio=sum(1 for f in filas if f["valor"]["estado"] == ESTADO_SIN_PRECIO),
        sin_cotizacion=len(sin_cotizacion),
        monedas_sin_cotizacion=sorted({(f["valor"]["moneda"] or "(sin moneda)") for f in sin_cotizacion}),
    )


def formato_monto(valor):
    """1234567.5 -> '1.234.567,50' (como se escribe en Uruguay)."""
    return f"{valor:,.2f}".replace(",", "X").replace(".", ",").replace("X", ".")


# ---------------------------------------------------------------------
# Planilla Excel
# ---------------------------------------------------------------------
def _grupos(filas):
    """Tipos, categorías y subcategorías que realmente tienen componentes, ordenados sin distinguir mayúsculas."""
    def nivel(fila, i):
        return (fila[("tipo_componente", "categoria", "subcategoria")[i]] or "").strip() or SIN_DEFINIR
    caminos = {tuple(nivel(f, i) for i in range(k)) for f in filas for k in (1, 2, 3)}
    return sorted(caminos, key=lambda c: tuple(x.lower() for x in c))


def subtotales(filas):
    """Subtotales por clasificación (para mostrar en pantalla).

    Devuelve [(camino, total_usd, componentes, unidades)] con los mismos grupos de `_grupos`
    (padres antes que hijos). Solo suma los componentes valorizados.
    """
    def nivel(fila, i):
        return (fila[("tipo_componente", "categoria", "subcategoria")[i]] or "").strip() or SIN_DEFINIR
    resultado = []
    for camino in _grupos(filas):
        k = len(camino)
        miembros = [f for f in filas if tuple(nivel(f, i) for i in range(k)) == camino]
        resultado.append((camino, sum(f["valor"]["total_usd"] for f in miembros), len(miembros),
                          sum(int(_numero(f.get("stock"))) for f in miembros)))
    return resultado


def exportar_xlsx(destino, datos=None):
    """Escribe la planilla en `destino` (.xlsx). `datos`: resultado de calcular() (si no se da, se calcula).

    Hojas: Resumen (total y subtotales por clasificación), Detalle (un renglón por componente) y
    Cotizaciones (las que se usaron; editables). Todo lo calculado son fórmulas.
    Lanza ExportError si falta openpyxl; los errores de escritura (archivo abierto en Excel, sin
    permiso) salen como OSError.
    """
    try:
        from openpyxl import Workbook
        from openpyxl.styles import Alignment, Border, Font, PatternFill, Side
        from openpyxl.utils import get_column_letter
        from openpyxl.worksheet.properties import PageSetupProperties
    except ImportError:
        raise ExportError("Falta la biblioteca openpyxl, necesaria para generar la planilla.\n" +
INSTRUCCIONES_PIP)

    datos = datos or calcular()
    filas, cotizaciones = datos["filas"], datos["cotizaciones"]
    otras = [m for m, _, _ in cotizaciones if m.upper() != db.MONEDA_BASE]

    # --- estilos ---
    arial = lambda **k: Font(name="Arial", size=k.pop("size", 10), **k)
    f_normal, f_negrita = arial(), arial(bold=True)
    f_titulo, f_cab = arial(size=14, bold=True), arial(bold=True, color="FFFFFF")
    f_entrada = arial(color="0000FF")              # valores que el usuario puede cambiar
    f_gris = arial(color="808080")
    fill_cab = PatternFill("solid", start_color="1F3864")
    fill_tipo = PatternFill("solid", start_color="D9E1F2")
    fill_cat = PatternFill("solid", start_color="EEF2F9")
    fill_falta = PatternFill("solid", start_color="FFFF00")
    borde = Border(bottom=Side(style="thin", color="BFBFBF"))
    f_monto, f_pct, f_ent = "#,##0.00", "0.0%", "#,##0"

    wb = Workbook()
    ws_res, ws_det = wb.active, wb.create_sheet("Detalle")
    ws_res.title = "Resumen"
```

```
    ws_cot = wb.create_sheet("Cotizaciones")

    # ================ Cotizaciones ================
    ws_cot["A1"], ws_cot["A1"].font = "Cotizaciones", f_titulo
    ws_cot["A2"] = ("Unidades de cada moneda que equivalen a 1 USD (ej.: UYU 40,50 significa 1 USD = 40,50 UYU). "
                    "Los valores en azul se pueden cambiar: toda la planilla se recalcula. En amarillo, las que
faltan cargar.")
    ws_cot["A2"].font = f_gris
    for c, titulo in enumerate(("Moneda", "Unidades por 1 USD", "Actualizada"), start=1):
        celda = ws_cot.cell(row=4, column=c, value=titulo)
        celda.font, celda.fill = f_cab, fill_cab
    for i, (moneda, por_usd, fecha) in enumerate(cotizaciones):
        r = 5 + i
        ws_cot.cell(row=r, column=1, value=moneda).font = f_normal
        celda = ws_cot.cell(row=r, column=2, value=por_usd)
        celda.number_format = "#,##0.0000"
        if moneda.upper() == db.MONEDA_BASE:
            celda.font = f_normal
        else:
            celda.font = f_entrada
            if por_usd is None:
                celda.fill = fill_falta
        ws_cot.cell(row=r, column=3, value=fecha or "").font = f_normal
    COT = "Cotizaciones!$A$5:$B$60"                 # holgura para monedas que se agreguen dentro de Excel
    ws_cot.column_dimensions["A"].width, ws_cot.column_dimensions["B"].width, ws_cot.column_dimensions["C"].width
= 14, 22, 16

    # ================ Detalle ================
    cab = ["ID", "Tipo", "Categoría", "Subcategoría", "Ítem", "Chip Principal", "Descripción", "Fabricante",
"Stock",
           "Precio Local", "Moneda Local", "Precio Exterior", "Moneda Exterior",
           "Precio usado", "Moneda usada", "Origen", "Precio unit. (USD)", "Total (USD)"]
    cab += [f"Total ({m})" for m in otras] + ["Estado"]
    for c, titulo in enumerate(cab, start=1):
        celda = ws_det.cell(row=1, column=c, value=titulo)
        celda.font, celda.fill = f_cab, fill_cab
        celda.alignment = Alignment(horizontal="center", vertical="center", wrap_text=True)
    ws_det.row_dimensions[1].height = 32

    col_estado = len(cab)
    letra_estado = get_column_letter(col_estado)
    for i, comp in enumerate(filas):
        r = 2 + i
        nivel = lambda clave: (comp[clave] or "").strip() or SIN_DEFINIR
        valores = [comp["id"], nivel("tipo_componente"), nivel("categoria"), nivel("subcategoria"),
(comp["item_especifico"] or ""),
                   comp["chip_principal"] or "", comp["descripcion"] or "", comp["fabricante"] or "",
                   int(_numero(comp["stock"])),
                   comp["precio_local"] or 0, comp["moneda_local"] or "", comp["precio_exterior"] or 0,
comp["moneda_exterior"] or ""]
        for c, v in enumerate(valores, start=1):
            ws_det.cell(row=r, column=c, value=v).font = f_normal
        formulas = [
            f'=IF(J{r}>0,J{r},IF(L{r}>0,L{r},0))',                                          # N precio usado
            f'=IF(J{r}>0,K{r},IF(L{r}>0,M{r},""))',                                         # O moneda usada
            f'=IF(J{r}>0,"Local",IF(L{r}>0,"Exterior","—"))',                               # P origen
            f'=IF(N{r}>0,IFERROR(N{r}/VLOOKUP(O{r},{COT},2,FALSE),""),"")',                  # Q unitario USD
            f'=IF(ISNUMBER(Q{r}),Q{r}*I{r},0)',                                             # R total USD
        ]
        for m in otras:
            formulas.append(f'=IFERROR(IF(AND(ISNUMBER(Q{r}),VLOOKUP("{m}",{COT},2,FALSE)>0),R{r}*VLOOKUP("{m}",
{COT},2,FALSE),""),"")')

formulas.append(f'=IF(N{r}<=0,"{ESTADO_SIN_PRECIO}",IF(ISNUMBER(Q{r}),"{ESTADO_OK}","{ESTADO_SIN_COTIZACION}"))')
        for c, formula in enumerate(formulas, start=14):
            ws_det.cell(row=r, column=c, value=formula).font = f_normal
        for c in (10, 12, 14, 17, 18) + tuple(range(19, col_estado)):
            ws_det.cell(row=r, column=c).number_format = f_monto
        ws_det.cell(row=r, column=9).number_format = f_ent
    n = len(filas)
    ultimo = max(n + 1, 2)
    # franjas alternadas (formato condicional: se mantienen al ordenar o filtrar)
    from openpyxl.formatting.rule import FormulaRule
    rango_franjas = f"A2:{letra_estado}{ultimo}"
    ws_det.conditional_formatting.add(rango_franjas, FormulaRule(formula=["MOD(ROW(),2)=0"],
fill=PatternFill("solid", start_color="FFF9D9", end_color="FFF9D9")))
    ws_det.conditional_formatting.add(rango_franjas, FormulaRule(formula=["MOD(ROW(),2)=1"],
fill=PatternFill("solid", start_color="E3ECF2", end_color="E3ECF2")))
    for c, ancho in enumerate([6, 24, 28, 28, 26, 18, 34, 22, 8, 12, 9, 12, 9, 12, 9, 10, 14, 14], start=1):
        ws_det.column_dimensions[get_column_letter(c)].width = ancho
    for c in range(19, col_estado + 1):
        ws_det.column_dimensions[get_column_letter(c)].width = 15
```

```
    ws_det.freeze_panes = "C2"
    ws_det.auto_filter.ref = f"A1:{letra_estado}{ultimo}"

    rango = lambda letra: f"Detalle!${letra}$2:${letra}${ultimo}"

    # ================ Resumen ================
    ws_res["A1"], ws_res["A1"].font = "Capital en existencia", f_titulo
    ws_res["A2"] = (f"Generado el {datetime.now():%d/%m/%Y %H:%M}  ·  Moneda base: {db.MONEDA_BASE}  ·  "
                    "Precio usado: el local si existe, si no el del exterior  ·  Capital = precio unitario ×
stock")
    ws_res["A2"].font = f_gris

    kpis = [
        ("Capital total (USD)", f"=SUM({rango('R')})", f_monto),
    ]
    for m in otras:
        kpis.append((f"Equivalente en {m}", f'=IFERROR(IF(VLOOKUP("{m}",{COT},2,FALSE)>0,$B$4*VLOOKUP("{m}",
{COT},2,FALSE),"sin cotización"),"sin cotización")', f_monto))
    kpis += [
        ("Componentes", f"=COUNTA({rango('A')})", f_ent),
        ("Unidades en existencia", f"=SUM({rango('I')})", f_ent),
        ("Sin precio (no suman)", f'=COUNTIF({rango(letra_estado)},"{ESTADO_SIN_PRECIO}")', f_ent),
        ("Sin cotización (no suman)", f'=COUNTIF({rango(letra_estado)},"{ESTADO_SIN_COTIZACION}")', f_ent),
    ]
    for i, (rotulo, formula, formato) in enumerate(kpis):
        r = 4 + i
        ws_res.cell(row=r, column=1, value=rotulo).font = f_negrita if i == 0 else f_normal
        celda = ws_res.cell(row=r, column=2, value=formula)
        celda.number_format, celda.font = formato, (arial(bold=True, size=12) if i == 0 else f_normal)
        celda.alignment = Alignment(horizontal="right")

    r0 = 4 + len(kpis) + 1
    ws_res.cell(row=r0, column=1, value="Subtotales por clasificación").font = arial(bold=True, size=12)
    r_cab = r0 + 1
    for c, titulo in enumerate(("Tipo", "Categoría", "Subcategoría", "Componentes", "Unidades", "Capital (USD)",
"% del total"), start=1):
        celda = ws_res.cell(row=r_cab, column=c, value=titulo)
        celda.font, celda.fill = f_cab, fill_cab
        celda.alignment = Alignment(horizontal="center")

    letras_nivel = ("B", "C", "D")      # columnas de Detalle: Tipo, Categoría, Subcategoría
    r = r_cab + 1
    filas_tipo = []
    for camino in _grupos(filas):
        k = len(camino)
        for i, nombre in enumerate(camino):
            celda = ws_res.cell(row=r, column=1 + i, value=nombre)
            celda.font = f_negrita if i == k - 1 else f_gris
        cond = "*".join(f"({rango(letras_nivel[i])}=${get_column_letter(1 + i)}{r})" for i in range(k))
        ws_res.cell(row=r, column=4, value=f"=SUMPRODUCT({cond}*1)")
        ws_res.cell(row=r, column=5, value=f"=SUMPRODUCT({cond}*{rango('I')})")
        ws_res.cell(row=r, column=6, value=f"=SUMPRODUCT({cond}*{rango('R')})")
        ws_res.cell(row=r, column=7, value=f"=IF($B$4=0,0,F{r}/$B$4)")
        for c, formato in ((4, f_ent), (5, f_ent), (6, f_monto), (7, f_pct)):
            ws_res.cell(row=r, column=c).number_format = formato
            ws_res.cell(row=r, column=c).font = f_negrita if k == 1 else f_normal
        relleno = fill_tipo if k == 1 else (fill_cat if k == 2 else None)
        for c in range(1, 8):
            ws_res.cell(row=r, column=c).border = borde
            if relleno:
                ws_res.cell(row=r, column=c).fill = relleno
        if k == 1:
            filas_tipo.append(r)
        r += 1

    # control: los subtotales por Tipo deben sumar el total
    r += 1
    suma = "+".join(f"F{x}" for x in filas_tipo) or "0"
    ws_res.cell(row=r, column=1, value="Control: suma de los Tipos").font = f_normal
    ws_res.cell(row=r, column=6, value=f"={suma}").number_format = f_monto
    ws_res.cell(row=r, column=7, value=f'=IF(ABS(F{r}-$B$4)<0.005,"OK","Revisar")').font = f_negrita
    ws_res.cell(row=r, column=6).font = f_normal

    for letra, ancho in zip("ABCDEFG", (32, 52, 34, 14, 12, 18, 12)):
        ws_res.column_dimensions[letra].width = ancho
    ws_res.freeze_panes = ws_res.cell(row=r_cab + 1, column=1)

    for hoja in (ws_res, ws_det, ws_cot):      # al imprimir: horizontal y todas las columnas en el ancho de la
hoja
        hoja.page_setup.orientation = "landscape"
        hoja.page_setup.fitToWidth, hoja.page_setup.fitToHeight = 1, 0
        hoja.sheet_properties.pageSetUpPr = PageSetupProperties(fitToPage=True)
```

```
ws_det.print_title_rows = "1:1"

wb.calculation.fullCalcOnLoad = True       # Excel calcula las fórmulas al abrir
wb.save(destino)
return destino
```

migrar_a_hash.py

```
"""Migra los documentos existentes al almacén por hash SHA-256 (etapa 7).

Uso (desde la carpeta de la aplicación):

    python migrar_a_hash.py                    # SOLO INFORMA qué haría; no cambia nada
    python migrar_a_hash.py --aplicar          # migra (antes hace una copia de la base de datos)
    python migrar_a_hash.py --aplicar --archivar-originales
                                               # además mueve los archivos viejos a adjuntos/originales_migrados

Qué hace, para cada documento registrado en la base de datos:
  1. Localiza el archivo (ruta absoluta antigua o ruta relativa a `adjuntos`).
  2. Lo copia al almacén con el nombre de su huella SHA-256. Si otro documento tiene
     exactamente el mismo contenido, comparten un único archivo.
  3. Cambia la ruta guardada en la base de datos por la ruta relativa del almacén.

Garantías:
  * Por defecto no modifica nada (modo informe).
  * Con --aplicar, primero copia la base de datos a un archivo .bak con fecha y hora.
  * Los archivos originales NO se borran. Con --archivar-originales se mueven (no se
    eliminan) a una carpeta aparte, y solo si ya están a salvo en el almacén.
  * Los documentos que no se encuentran en el disco quedan como estaban y se listan.
  * Se puede ejecutar varias veces: lo ya migrado se reconoce y no se repite.
"""
import argparse
import os
import shutil
import sqlite3
import sys
import time

import almacenamiento
import config
import db


def _respaldar_base():
    base = f"{config.DB_PATH}.bak_{time.strftime('%Y%m%d_%H%M%S')}"
    destino, n = base, 1
    while os.path.exists(destino):          # nunca pisar una copia anterior
        n += 1
        destino = f"{base}_{n}"
    origen = sqlite3.connect(config.DB_PATH)
    try:
        copia = sqlite3.connect(destino)
        try:
            origen.backup(copia)
        finally:
            copia.close()
    finally:
        origen.close()
    return destino


def migrar(aplicar=False, archivar_originales=False, salida=print):
    """Devuelve un dict con el resumen. Con aplicar=False no escribe nada (ni en disco ni en la base)."""
    filas = db.listar_todos_los_archivos()
    resumen = dict(filas=len(filas), ya_migrados=0, migrados=0, faltantes=[], errores=[],
                   documentos_unicos=0, bytes_originales=0, bytes_almacen=0, respaldo=None)
    cambios = []                        # (id, ruta_nueva)
    originales = {}                     # ruta absoluta vieja -> ruta relativa nueva
    vistos = {}                         # huella -> tamaño

    pendientes = [1 for _, r in filas if r and not almacenamiento.esta_en_almacen(r) and
os.path.isfile(almacenamiento.resolver(r))]
    if aplicar and pendientes:                      # solo si hay algo que cambiar
        resumen["respaldo"] = _respaldar_base()
        salida(f"Copia de seguridad de la base: {resumen['respaldo']}")

    for fila_id, ruta in filas:
        absoluta = almacenamiento.resolver(ruta)
        if not ruta or not os.path.isfile(absoluta):
            resumen["faltantes"].append((fila_id, ruta))
            continue
        tam = os.path.getsize(absoluta)
        try:
            if almacenamiento.esta_en_almacen(absoluta):
                resumen["ya_migrados"] += 1
                huella = os.path.splitext(os.path.basename(absoluta))[0]
                nueva = ruta
```

```
            elif aplicar:
                nueva, huella = almacenamiento.incorporar(absoluta)
                originales[absoluta] = nueva
            else:
                huella = almacenamiento.calcular_hash(absoluta)
                nueva = None
        except Exception as e:                          # un archivo ilegible no detiene al resto
            resumen["errores"].append((fila_id, ruta, str(e)))
            continue
        if not almacenamiento.esta_en_almacen(absoluta):
            resumen["migrados"] += 1
            resumen["bytes_originales"] += tam
        if huella not in vistos:
            vistos[huella] = tam
            resumen["bytes_almacen"] += tam
        if nueva is not None and nueva != ruta:
            cambios.append((fila_id, nueva))

    resumen["documentos_unicos"] = len(vistos)

    if aplicar and cambios:
        db.actualizar_rutas_archivos(cambios)

    resumen["sin_archivar"] = []
    if aplicar and archivar_originales and not resumen["errores"]:
        base_adj = os.path.normcase(os.path.abspath(almacenamiento.ATTACHMENTS_DIR))
        carpeta = os.path.join(almacenamiento.ATTACHMENTS_DIR, "originales_migrados")
        for vieja, nueva in originales.items():
            if not (almacenamiento.existe(nueva) and os.path.isfile(vieja)):
                continue
            # Solo se archivan los originales que están dentro de la carpeta `adjuntos` de esta
            # aplicación. Los que están en otro lugar (p. ej. una instalación anterior) no se tocan.
            if not os.path.normcase(os.path.abspath(vieja)).startswith(base_adj + os.sep):
                resumen["sin_archivar"].append(vieja)
                continue
            destino = os.path.join(carpeta, os.path.relpath(vieja, almacenamiento.ATTACHMENTS_DIR))
            os.makedirs(os.path.dirname(destino), exist_ok=True)
            if os.path.exists(destino):
                base, ext = os.path.splitext(destino)
                destino = f"{base}.{int(time.time())}{ext}"
            shutil.move(vieja, destino)
    return resumen


def _mb(n):
    return f"{n / (1024 * 1024):.2f} MB"


def main(argv=None):
    ap = argparse.ArgumentParser(description="Migra los documentos al almacén por hash SHA-256.")
    ap.add_argument("--aplicar", action="store_true", help="realiza los cambios (sin esta opción solo informa)")
    ap.add_argument("--archivar-originales", action="store_true",
                    help="mueve los archivos viejos a adjuntos/originales_migrados (requiere --aplicar)")
    args = ap.parse_args(argv)
    if args.archivar_originales and not args.aplicar:
        ap.error("--archivar-originales requiere --aplicar")

    config.asegurar_directorios()
    db.inicializar_base_de_datos()
    print("MODO:", "APLICAR CAMBIOS" if args.aplicar else "INFORME (no se modifica nada)")
    r = migrar(aplicar=args.aplicar, archivar_originales=args.archivar_originales)

    print(f"\nRegistros de archivos en la base : {r['filas']}")
    print(f"Ya estaban en el almacén         : {r['ya_migrados']}")
    print(f"{'Migrados' if args.aplicar else 'Por migrar'}                          : {r['migrados']}")
    print(f"Documentos distintos (por hash)  : {r['documentos_unicos']}")
    print(f"Espacio de los originales        : {_mb(r['bytes_originales'])}")
    print(f"Espacio en el almacén            : {_mb(r['bytes_almacen'])}")
    if r["faltantes"]:
        print(f"\nNo se encontró el archivo de {len(r['faltantes'])} registro(s) (quedan sin cambios):")
        for fila_id, ruta in r["faltantes"]:
            print(f"   id {fila_id}: {ruta}")
    if r["errores"]:
        print(f"\nERRORES en {len(r['errores'])} registro(s):")
        for fila_id, ruta, msg in r["errores"]:
            print(f"   id {fila_id}: {ruta} -> {msg}")
    if r.get("sin_archivar"):
        print(f"\n{len(r['sin_archivar'])} original(es) están fuera de la carpeta 'adjuntos' de esta aplicación y
NO se movieron:")
        for ruta in r["sin_archivar"]:
            print(f"   {ruta}")
    if not args.aplicar:
```

```
        print("\nPara migrar de verdad: python migrar_a_hash.py --aplicar")
    return 1 if r["errores"] else 0


if __name__ == "__main__":
    sys.exit(main())
```

interfaz__init__.py

```
"""Interfaz gráfica (PyQt5): un módulo por ventana o diálogo."""
```

interfaz\widgets.py

```
"""Widgets propios reutilizables."""
from PyQt5.QtCore import QPoint, Qt
from PyQt5.QtWidgets import (QCheckBox, QComboBox, QInputDialog, QLineEdit, QListWidget, QMenu, QMessageBox,
                             QTableWidget, QTreeWidget, QWidgetAction)


COLOR_FILA_A = "#FFF9D9"      # amarillo pálido
COLOR_FILA_B = "#E3ECF2"      # celeste grisáceo
COLOR_ALERTA = "#F8D7DA"      # rosado: filas con problemas (distinto de las dos franjas)


def aplicar_franjas(widget):
    """Pinta las filas de una grilla (tabla, árbol o lista) en colores alternados, con selección azul."""
    nombre = next(n for n, tipo in (("QTableWidget", QTableWidget), ("QTreeWidget", QTreeWidget), ("QListWidget",
QListWidget))
                  if isinstance(widget, tipo))      # el selector del estilo vale también para las subclases
    widget.setAlternatingRowColors(True)
    widget.setStyleSheet(f"""
        {nombre} {{
            background-color: {COLOR_FILA_A};
            alternate-background-color: {COLOR_FILA_B};
            gridline-color: #CBD2D6;
            color: #1B2631;
        }}
        {nombre}::item:selected {{
            background-color: #007bff;
            color: #ffffff;
        }}
    """)


class LineEditSeguro(QLineEdit):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.setStyleSheet("background-color: #ffffff; color: #000000; border: 1px solid #ccc; padding: 3px;")

    def keyPressEvent(self, event):
        if event.key() in (Qt.Key_Return, Qt.Key_Enter):
            event.ignore()
        else:
            super().keyPressEvent(event)


# ---------------------------------------------------------------------
# COMBOBOX CON CHECKBOXES MÚLTIPLES UNIVERSAL
# ---------------------------------------------------------------------
class ComboBoxMultiCheckbox(QComboBox):
    """Desplegable con casillas. Si se da `on_nueva`, el menú ofrece «Agregar nueva opción…».

    on_nueva(texto) debe guardar la opción donde corresponda y devolver el texto definitivo
    (o lanzar ValueError con el motivo si no es válida).
    """
    def __init__(self, items, parent=None, on_nueva=None):
        super().__init__(parent)
        self.items = items
        self.checkboxes = {}
        self._on_nueva = on_nueva
        self._separador = None

        self.setStyleSheet("background-color: #ffffff; color: #000000;")
        self.menu_desplegable = QMenu(self)

        for item_texto in self.items:
            self._crear_casilla(item_texto)

        if on_nueva is not None:
            self._separador = self.menu_desplegable.addSeparator()
            accion_nueva = self.menu_desplegable.addAction("➕ Agregar nueva opción…")
            accion_nueva.triggered.connect(self._pedir_nueva_opcion)

        self.setMaxVisibleItems(20)
        self.actualizar_texto_seleccionado()

    def _crear_casilla(self, texto):
        chk = QCheckBox(texto, self.menu_desplegable)
        chk.stateChanged.connect(self.actualizar_texto_seleccionado)
        self.checkboxes[texto] = chk

        action = QWidgetAction(self.menu_desplegable)
```

```
    action.setDefaultWidget(chk)
    if self._separador is not None:
        self.menu_desplegable.insertAction(self._separador, action)   # antes del «Agregar nueva…»
    else:
        self.menu_desplegable.addAction(action)
    return chk

def agregar_opcion(self, texto, marcar=False):
    """Agrega una opción al menú (si no estaba) y, con `marcar`, la deja elegida."""
    chk = self.checkboxes.get(texto) or self._crear_casilla(texto)
    if marcar:
        chk.setChecked(True)

def _pedir_nueva_opcion(self):
    texto, aceptado = QInputDialog.getText(self, "Nueva opción", "Nombre de la nueva opción:")
    if not aceptado or not texto.strip():
        return
    try:
        definitivo = self._on_nueva(texto)
    except ValueError as e:
        QMessageBox.warning(self, "Opción no válida", str(e))
        return
    self.agregar_opcion(definitivo, marcar=True)

def showPopup(self):
    point = self.mapToGlobal(QPoint(0, self.height()))
    self.menu_desplegable.exec_(point)

def actualizar_texto_seleccionado(self):
    seleccionadas = []
    chk_na = self.checkboxes.get("N/A")
    sender_chk = self.sender()

    if sender_chk == chk_na and chk_na and chk_na.isChecked():
        for texto, chk in self.checkboxes.items():
            if texto != "N/A":
                chk.blockSignals(True)
                chk.setChecked(False)
                chk.blockSignals(False)
    elif sender_chk and sender_chk != chk_na and chk_na and chk_na.isChecked():
        if sender_chk.isChecked():
            chk_na.blockSignals(True)
            chk_na.setChecked(False)
            chk_na.blockSignals(False)

    for texto, chk in self.checkboxes.items():
        if chk.isChecked():
            seleccionadas.append(texto)

    texto_final = ", ".join(seleccionadas) if seleccionadas else "N/A"
    if not seleccionadas and chk_na:
        chk_na.blockSignals(True)
        chk_na.setChecked(True)
        chk_na.blockSignals(False)
        texto_final = "N/A"

    self.blockSignals(True)
    self.clear()
    self.addItem(texto_final)
    self.setCurrentIndex(0)
    self.blockSignals(False)

def text(self):
    return self.itemText(0)

def set_seleccionadas(self, texto_guardado):
    if not texto_guardado:
        texto_guardado = "N/A"

    elementos = [e.strip() for e in str(texto_guardado).split(",")]
    for e in elementos:
        if e and e not in self.checkboxes:
            self.agregar_opcion(e)       # valor guardado que ya no está en la lista: se conserva
    for texto, chk in self.checkboxes.items():
        chk.blockSignals(True)
        chk.setChecked(texto in elementos)
        chk.blockSignals(False)

    if "N/A" in elementos and len(elementos) == 1:
        if "N/A" in self.checkboxes:
            self.checkboxes["N/A"].setChecked(True)
```

```
self.actualizar_texto_seleccionado()
```

interfaz\arbol_parametros.py

```
"""Árbol de la taxonomía con los parámetros propios de cada nodo.

Lo usan la ventana de parámetros del componente (consulta) y la ventana de
administración (edición). No tiene consultas propias: lee de db.py.
"""
from PyQt5.QtCore import Qt
from PyQt5.QtWidgets import QTreeWidget, QTreeWidgetItem

import db
from interfaz.widgets import aplicar_franjas

NOMBRES_NIVEL = ("Tipo", "Categoría", "Subcategoría", "Ítem")
ROL_RUTA = Qt.UserRole          # camino del nodo (tupla de nombres)
ROL_TIPO = Qt.UserRole + 1      # 'nodo' o 'param'
ROL_NOMBRE = Qt.UserRole + 2    # nombre del parámetro (solo en 'param')


class ArbolParametros(QTreeWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.setHeaderLabels(["Clasificación  /  📏 Parámetros propios", "Propios"])
        self.setColumnWidth(0, 430)
        aplicar_franjas(self)
        self.refrescar()

    def refrescar(self, abrir=None):
        """Reconstruye el árbol conservando los nodos abiertos.

        `abrir`: camino que se despliega y se deja seleccionado (si no existe, el nodo más
        profundo que sí exista de ese camino).
        """
        abiertos = set(self._rutas_abiertas())
        abrir = tuple(abrir) if abrir else ()
        abiertos |= {abrir[:k] for k in range(1, len(abrir) + 1)}
        self.clear()

        propios = {}
        for ruta, parametro in db.listar_plantillas():
            propios.setdefault(ruta, []).append(parametro)

        rutas = set()
        for fila in db.listar_catalogo():
            camino = tuple(x for x in fila if x)
            rutas |= {camino[:k] for k in range(1, len(camino) + 1)}
        for camino in propios:
            rutas |= {camino[:k] for k in range(1, len(camino) + 1)}

        nodos = {}
        # ordenar por camino deja cada padre antes que sus hijos
        for ruta in sorted(rutas, key=lambda r: tuple(x.lower() for x in r)):
            padre = nodos.get(ruta[:-1]) or self.invisibleRootItem()
            cantidad = len(propios.get(ruta, []))
            nodo = QTreeWidgetItem(padre, [ruta[-1], str(cantidad) if cantidad else ""])
            nodo.setData(0, ROL_RUTA, ruta)
            nodo.setData(0, ROL_TIPO, "nodo")
            nodo.setToolTip(0, f"{NOMBRES_NIVEL[len(ruta) - 1]}: {' › '.join(ruta)}")
            nodos[ruta] = nodo
            for parametro in propios.get(ruta, []):
                hoja = QTreeWidgetItem(nodo, ["📏 " + parametro, ""])
                hoja.setData(0, ROL_RUTA, ruta)
                hoja.setData(0, ROL_TIPO, "param")
                hoja.setData(0, ROL_NOMBRE, parametro)

        for ruta, nodo in nodos.items():
            nodo.setExpanded(ruta in abiertos)
        for k in range(len(abrir), 0, -1):
            if abrir[:k] in nodos:
                self.setCurrentItem(nodos[abrir[:k]])
                break

    def _rutas_abiertas(self):
        pila = [self.topLevelItem(i) for i in range(self.topLevelItemCount())]
        while pila:
            nodo = pila.pop()
            if nodo.data(0, ROL_TIPO) == "nodo" and nodo.isExpanded():
                yield nodo.data(0, ROL_RUTA)
            pila.extend(nodo.child(i) for i in range(nodo.childCount()))

    def nodo_actual(self):
```

```
    """Camino del nodo elegido (si hay un parámetro elegido, el de su nodo), o None."""
    item = self.currentItem()
    return tuple(item.data(0, ROL_RUTA)) if item else None

def parametro_actual(self):
    """(camino, nombre) si lo elegido es un parámetro; si no, None."""
    item = self.currentItem()
    if item and item.data(0, ROL_TIPO) == "param":
        return tuple(item.data(0, ROL_RUTA)), item.data(0, ROL_NOMBRE)
    return None
```

interfaz\ventana_principal.py

```
"""Ventana principal: búsqueda, listado y acciones sobre componentes."""
import qrcode

from PyQt5.QtCore import QSize, Qt
from PyQt5.QtGui import QFont, QImage, QPixmap
from PyQt5.QtWidgets import (
    QComboBox,
    QDialog,
    QFrame,
    QGroupBox,
    QHBoxLayout,
    QHeaderView,
    QInputDialog,
    QLabel,
    QLineEdit,
    QMainWindow,
    QMessageBox,
    QPushButton,
    QTableWidget,
    QTableWidgetItem,
    QVBoxLayout,
    QWidget
)

import almacenamiento
import db
from interfaz.dialogo_capital import DialogoCapital
from interfaz.dialogo_catalogos import DialogoAdministrarCatalogos
from interfaz.dialogo_componente import DialogoNuevoComponente
from interfaz.dialogo_etiquetas import DialogoImpresionEtiquetas
from interfaz.dialogo_parametros import DialogoParametrosElectricos
from interfaz.visor_pdf import DialogoVisorPDF
from interfaz.widgets import aplicar_franjas
from utilidades import limpiar_prefijo, sin_comodin


# ---------------------------------------------------------------------
# VENTANA PRINCIPAL DE LA APLICACIÓN
# ---------------------------------------------------------------------
class AppInventario(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Gestión de Inventario de Componentes")
        self.setWindowState(Qt.WindowMaximized)   # siempre abre maximizada
        self.selected_component = None
        self.ventanas_hijas_principales = []
        self.init_ui()
        self.cargar_comboboxes_cascada()
        self.filtrar_tabla()

    def init_ui(self):
        main_widget = QWidget()
        main_layout = QVBoxLayout()

        panel_preview = QFrame()
        panel_preview.setFrameShape(QFrame.StyledPanel)
        panel_preview.setStyleSheet("background-color: #f8f9fa;")
        layout_preview = QHBoxLayout()

        self.lbl_imagen = QLabel("Sin Imagen")
        self.lbl_imagen.setFixedSize(140, 110)
        self.lbl_imagen.setStyleSheet("border: 1px solid #ccc; background-color: #ffffff;")
        self.lbl_imagen.setAlignment(Qt.AlignCenter)
        layout_preview.addWidget(self.lbl_imagen)

        layout_info = QVBoxLayout()
        self.lbl_titulo = QLabel("Seleccione un componente...")
        self.lbl_titulo.setFont(QFont("Arial", 12, QFont.Bold))
        self.lbl_detalles = QLabel("Categoría: - | Subcat: - | Ítem: - \nUbicación: - | Stock: - | Precio: $\
nChip: - | Interfaz: -\nDescripción: -")
        self.lbl_detalles.setStyleSheet("color: #495057;")
        self.lbl_detalles.setWordWrap(True)      # sin esto, una línea larga fija el ancho mínimo de la ventana y
no entra en pantallas chicas
        layout_info.addWidget(self.lbl_titulo)
        layout_info.addWidget(self.lbl_detalles)
        layout_preview.addLayout(layout_info)

        # --- CREACIÓN DEL WIDGET PARA EL CÓDIGO QR ---
        self.lbl_qr = QLabel("Sin QR")
```

```
        self.lbl_qr.setFixedSize(110, 110)
        self.lbl_qr.setStyleSheet("border: 1px solid #ccc; background-color: #ffffff;")
        self.lbl_qr.setAlignment(Qt.AlignCenter)
        layout_preview.addWidget(self.lbl_qr)
        # -----------------------------------------
        layout_docs = QVBoxLayout()
        tamanio_boton = QSize(120, 26)

        # Botón de Datasheets
        self.btn_open_ds = QPushButton("📄 Ver Datasheets")
        self.btn_open_ds.setFixedSize(tamanio_boton)
        self.btn_open_ds.setStyleSheet("background-color: #17a2b8; color: white; font-weight: bold; padding:
6px;")
        self.btn_open_ds.clicked.connect(lambda: self.abrir_archivo_asociado("datasheet"))

        # Botón de tutoriales
        self.btn_open_tut = QPushButton("📘 Ver Tutoriales")
        self.btn_open_tut.setFixedSize(tamanio_boton)
        self.btn_open_tut.setStyleSheet("background-color: #17a2b8; color: white; font-weight: bold; padding:
6px;")
        self.btn_open_tut.clicked.connect(lambda: self.abrir_archivo_asociado("tutorial"))

        # Botón de Esquemas
        self.btn_open_esq = QPushButton(" Ver Esquemas")
        self.btn_open_esq.setFixedSize(tamanio_boton)
        self.btn_open_esq.setStyleSheet("background-color: #17a2b8; color: white; font-weight: bold; padding:
6px;")
        self.btn_open_esq.clicked.connect(lambda: self.abrir_archivo_asociado("esquema"))

        # Botón de Parámetros Avanzados (Único y correctamente conectado)
        self.btn_ver_parametros = QPushButton("⚙ Ver Parámetros")
        self.btn_ver_parametros.setFixedSize(tamanio_boton)
        self.btn_ver_parametros.setStyleSheet("background-color: #17a2b8; color: white; font-weight: bold;
padding: 6px;")
        self.btn_ver_parametros.clicked.connect(self.abrir_dialogo_parametros_avanzados)

        # layout_docs.addStretch()
        layout_docs.addWidget(self.btn_open_ds)
        layout_docs.addWidget(self.btn_open_tut)
        layout_docs.addWidget(self.btn_open_esq) # <--- Añadido al panel lateral
        layout_docs.addWidget(self.btn_ver_parametros) # <--- Añadido al panel lateral correctamente

        layout_preview.addLayout(layout_docs)
        panel_preview.setLayout(layout_preview)
        main_layout.addWidget(panel_preview)

        box_filtros = QGroupBox("Filtros de Seleccion Indepediente")
        layout_filtros = QHBoxLayout()

        self.combo_tipo = QComboBox(); self.combo_tipo.setMaximumWidth(200);
self.combo_tipo.setStyleSheet("background-color: #ffffff; color: #000000;")
        self.combo_cat = QComboBox(); self.combo_cat.setMaximumWidth(200);
self.combo_cat.setStyleSheet("background-color: #ffffff; color: #000000;")
        self.combo_subcat = QComboBox(); self.combo_subcat.setMaximumWidth(200);
self.combo_subcat.setStyleSheet("background-color: #ffffff; color: #000000;")
        self.combo_item = QComboBox(); self.combo_item.setMaximumWidth(200);
self.combo_item.setStyleSheet("background-color: #ffffff; color: #000000;")

        self.combo_tipo.currentIndexChanged.connect(self.on_tipo_changed)
        self.combo_cat.currentIndexChanged.connect(self.on_cat_changed)
        self.combo_subcat.currentIndexChanged.connect(self.on_subcat_changed)
        self.combo_item.currentIndexChanged.connect(self.filtrar_tabla)

        layout_filtros.addWidget(QLabel("1. Tipo:"))
        layout_filtros.addWidget(self.combo_tipo)
        layout_filtros.addWidget(QLabel("2. Cat:"))
        layout_filtros.addWidget(self.combo_cat)
        layout_filtros.addWidget(QLabel("3. Sub:"))
        layout_filtros.addWidget(self.combo_subcat)
        layout_filtros.addWidget(QLabel("4. Ítem:"))
        layout_filtros.addWidget(self.combo_item)
        box_filtros.setLayout(layout_filtros)
        main_layout.addWidget(box_filtros)

        layout_acciones = QHBoxLayout()
        btn_eliminar = QPushButton("❌ Eliminar")
        btn_eliminar.setStyleSheet("background-color: #dc3545; color: white; font-weight: bold; padding: 6px;")
        btn_eliminar.clicked.connect(self.eliminar_registro)
        layout_acciones.addWidget(btn_eliminar)

        self.txt_buscar = QLineEdit()
        self.txt_buscar.setPlaceholderText("🔍 Buscar por Nombre, Descripción, Fabricante, Chip...")
```

```
        self.txt_buscar.setStyleSheet("background-color: #ffffff; color: #000000;")
        self.txt_buscar.textChanged.connect(self.filtrar_tabla)
        layout_acciones.addWidget(self.txt_buscar)

        btn_nuevo = QPushButton("+ Nuevo")
        btn_nuevo.setStyleSheet("background-color: #28a745; color: white; font-weight: bold; padding: 6px;")
        btn_nuevo.clicked.connect(self.abrir_nuevo_dialogo)

        btn_editar = QPushButton("✏ Editar")
        btn_editar.setStyleSheet("background-color: #ffc107; color: #212529; font-weight: bold; padding: 6px;")
        btn_editar.clicked.connect(self.abrir_editar_dialogo)

        btn_imprimir = QPushButton("🖨️ Imprimir Selección")
        btn_imprimir.setStyleSheet("background-color: #000080; color: white; font-weight: bold; padding: 6px;")
        btn_imprimir.clicked.connect(self.abrir_impresion)

        btn_catalogos = QPushButton("📚 Listas y Catálogo")
        btn_catalogos.setStyleSheet("background-color: #6f42c1; color: white; font-weight: bold; padding: 6px;")
        btn_catalogos.clicked.connect(self.abrir_administrador_catalogos)

        btn_capital = QPushButton("💰 Capital")
        btn_capital.setStyleSheet("background-color: #198754; color: white; font-weight: bold; padding: 6px;")
        btn_capital.clicked.connect(self.abrir_capital)

        layout_acciones.addWidget(btn_nuevo)
        layout_acciones.addWidget(btn_editar)
        layout_acciones.addWidget(btn_catalogos)
        layout_acciones.addWidget(btn_capital)
        layout_acciones.addWidget(btn_imprimir)
        main_layout.addLayout(layout_acciones)

        self.table = QTableWidget()
        self.table.setColumnCount(19)
        self.table.setHorizontalHeaderLabels([
            "ID", "Nombre", "Chip Principal", "Tipo", "Categoría", "Subcategoría", "Ítem Específico",
            "Descripción", "Fabricante", "Interfaz", "Pines", "Stock", "Precio ($)", "Ubicación", "QR Path",
            "Precio Ext.", "Prov. Ext.", "Precio Loc.", "Prov. Loc."
        ])
        self.table.setColumnHidden(0, True)
        self.table.setColumnHidden(1, True)
        self.table.setColumnHidden(14, True)
        self.table.horizontalHeader().setSectionResizeMode(QHeaderView.Interactive)
        self.table.setSelectionBehavior(QTableWidget.SelectRows)
        self.table.itemSelectionChanged.connect(self.on_row_selected)

        header = self.table.horizontalHeader()
        header.setStyleSheet("""
            QHeaderView::section {
                background-color: #000000 !important;
                color: #FFFFFF !important;
                font-weight: bold;
                font-size: 11px;
                padding: 6px;
                border: 1px solid #333333;
            }
        """)

        aplicar_franjas(self.table)

        main_layout.addWidget(self.table)
        main_widget.setLayout(main_layout)
        self.setCentralWidget(main_widget)

    def cargar_comboboxes_cascada(self):
        self.combo_tipo.blockSignals(True)
        self.combo_tipo.clear()
        self.combo_tipo.addItem("-- Todos los Tipos --")
        for t in db.listar_tipos():
            self.combo_tipo.addItem(t)
        self.combo_tipo.blockSignals(False)
        self.on_tipo_changed()

    def on_tipo_changed(self):
        tipo = limpiar_prefijo(self.combo_tipo.currentText())
        self.combo_subcat.blockSignals(True); self.combo_subcat.clear(); self.combo_subcat.addItem("-- Todas las
Subcat --"); self.combo_subcat.blockSignals(False)
        self.combo_item.blockSignals(True); self.combo_item.clear(); self.combo_item.addItem("-- Todos los Ítems
--"); self.combo_item.blockSignals(False)

        categorias = db.listar_categorias(sin_comodin(tipo, "-- Todos los Tipos --"))

        self.combo_cat.blockSignals(True)
```

```
        self.combo_cat.clear()
        self.combo_cat.addItem("-- Todas las Categorías --")
        self.combo_cat.addItems(categorias)
        self.combo_cat.blockSignals(False)

        # Filtra inmediatamente al cambiar el Tipo
        self.filtrar_tabla()

    def on_cat_changed(self):
        tipo = limpiar_prefijo(self.combo_tipo.currentText())
        cat = self.combo_cat.currentText()
        self.combo_item.blockSignals(True); self.combo_item.clear(); self.combo_item.addItem("-- Todos los Ítems
--"); self.combo_item.blockSignals(False)

        subcats = db.listar_subcategorias(sin_comodin(tipo, "-- Todos los Tipos --"), sin_comodin(cat, "-- Todas
las Categorías --"))

        self.combo_subcat.blockSignals(True)
        self.combo_subcat.clear()
        self.combo_subcat.addItem("-- Todas las Subcat --")
        self.combo_subcat.addItems(subcats)
        self.combo_subcat.blockSignals(False)

        # Filtra inmediatamente al cambiar la Categoría
        self.filtrar_tabla()

    def on_subcat_changed(self):
        tipo = limpiar_prefijo(self.combo_tipo.currentText())
        cat = self.combo_cat.currentText()
        subcat = self.combo_subcat.currentText()

        items = db.listar_items(
            tipo=sin_comodin(tipo, "-- Todos los Tipos --"),
            cat=sin_comodin(cat, "-- Todas las Categorías --"),
            subcat=sin_comodin(subcat, "-- Todas las Subcat --"),
        )

        self.combo_item.blockSignals(True)
        self.combo_item.clear()
        self.combo_item.addItem("-- Todos los Ítems --")
        self.combo_item.addItems(items)
        self.combo_item.blockSignals(False)

        # Filtra inmediatamente al cambiar la Subcategoría
        self.filtrar_tabla()

    def filtrar_tabla(self):
        tipo = limpiar_prefijo(self.combo_tipo.currentText())
        cat = self.combo_cat.currentText()
        subcat = self.combo_subcat.currentText()
        item = self.combo_item.currentText()
        busqueda = self.txt_buscar.text().strip()

        rows = db.buscar_componentes(
            tipo=sin_comodin(tipo, "-- Todos los Tipos --"),
            cat=sin_comodin(cat, "-- Todas las Categorías --"),
            subcat=sin_comodin(subcat, "-- Todas las Subcat --"),
            item=sin_comodin(item, "-- Todos los Ítems --"),
            texto=busqueda,
        )

        self.table.setRowCount(0)
        for row_idx, row_data in enumerate(rows):
            self.table.insertRow(row_idx)

            # El índice 6 es el ítem específico exacto de la base de datos
            item_especifico_exacto = row_data[6] if len(row_data) > 6 and row_data[6] else ""

            for col_idx, value in enumerate(row_data):
                val_str = str(value if value is not None else "")
                if col_idx in [12, 15, 17]: # Precios
                    try:
                        val_str = f"{float(val_str):.2f}"
                    except ValueError:
                        pass
                item_table = QTableWidgetItem(val_str)
                item_table.setFlags(item_table.flags() & ~Qt.ItemIsEditable)

                # INYECTAMOS EL ÍTEM ESPECÍFICO DE FORMA OCULTA EN CADA CELDA DE LA FILA
                item_table.setData(Qt.UserRole, item_especifico_exacto)

                self.table.setItem(row_idx, col_idx, item_table)
```

```
        self.table.resizeColumnsToContents()

    def limpiar_y_verificar_archivos(self, componente_id):
        if not componente_id:
            return

        # Obtener todos los archivos asociados al componente
        registros = db.listar_archivos_con_id(componente_id)

        ids_a_borrar = []
        for reg_id, tipo, ruta in registros:
            # Si la ruta está vacía o el archivo no existe físicamente en disco
            if not ruta or not almacenamiento.existe(ruta):
                ids_a_borrar.append(reg_id)

        # Purgar de la base de datos las referencias rotas de forma totalmente transparente
        if ids_a_borrar:
            db.eliminar_archivos_por_id(ids_a_borrar)

        # Volver a consultar los archivos limpios para la interfaz
        self.archivos_seleccionado = almacenamiento.resolver_archivos(db.listar_archivos(componente_id))

    def on_row_selected(self):
        selected_rows = self.table.selectionModel().selectedRows()

        # Si no hay nada seleccionado o se marcan múltiples filas, limpiamos la vista previa y la imagen
        if not selected_rows or len(selected_rows) > 1:
            self.selected_component = None
            self.lbl_titulo.setText("Seleccione un componente...")
            self.lbl_detalles.setText("Categoría: - | Subcat: - | Ítem: - \nUbicación: - | Stock: - | Precio: $\
nChip: - | Interfaz: -\nDescripción: -")
            self.lbl_imagen.clear()
            self.lbl_imagen.setText("Sin Imagen")
            self.lbl_qr.clear()          # <--- Limpiar QR al deseleccionar
            self.lbl_qr.setText("Sin QR")  # <--- Texto por defecto
            return

        row = selected_rows[0].row()
        item_id = self.table.item(row, 0).text()

        # ---> LLAMADA CLAVE: Se ejecuta de forma invisible y automática al hacer clic <---
        self.limpiar_y_verificar_archivos(item_id)

        componente = db.obtener_componente(item_id)
        self.archivos_seleccionado = almacenamiento.resolver_archivos(db.listar_archivos(item_id))
        self.params_seleccionado = db.listar_parametros(item_id)

        if componente:
            self.selected_component = componente

            str_params = ""
            if self.params_seleccionado:
                lista_p = [f"{k}: {v}" for k, v in self.params_seleccionado if v.strip()]
                if lista_p:
                    str_params = f" | Parámetros: [{', '.join(lista_p)}]"

            self.lbl_titulo.setText(f"{self.selected_component.get('nombre', '')} -
{self.selected_component.get('fabricante', '')}")

            try:
                precio_ext = float(self.selected_component.get('precio_exterior', 0) or 0)
            except (ValueError, TypeError):
                precio_ext = 0.00

            try:
                precio_loc = float(self.selected_component.get('precio_local', 0) or 0)
            except (ValueError, TypeError):
                precio_loc = 0.00

            prov_ext = self.selected_component.get('proveedor_exterior', '-') or '-'
            prov_loc = self.selected_component.get('proveedor_local', '-') or '-'

            detalles = (
                f"Categoría: {self.selected_component.get('categoria', '')} | Subcat:
{self.selected_component.get('subcategoria', '')} | Ítem: {self.selected_component.get('item_especifico', '')}\n"
                f"Ubicación: {self.selected_component.get('ubicacion', '')} | Stock:
{self.selected_component.get('stock', 0)} | Ext: ${precio_ext:.2f} ({prov_ext}) | Loc: ${precio_loc:.2f}
({prov_loc}){str_params}\n"
                f"Fabricante: {self.selected_component.get('fabricante', 'N/A')} | Chip:
{self.selected_component.get('chip_principal', 'N/A')}\n"
                f"Interfaz: {self.selected_component.get('interfaz_comunicacion', 'N/A')} | Pines:
{self.selected_component.get('pines_relevantes', 'N/A')}\n"
```

```
                f"Descripción: {self.selected_component.get('descripcion', '')[:45]}"
            )
            self.lbl_detalles.setText(detalles)

            foto_ruta = ""
            for t, nom, ruta in self.archivos_seleccionado:
                if t == 'imagen' and ruta and almacenamiento.existe(ruta):
                    foto_ruta = ruta
                    break

            if foto_ruta:
                pixmap = QPixmap(foto_ruta).scaled(140, 110, Qt.KeepAspectRatio, Qt.SmoothTransformation)
                self.lbl_imagen.setPixmap(pixmap)
            else:
                self.lbl_imagen.clear()
                self.lbl_imagen.setText("Sin Imagen")

            chip = str(self.selected_component.get('chip_principal', '')).strip() or "N/A"
            item = str(self.selected_component.get('item_especifico', '')).strip()
            if not item or item.startswith("--"):
                item = "N/A"
            desc = str(self.selected_component.get('descripcion', '')[:45]).strip()[:45] or "N/A"
            stock = str(self.selected_component.get('stock', '0')).strip() or "0"
            ubicacion = str(self.selected_component.get('ubicacion', '')).strip() or "N/A"

            qr_texto = f"CHIP: {chip} | ITEM: {item} | DESC: {desc} | STOCK: {stock} | UBIC: {ubicacion}"

            try:
                import qrcode
                import io
                from PyQt5.QtGui import QImage

                qr = qrcode.QRCode(version=1, box_size=3, border=1)
                qr.add_data(qr_texto)
                qr.make(fit=True)
                img_qr = qr.make_image(fill_color="black", back_color="white")

                buffer = io.BytesIO()
                img_qr.save(buffer, format="PNG")
                qimage = QImage.fromData(buffer.getvalue())
                pixmap_qr = QPixmap.fromImage(qimage)
                self.lbl_qr.setPixmap(pixmap_qr.scaled(100, 100, Qt.KeepAspectRatio, Qt.SmoothTransformation))
            except Exception as e:
                self.lbl_qr.setText("Error QR")

    def abrir_archivo_asociado(self, tipo):
        # Validación de seguridad: Verificar si hay un componente seleccionado en la grilla
        if not self.selected_component:
            QMessageBox.warning(self, "Atención", f"Por favor, seleccione un componente de la grilla para ver sus
{tipo}s.")
            return

        if not hasattr(self, 'archivos_seleccionado') or not self.archivos_seleccionado:
            QMessageBox.information(self, "Información", f"No hay archivos de tipo '{tipo}' adjuntos para este
componente.")
            return

        # Buscamos tanto la ruta como el nombre amigable que le diste al archivo
        filtrados = [(nom, ruta) for t, nom, ruta in self.archivos_seleccionado if t == tipo and
almacenamiento.existe(ruta)]

        if not filtrados:
            QMessageBox.information(self, "Información", f"No se encontraron archivos físicos de tipo '{tipo}'.")
            return

        # CASO 1: Si hay un solo archivo, lo abre directamente como antes
        if len(filtrados) == 1:
            nombre_amigable, ruta_archivo = filtrados[0]
            visor = DialogoVisorPDF(ruta_archivo, None, self, titulo=nombre_amigable)
            self.ventanas_hijas_principales.append(visor)
            visor.show()
            return

        # CASO 2: Si hay MÚLTIPLES archivos, mostramos una ventanita de selección con sus nombres amigables
        nombres = [item[0] for item in filtrados]
        item_seleccionado, ok = QInputDialog.getItem(
            self,
            f"Seleccionar {tipo.capitalize()}",
            f"Este componente tiene varios {tipo}s. Elija uno para abrir:",
            nombres,
            0,
            False
```

```
        )

        if ok and item_seleccionado:
            # Buscamos la ruta correspondiente al nombre amigable elegido
            ruta_elegida = next(ruta for nom, ruta in filtrados if nom == item_seleccionado)
            visor = DialogoVisorPDF(ruta_elegida, None, self, titulo=item_seleccionado)
            self.ventanas_hijas_principales.append(visor)
            visor.show()

    def abrir_nuevo_dialogo(self):
        """Flujo exclusivo y limpio para crear un componente nuevo."""
        dialogo = DialogoNuevoComponente(datos_componente=None, parent=self)
        self.ventanas_hijas_principales.append(dialogo)
        dialogo.show()
        dialogo.finished.connect(lambda: (self.cargar_comboboxes_cascada(), self.filtrar_tabla()))

    def abrir_administrador_catalogos(self):
        """Ventana para agregar, renombrar y quitar entradas del catálogo y de las listas de los desplegables."""
        DialogoAdministrarCatalogos(parent=self).exec_()
        self.cargar_comboboxes_cascada()
        self.filtrar_tabla()

    def abrir_capital(self):
        """Muestra el capital en existencia en una grilla (con opción de exportarlo a Excel)."""
        DialogoCapital(self).exec_()

    def abrir_dialogo_parametros_avanzados(self):
        """Consulta los parámetros guardados del componente seleccionado, junto al árbol de la taxonomía."""
        if not self.selected_component:
            QMessageBox.warning(self, "Atención", "Por favor, seleccione un componente de la grilla para ver sus
parámetros.")
            return
        c = self.selected_component
        ruta = db.ruta_de_clasificacion(c.get('tipo_componente'), c.get('categoria'), c.get('subcategoria'),
c.get('item_especifico'))
        dialogo = DialogoParametrosElectricos(
            ruta=ruta,
            parametros_existentes=getattr(self, 'params_seleccionado', []),
            solo_lectura=True,
            parent=self
        )
        dialogo.exec_()

    def abrir_editar_dialogo(self):
        """Flujo exclusivo para editar un componente existente de la grilla."""
        if not self.selected_component:
            QMessageBox.warning(self, "Atención", "Por favor, seleccione un componente de la grilla para editar.")
            return

        dialogo = DialogoNuevoComponente(datos_componente=self.selected_component, parent=self)
        self.ventanas_hijas_principales.append(dialogo)
        dialogo.show()
        dialogo.finished.connect(lambda: (self.cargar_comboboxes_cascada(), self.filtrar_tabla()))

    def eliminar_registro(self):
        if not self.selected_component:
            QMessageBox.warning(self, "Atención", "Por favor, seleccione un componente para eliminar.")
            return
        confirm = QMessageBox.question(self, "Confirmar", f"¿Eliminar {self.selected_component['nombre']}?",
QMessageBox.Yes | QMessageBox.No)
        if confirm == QMessageBox.Yes:
            db.eliminar_componente(self.selected_component['id'])
            self.selected_component = None
            self.lbl_titulo.setText("Seleccione un componente...")
            self.lbl_detalles.setText("Categoría: - | Subcat: - | Ítem: - \nUbicación: - | Stock: - | Precio: $\
nChip: - | Interfaz: -\nDescripción: -")
            self.lbl_imagen.clear()
            self.lbl_imagen.setText("Sin Imagen")
            self.cargar_comboboxes_cascada()
            self.filtrar_tabla()

    def abrir_impresion(self):
        selected_rows = self.table.selectionModel().selectedRows()
        if not selected_rows:
            QMessageBox.warning(self, "Atención", "Seleccione al menos un componente en la tabla para imprimir.")
            return

        componentes_imprimir = []
        for r in selected_rows:
            item_id = self.table.item(r.row(), 0).text()
            nombre = self.table.item(r.row(), 1).text()
            fabricante = self.table.item(r.row(), 8).text()
```

```
            encapsulado = "N/A"
            ubicacion = self.table.item(r.row(), 13).text()
            componentes_imprimir.append({
                'id': item_id, 'nombre': nombre, 'ubicacion': ubicacion,
                'fabricante': fabricante, 'encapsulado': encapsulado
            })

        dialogo = DialogoImpresionEtiquetas(componentes_imprimir, self)
        self.ventanas_hijas_principales.append(dialogo)
        dialogo.show()
```

interfaz\dialogo_componente.py

```
"""Diálogo de alta y edición de un componente."""
import qrcode

from PyQt5.QtCore import Qt, QTimer
from PyQt5.QtGui import QFont, QPixmap, QTextCharFormat, QTextCursor
from PyQt5.QtWidgets import (
    QButtonGroup,
    QCheckBox,
    QComboBox,
    QDialog,
    QFileDialog,
    QFrame,
    QGridLayout,
    QHBoxLayout,
    QLabel,
    QMessageBox,
    QPushButton,
    QRadioButton,
    QScrollArea,
    QTextEdit,
    QVBoxLayout,
    QWidget
)

import almacenamiento
import db
from interfaz.dialogo_parametros import DialogoParametrosElectricos
from interfaz.editor_imagen import DialogoEditorImagen
from interfaz.visor_pdf import DialogoVisorPDF
from interfaz.widgets import ComboBoxMultiCheckbox, LineEditSeguro
from utilidades import limpiar_prefijo, sin_comodin


# ---------------------------------------------------------------------
# VENTANA: ALTA Y EDICIÓN DE COMPONENTE ANTERIOR
# ---------------------------------------------------------------------
class DialogoNuevoComponente(QDialog):
    def __init__(self, datos_componente=None, parent=None):
        super().__init__(parent)
        self.datos_componente = datos_componente
        self.is_editing = datos_componente is not None

        self.setWindowFlags(Qt.Window | Qt.WindowMaximizeButtonHint | Qt.WindowMinimizeButtonHint |
Qt.WindowCloseButtonHint)
        self.setWindowTitle("✏ Editar Componente" if self.is_editing else "+ Nuevo Componente")
        self.setWindowState(Qt.WindowMaximized)   # siempre abre maximizada

        self.archivos_lista = []
        self.parametros_electricos_temp = []
        self.ventanas_hijas_abiertas = []
        self._estilos_originales = {}      # estilo de cada campo, para atenuarlo al deshabilitarlo
        self._cargando_datos = False       # True mientras se vuelcan los datos guardados al formulario
        self._marco_chip_encendido = False
        self._timer_chip = QTimer(self)    # hace parpadear el marco del Chip Principal al crear un componente
        self._timer_chip.timeout.connect(self._alternar_marco_chip)

        self.init_ui()
        self.cargar_combos_enlazados()

        if self.is_editing:
            self._cargando_datos = True
            try:
                self.cargar_datos_existentes()
            finally:
                self._cargando_datos = False
            self.bloquear_campos_completos()
        else:
            self.actualizar_habilitacion_campos()

        self.actualizar_qr_en_vivo()

    def init_ui(self):
        main_layout = QVBoxLayout(self)
        main_layout.setContentsMargins(0, 0, 0, 0)

        scroll_area = QScrollArea()
        scroll_area.setWidgetResizable(True)
        scroll_area.setFrameShape(QFrame.NoFrame)
```

```
        container_widget = QWidget()
        container_widget.setStyleSheet("background-color: #f8f9fa;")

        layout_principal = QVBoxLayout(container_widget)
        layout_principal.setContentsMargins(25, 20, 25, 20)
        layout_principal.setSpacing(15)
        self._construir_cabecera(layout_principal)
        self._construir_checkbox_edicion(layout_principal)
        self._construir_clasificacion(layout_principal)
        self._construir_proveedores(layout_principal)
        self._construir_precios(layout_principal)
        self._construir_obsolescencia(layout_principal)
        self._construir_boton_parametros(layout_principal)
        self._construir_archivos(layout_principal)
        self._construir_boton_guardar(layout_principal)
        self._activar_mayusculas()

        scroll_area.setWidget(container_widget)
        main_layout.addWidget(scroll_area)
        self.refrescar_combos_multianexo()

    def _construir_cabecera(self, layout_principal):
        """Miniatura, título del componente y vista previa del QR."""
        header_layout = QHBoxLayout()
        self.lbl_miniatura = QLabel("Sin Imagen")
        self.lbl_miniatura.setFixedSize(110, 110)
        self.lbl_miniatura.setAlignment(Qt.AlignCenter)
        self.lbl_miniatura.setStyleSheet("background-color: #ffffff; color: #555; font-size: 11px; border: 1px
solid #ccc;")
        header_layout.addWidget(self.lbl_miniatura)

        self.lbl_nombre_titulo = QLabel("NOMBRE DEL COMPONENTE")
        self.lbl_nombre_titulo.setFont(QFont("Arial", 18, QFont.Bold))
        self.lbl_nombre_titulo.setAlignment(Qt.AlignCenter)
        self.lbl_nombre_titulo.setStyleSheet("color: #000000;")
        header_layout.addWidget(self.lbl_nombre_titulo)

        qr_container = QVBoxLayout()
        qr_container.setAlignment(Qt.AlignCenter)
        self.lbl_qr_preview = QLabel("QR Preview")
        self.lbl_qr_preview.setFixedSize(110, 110)
        self.lbl_qr_preview.setAlignment(Qt.AlignCenter)
        self.lbl_qr_preview.setStyleSheet("background-color: #ffffff; color: #555; font-size: 10px; border: 1px
solid #ccc;")
        lbl_qr_titulo = QLabel("Código QR Etiqueta")
        lbl_qr_titulo.setFont(QFont("Arial", 8, QFont.Bold))
        lbl_qr_titulo.setAlignment(Qt.AlignCenter)
        qr_container.addWidget(self.lbl_qr_preview)
        qr_container.addWidget(lbl_qr_titulo)
        header_layout.addLayout(qr_container)

        layout_principal.addLayout(header_layout)
        layout_principal.addSpacing(10)

    def _construir_checkbox_edicion(self, layout_principal):
        """Casilla para habilitar la edición completa (solo al editar)."""
        if self.is_editing:
            self.chk_habilitar_edicion = QCheckBox("🔓 Habilitar edición completa (permitir modificar campos
llenos)")
            self.chk_habilitar_edicion.setFont(QFont("Arial", 9, QFont.Bold))
            self.chk_habilitar_edicion.setStyleSheet("color: #007bff; margin-bottom: 5px;")
            self.chk_habilitar_edicion.stateChanged.connect(self.toggle_habilitar_edicion)
            layout_principal.addWidget(self.chk_habilitar_edicion)

    def _construir_clasificacion(self, layout_principal):
        """Clasificación, descripción y datos técnicos en cuadrícula."""
        self.combo_tipo = QComboBox(); self.combo_tipo.setEditable(True);
self.combo_tipo.setPlaceholderText("Seleccione Tipo..."); self.combo_tipo.setMaximumWidth(280);
self.combo_tipo.setStyleSheet("background-color: #ffffff; color: #000000;")
        self.combo_cat = QComboBox(); self.combo_cat.setEditable(True);
self.combo_cat.setPlaceholderText("Seleccione Categoría..."); self.combo_cat.setMaximumWidth(280);
self.combo_cat.setStyleSheet("background-color: #ffffff; color: #000000;")
        self.combo_subcat = QComboBox(); self.combo_subcat.setEditable(True);
self.combo_subcat.setPlaceholderText("Seleccione Subcategoría..."); self.combo_subcat.setMaximumWidth(280);
self.combo_subcat.setStyleSheet("background-color: #ffffff; color: #000000;")
        self.combo_item = QComboBox(); self.combo_item.setEditable(True);
self.combo_item.setPlaceholderText("Seleccione Ítem Específico..."); self.combo_item.setMaximumWidth(280);
self.combo_item.setStyleSheet("background-color: #ffffff; color: #000000;")

        self.combo_tipo.currentIndexChanged.connect(self.on_tipo_changed)
        self.combo_cat.currentIndexChanged.connect(self.on_cat_changed)
        self.combo_subcat.currentIndexChanged.connect(self.on_subcat_changed)
```

```
        self.combo_item.currentIndexChanged.connect(self.actualizar_qr_en_vivo)
        # Los cuatro combos de clasificación gobiernan qué campos están habilitados al crear un componente.
        self.combo_item.currentIndexChanged.connect(self.actualizar_habilitacion_campos)
        for combo in (self.combo_tipo, self.combo_cat, self.combo_subcat, self.combo_item):
            combo.editTextChanged.connect(self.actualizar_habilitacion_campos)

        self.txt_descripcion = QTextEdit()
        self.txt_descripcion.setPlaceholderText("Ej: Conv. Analógico-Digital I2C 16-bit...")
        self.txt_descripcion.setFixedHeight(60)
        self.txt_descripcion.setMaximumWidth(280)
        self.txt_descripcion.setStyleSheet("background-color: #ffffff; color: #000000;")
        self.txt_descripcion.setLineWrapMode(QTextEdit.NoWrap)
        self.txt_descripcion.textChanged.connect(self.formatear_descripcion_en_vivo)

        self.lbl_contador_desc = QLabel("Max caracteres imprimibles: 45 | Actuales: 0")
        self.lbl_contador_desc.setFont(QFont("Arial", 8, QFont.Bold))
        self.lbl_contador_desc.setStyleSheet("color: #28a745;")

        layout_desc_container = QVBoxLayout()
        layout_desc_container.setSpacing(2)
        layout_desc_container.addWidget(self.txt_descripcion)
        layout_desc_container.addWidget(self.lbl_contador_desc)

        self.combo_fabricante = QComboBox()
        self.combo_fabricante.setEditable(True)
        self.combo_fabricante.addItems(db.listar_valores('fabricante'))
        self.combo_fabricante.setMaximumWidth(280)
        self.combo_fabricante.setStyleSheet("background-color: #ffffff; color: #000000;")

        self.txt_chip = LineEditSeguro()
        self.txt_chip.setPlaceholderText("Ej: HD44780")
        self.txt_chip.setMaximumWidth(280)
        self.txt_chip.setStyleSheet("background-color: #ffffff; color: #000000; border: 2px solid red; padding:
3px;")
        self.txt_chip.textChanged.connect(self.actualizar_titulo_componente)
        self.txt_chip.textChanged.connect(self.actualizar_habilitacion_campos)

        self.combo_interfaz = ComboBoxMultiCheckbox(db.listar_valores('interfaz'), on_nueva=lambda t:
db.agregar_valor('interfaz', t))
        self.combo_interfaz.setMaximumWidth(280)

        self.combo_pines = ComboBoxMultiCheckbox(db.listar_valores('pines'), on_nueva=lambda t:
db.agregar_valor('pines', t))
        self.combo_pines.setMaximumWidth(280)

        self.combo_encapsulado = ComboBoxMultiCheckbox(db.listar_valores('encapsulado'), on_nueva=lambda t:
db.agregar_valor('encapsulado', t))
        self.combo_encapsulado.setMaximumWidth(280)

        self.txt_ubicacion = LineEditSeguro(); self.txt_ubicacion.setPlaceholderText("Ej: BOX-1");
self.txt_ubicacion.setMaximumWidth(280)
        self.txt_ubicacion.textChanged.connect(self.actualizar_qr_en_vivo)

        self.txt_stock = LineEditSeguro(); self.txt_stock.setPlaceholderText("Ej: 10");
self.txt_stock.setMaximumWidth(280)
        self.txt_stock.textChanged.connect(self.actualizar_qr_en_vivo)

        grid_layout = QGridLayout()
        grid_layout.setSpacing(10)

        campos_pares = [
            (0, "1. Tipo de Componente:", self.combo_tipo, "7. Chip Principal:", self.txt_chip),
            (1, "2. Categoría:", self.combo_cat, "8. Interfaz de Comunicación:", self.combo_interfaz),
            (2, "3. Subcategoría:", self.combo_subcat, "9. Pines Relevantes:", self.combo_pines),
            (3, "4. Ítem Específico:", self.combo_item, "10. Encapsulado:", self.combo_encapsulado),
            (4, "5. Descripción:", layout_desc_container, "11. Ubicación Física:", self.txt_ubicacion),
            (5, "6. Fabricante:", self.combo_fabricante, "12. Stock:", self.txt_stock)
        ]

        for row, lbl1_text, widget1, lbl2_text, widget2 in campos_pares:
            lbl1 = QLabel(lbl1_text); lbl1.setFont(QFont("Arial", 9, QFont.Bold))
            lbl2 = QLabel(lbl2_text); lbl2.setFont(QFont("Arial", 9, QFont.Bold))

            grid_layout.addWidget(lbl1, row, 0)
            if isinstance(widget1, QVBoxLayout):
                grid_layout.addLayout(widget1, row, 1)
            else:
                grid_layout.addWidget(widget1, row, 1)

            grid_layout.addWidget(lbl2, row, 2)
            grid_layout.addWidget(widget2, row, 3)
```

```
        layout_principal.addLayout(grid_layout)
        layout_principal.addSpacing(5)

    def _crear_combo_proveedor(self, lista):
        """Desplegable editable de proveedores (encabezado «N/A»); se puede escribir uno nuevo."""
        combo = QComboBox()
        combo.setEditable(True)
        combo.addItems(db.listar_valores(lista))
        combo.setStyleSheet("background-color: #ffffff; color: #000000;")
        return combo

    def _construir_proveedores(self, layout_principal):
        """Fila de proveedores (exterior y local)."""
        # --- Fila 1: Proveedores ---
        row1_layout = QHBoxLayout()
        row1_layout.setSpacing(10)

        lbl_pve = QLabel("Proveedor Exterior:"); lbl_pve.setFont(QFont("Arial", 9, QFont.Bold))
        self.combo_proveedor_exterior = self._crear_combo_proveedor('proveedor_exterior')

        lbl_pvl = QLabel("Proveedor Local:"); lbl_pvl.setFont(QFont("Arial", 9, QFont.Bold))
        self.combo_proveedor_local = self._crear_combo_proveedor('proveedor_local')

        row1_layout.addWidget(lbl_pve)
        row1_layout.addWidget(self.combo_proveedor_exterior, 2)
        row1_layout.addWidget(lbl_pvl)
        row1_layout.addWidget(self.combo_proveedor_local, 2)

        layout_principal.addLayout(row1_layout)
        layout_principal.addSpacing(5)

    def _construir_precios(self, layout_principal):
        """Fila de precios y monedas."""
# --- Fila 2: Precios, Monedas y Campos de Precio Compactos ---
        row2_layout = QHBoxLayout()
        row2_layout.setSpacing(5)  # Menor separación entre elementos contiguos

        lbl_pe = QLabel("Precio Exterior:"); lbl_pe.setFont(QFont("Arial", 9, QFont.Bold))
        self.combo_moneda_ext = QComboBox()
        self.combo_moneda_ext.addItems(db.listar_valores('moneda'))
        self.combo_moneda_ext.setMaximumWidth(60)
        self.txt_precio_exterior = LineEditSeguro(); self.txt_precio_exterior.setPlaceholderText("0.00");
self.txt_precio_exterior.setMaximumWidth(70)

        lbl_pl = QLabel("Precio Local:"); lbl_pl.setFont(QFont("Arial", 9, QFont.Bold))
        self.combo_moneda_local = QComboBox()
        self.combo_moneda_local.addItems(db.listar_valores('moneda'))
        self.combo_moneda_local.setMaximumWidth(60)
        self.txt_precio_local = LineEditSeguro(); self.txt_precio_local.setPlaceholderText("0.00");
self.txt_precio_local.setMaximumWidth(70)

# Obtener valores guardados solo si estamos editando y el diccionario existe
        if self.datos_componente:
            moneda_local_guardada = self.datos_componente.get('moneda_local', 'USD')
            moneda_ext_guardada = self.datos_componente.get('moneda_exterior', 'USD')
        else:
            moneda_local_guardada = 'USD'
            moneda_ext_guardada = 'USD'

        # Una moneda guardada que ya no esté en la lista se agrega al desplegable para no cambiarla sin querer
        moneda_local_guardada = moneda_local_guardada or 'USD'
        moneda_ext_guardada = moneda_ext_guardada or 'USD'
        for combo, moneda in ((self.combo_moneda_local, moneda_local_guardada), (self.combo_moneda_ext,
moneda_ext_guardada)):
            if combo.findText(moneda) < 0:
                combo.addItem(moneda)
        self.combo_moneda_local.setCurrentText(moneda_local_guardada)
        self.combo_moneda_ext.setCurrentText(moneda_ext_guardada)

        # Añadimos los elementos del bloque exterior
        row2_layout.addWidget(lbl_pe)
        row2_layout.addWidget(self.combo_moneda_ext, 0)
        row2_layout.addWidget(self.txt_precio_exterior, 0)

        # Espacio de separación prudente entre el grupo exterior y el local
        row2_layout.addSpacing(25)

        # Añadimos los elementos del bloque local
        row2_layout.addWidget(lbl_pl)
        row2_layout.addWidget(self.combo_moneda_local, 0)
        row2_layout.addWidget(self.txt_precio_local, 0)
```

```
        # El stretch final empuja todo a la izquierda, evitando espacios vacíos gigantes entre los controles
        row2_layout.addStretch(1)

        layout_principal.addLayout(row2_layout)
        layout_principal.addSpacing(10)        # layout_principal.addLayout(grid_layout)

    def _construir_obsolescencia(self, layout_principal):
        """Botones de opción de obsolescencia."""
        # --- INICIO: Bloque de Radio Buttons para Obsolescencia ---

        obsolescencia_layout = QHBoxLayout()
        obsolescencia_layout.setSpacing(10)

        lbl_obsolescencia = QLabel("Obsolescencia:")
        lbl_obsolescencia.setFont(QFont("Arial", 9, QFont.Bold))

        self.rb_obsolescencia_si = QRadioButton("Sí")
        self.rb_obsolescencia_no = QRadioButton("No")

        self.grupo_obsolescencia = QButtonGroup(self)
        self.grupo_obsolescencia.addButton(self.rb_obsolescencia_si)
        self.grupo_obsolescencia.addButton(self.rb_obsolescencia_no)
        self.rb_obsolescencia_no.setChecked(True)  # Por defecto seleccionado "No"

        obsolescencia_layout.addWidget(lbl_obsolescencia)
        obsolescencia_layout.addWidget(self.rb_obsolescencia_si)
        obsolescencia_layout.addWidget(self.rb_obsolescencia_no)
        obsolescencia_layout.addStretch()# Empuja los radio buttons hacia la izquierda

        layout_principal.addLayout(obsolescencia_layout)
        layout_principal.addSpacing(10)
        # --- FIN: Bloque de Radio Buttons para Obsolescencia ---

    def _construir_boton_parametros(self, layout_principal):
        """Botón de parámetros eléctricos y técnicos avanzados."""
        self.btn_params_avanzados = QPushButton("⚡ Configurar Parámetros Eléctricos y Técnicos Avanzados")
        self.btn_params_avanzados.setAutoDefault(False); self.btn_params_avanzados.setDefault(False)
        self.btn_params_avanzados.setStyleSheet("background-color: #6f42c1; color: white; font-weight: bold;
padding: 10px; font-size: 13px;")
        self.btn_params_avanzados.clicked.connect(self.abrir_dialogo_parametros_avanzados)
        layout_principal.addWidget(self.btn_params_avanzados)

    def _construir_archivos(self, layout_principal):
        """Sección de archivos múltiples (imágenes, datasheets, tutoriales, esquemas)."""
        layout_principal.addSpacing(10)
        lbl_adj = QLabel("Gestión de Archivos Múltiples (Imágenes, Datasheets, Tutoriales, Esquemas)")
        lbl_adj.setFont(QFont("Arial", 11, QFont.Bold))
        layout_principal.addWidget(lbl_adj)

        self.chk_habilitar_quitar = QCheckBox("🗑️ Habilitar botón 'Quitar' en archivos múltiples")
        self.chk_habilitar_quitar.setFont(QFont("Arial", 9, QFont.Bold))
        self.chk_habilitar_quitar.setStyleSheet("color: #dc3545; margin-bottom: 5px;")
        self.chk_habilitar_quitar.setChecked(False)
        self.chk_habilitar_quitar.stateChanged.connect(self.toggle_habilitar_quitar)
        layout_principal.addWidget(self.chk_habilitar_quitar)

        self.botones_quitar = []

        layout_principal.addLayout(self.create_seccion_multianexo("imagen", "📸 Imágenes:"))
        layout_principal.addLayout(self.create_seccion_multianexo("datasheet", "📄 Datasheets:"))
        layout_principal.addLayout(self.create_seccion_multianexo("tutorial", "📘 Tutoriales:"))
        layout_principal.addLayout(self.create_seccion_multianexo("esquema", "📐 Esquemas:"))

    def _construir_boton_guardar(self, layout_principal):
        """Botón de guardar o actualizar."""
        layout_principal.addSpacing(15)
        btn_text = "💾 Actualizar Registro" if self.is_editing else "💾 Guardar Registro Completo"
        btn_guardar = QPushButton(btn_text)
        btn_guardar.setAutoDefault(False); btn_guardar.setDefault(False)
        btn_guardar.setStyleSheet("background-color: #28a745; color: white; font-weight: bold; padding: 12px;
font-size: 14px;")
        btn_guardar.clicked.connect(self.guardar_todo_unificado)
        layout_principal.addWidget(btn_guardar)


    def recibir_parametros_avanzados(self, parametros):
        self.parametros_electricos_temp = parametros
        QMessageBox.information(self, "Parámetros Retenidos", f"Se han sincronizado {len(parametros)} parámetros
eléctricos técnicos desde la ventana auxiliar.")

    def generar_texto_qr_actual(self):
        chip = self.txt_chip.text().strip() or "N/A"
```

```
        item = self.combo_item.currentText().strip()
        if not item or item.startswith("--"):
            item = "N/A"
        desc = self.txt_descripcion.toPlainText().strip()[:45] or "N/A"
        stock = self.txt_stock.text().strip() or "0"
        ubicacion = self.txt_ubicacion.text().strip() or "N/A"

        return f"CHIP: {chip} | ITEM: {item} | DESC: {desc} | STOCK: {stock} | UBIC: {ubicacion}"

    def actualizar_qr_en_vivo(self):
        texto_qr = self.generar_texto_qr_actual()
        qr = qrcode.QRCode(version=1, box_size=3, border=1)
        qr.add_data(texto_qr)
        qr.make(fit=True)
        img_qr = qr.make_image(fill_color="black", back_color="white")

        path_tmp = almacenamiento.guardar_temporal(img_qr, "qr_preview_actual.png")

        if almacenamiento.existe(path_tmp):
            pix = QPixmap(path_tmp).scaled(110, 110, Qt.KeepAspectRatio, Qt.SmoothTransformation)
            self.lbl_qr_preview.setPixmap(pix)

    def formatear_descripcion_en_vivo(self):
        self.txt_descripcion.blockSignals(True)
        cursor = self.txt_descripcion.textCursor()
        pos_actual = cursor.position()

        texto_plano = self.txt_descripcion.toPlainText()
        if not self._cargando_datos:                # lo que se escribe va en mayúsculas; lo guardado se muestra
tal cual
            texto_plano = texto_plano.upper()
        largo = len(texto_plano)

        self.txt_descripcion.setPlainText(texto_plano)

        documento = self.txt_descripcion.document()
        cursor_formato = QTextCursor(documento)

        cursor_formato.setPosition(0)
        cursor_formato.setPosition(min(45, largo), QTextCursor.KeepAnchor)
        fmt_negrita = QTextCharFormat()
        fmt_negrita.setFontWeight(QFont.Bold)
        cursor_formato.mergeCharFormat(fmt_negrita)

        if largo > 45:
            cursor_formato.setPosition(45)
            cursor_formato.setPosition(largo, QTextCursor.KeepAnchor)
            fmt_normal = QTextCharFormat()
            fmt_normal.setFontWeight(QFont.Normal)
            cursor_formato.mergeCharFormat(fmt_normal)

        cursor.setPosition(min(pos_actual, largo))
        self.txt_descripcion.setTextCursor(cursor)
        self.txt_descripcion.blockSignals(False)

        if largo <= 45:
            self.lbl_contador_desc.setText(f"Max caracteres imprimibles: 45 | Actuales: {largo}")
            self.lbl_contador_desc.setStyleSheet("color: #28a745; font-weight: bold;")
        elif largo <= 60:
            self.lbl_contador_desc.setText(f"Max caracteres imprimibles: 45 | Actuales: {largo}")
            self.lbl_contador_desc.setStyleSheet("color: #e67e22; font-weight: bold;")
        else:
            self.lbl_contador_desc.setText(f"Max caracteres imprimibles: 45 | Actuales: {largo}")
            self.lbl_contador_desc.setStyleSheet("color: #dc3545; font-weight: bold;")

        self.actualizar_qr_en_vivo()


    def abrir_dialogo_parametros_avanzados(self):
        """Abre la ventana de parámetros ubicada en la clasificación que tiene el formulario en este momento."""
        ruta = db.ruta_de_clasificacion(
            limpiar_prefijo(self.combo_tipo.currentText()),
            self.combo_cat.currentText().strip(),
            self.combo_subcat.currentText().strip(),
            self.combo_item.currentText().strip(),
        )
        dialogo = DialogoParametrosElectricos(
            ruta=ruta,
            parametros_existentes=getattr(self, 'parametros_electricos_temp', []),
            parent=self
        )
```

```
    if dialogo.exec_() == QDialog.Accepted:
        self.parametros_electricos_temp = dialogo.parametros_resultado
        if hasattr(self, 'recibir_parametros_avanzados'):
            self.recibir_parametros_avanzados(self.parametros_electricos_temp)

def create_seccion_multianexo(self, tipo, etiqueta_texto):
    hbox = QHBoxLayout()
    lbl = QLabel(etiqueta_texto)
    lbl.setFixedWidth(120)
    lbl.setFont(QFont("Arial", 9, QFont.Bold))

    combo = QComboBox()
    combo.setMinimumWidth(250)
    combo.setMaximumWidth(280)
    combo.setStyleSheet("background-color: #ffffff; color: #000000;")

    if tipo == "imagen":
        self.combo_imagenes = combo
        combo.currentIndexChanged.connect(self.on_imagen_seleccionada)
    elif tipo == "datasheet":
        self.combo_datasheets = combo
    elif tipo == "tutorial":
        self.combo_tutoriales = combo
    elif tipo == "esquema":
        self.combo_esquemas = combo

    btn_add = QPushButton("➕ Agregar")
    btn_add.setAutoDefault(False); btn_add.setDefault(False)
    btn_add.setFixedWidth(90)
    btn_add.setStyleSheet("background-color: #28a745; color: white; font-weight: bold; padding: 5px;")
    btn_add.clicked.connect(lambda checked, t=tipo: self.agregar_archivo_multianexo(t))

    btn_ver = QPushButton("👁️ Ver")
    btn_ver.setAutoDefault(False); btn_ver.setDefault(False)
    btn_ver.setFixedWidth(80)
    btn_ver.setStyleSheet("background-color: #007bff; color: white; font-weight: bold; padding: 5px;")
    btn_ver.clicked.connect(lambda checked, t=tipo: self.ver_archivo_multianexo(t))

    btn_del = QPushButton("🗑️ Quitar")
    btn_del.setAutoDefault(False); btn_del.setDefault(False)
    btn_del.setFixedWidth(80)
    btn_del.setStyleSheet("background-color: #dc3545; color: white; font-weight: bold; padding: 5px;")
    btn_del.setEnabled(False)
    btn_del.clicked.connect(lambda checked, t=tipo: self.quitar_archivo_multianexo(t))
    self.botones_quitar.append(btn_del)

    hbox.addWidget(lbl)
    hbox.addWidget(combo)
    hbox.addWidget(btn_add)
    hbox.addWidget(btn_ver)
    hbox.addWidget(btn_del)
    return hbox

def actualizar_titulo_componente(self, texto):
    nombre = texto.strip()
    if nombre:
        self.lbl_nombre_titulo.setText(nombre.upper())
    else:
        self.lbl_nombre_titulo.setText("NOMBRE DEL COMPONENTE")
    self.actualizar_qr_en_vivo()

# ------------------------------------------------------------------
# Habilitación de campos
#   Nuevo componente: solo los 4 combos de clasificación hasta que los cuatro tengan valor.
#   Edición: los 4 combos siempre bloqueados; el resto, solo los campos vacíos
#            (o todos, con la casilla "Habilitar edición completa").
#   Archivos: agregar siempre habilitado; "Quitar" solo con su casilla.
# ------------------------------------------------------------------
_GRIS = "; background-color: #e9ecef; color: #6c757d;"

def _combos_clasificacion(self):
    return [self.combo_tipo, self.combo_cat, self.combo_subcat, self.combo_item]

def _campos_dependientes(self):
    """Todos los campos del formulario salvo los 4 combos de clasificación."""
    return [
        self.txt_descripcion, self.combo_fabricante, self.txt_chip,
        self.combo_interfaz, self.combo_pines, self.combo_encapsulado,
        self.txt_ubicacion, self.txt_stock,
        self.combo_proveedor_exterior, self.txt_precio_exterior, self.combo_moneda_ext,
        self.combo_proveedor_local, self.txt_precio_local, self.combo_moneda_local,
        self.rb_obsolescencia_si, self.rb_obsolescencia_no,
```

```
            self.btn_params_avanzados,
        ]

    def _poner_habilitado(self, widget, habilitado):
        """Habilita o deshabilita un campo y lo atenúa (sus estilos propios no lo harían solos)."""
        original = self._estilos_originales.setdefault(id(widget), widget.styleSheet())
        widget.setEnabled(habilitado)
        if original:
            nuevo = original if habilitado else original + self._GRIS
            if widget.styleSheet() != nuevo:
                widget.setStyleSheet(nuevo)

    @staticmethod
    def _combo_definido(combo):
        """True si el combo tiene una selección real (no vacío ni '-- Todas las ... --')."""
        texto = combo.currentText().strip()
        return bool(texto) and not texto.startswith("--")

    def actualizar_habilitacion_campos(self, *_):
        """Componente nuevo, en tres pasos:
        1) solo los 4 combos de clasificación;
        2) con los 4 definidos, se habilita el Chip Principal (con el marco parpadeando);
        3) con el chip escrito, se habilita el resto de los campos.
        """
        if self.is_editing:
            return
        clasificacion = all(self._combo_definido(c) for c in self._combos_clasificacion())
        chip_escrito = bool(self.txt_chip.text().strip())
        self._poner_habilitado(self.txt_chip, clasificacion)
        for widget in self._campos_dependientes():
            if widget is not self.txt_chip:
                self._poner_habilitado(widget, clasificacion and chip_escrito)
        self._parpadeo_chip(clasificacion and not chip_escrito)

    # Marco grueso intermitente del Chip Principal (de 4 px, igual tamaño total que el estilo normal)
    _MARCO_CHIP_ON = "background-color: #ffffff; color: #000000; border: 4px solid #ff0000; padding: 1px;"
    _MARCO_CHIP_OFF = "background-color: #ffffff; color: #000000; border: 4px solid #f8f9fa; padding: 1px;"

    def _parpadeo_chip(self, activo):
        if activo:
            if not self._timer_chip.isActive():
                self._marco_chip_encendido = False
                self._alternar_marco_chip()
                self._timer_chip.start(500)
        else:
            self._timer_chip.stop()
            self._marco_chip_encendido = False

    def _alternar_marco_chip(self):
        if not self.txt_chip.isEnabled():
            return
        self._marco_chip_encendido = not self._marco_chip_encendido
        self.txt_chip.setStyleSheet(self._MARCO_CHIP_ON if self._marco_chip_encendido else self._MARCO_CHIP_OFF)

    # Mayúsculas al escribir
    @staticmethod
    def _a_mayusculas(line_edit):
        """Convierte a mayúsculas lo que el usuario escribe en un QLineEdit (no lo que se carga por programa)."""
        def convertir(texto, le=line_edit):
            if texto != texto.upper():
                pos = le.cursorPosition()
                le.setText(texto.upper())
                le.setCursorPosition(pos)
        line_edit.textEdited.connect(convertir)

    @staticmethod
    def _a_mayusculas_texto(text_edit):
        """Igual que _a_mayusculas, para un QTextEdit. Solo actúa mientras el usuario escribe (el campo tiene el
foco)."""
        def convertir(te=text_edit):
            if not te.hasFocus():
                return
            texto = te.toPlainText()
            if texto != texto.upper():
                cursor = te.textCursor()
                pos = cursor.position()
                te.blockSignals(True)
                te.setPlainText(texto.upper())
                te.blockSignals(False)
                cursor = te.textCursor()
                cursor.setPosition(min(pos, len(te.toPlainText())))
                te.setTextCursor(cursor)
```

```
        text_edit.textChanged.connect(convertir)

    def _activar_mayusculas(self):
        """Chip, ubicación, proveedores, fabricante y descripción. (Tipo, categoría, subcategoría e ítem se eligen
de listas.)"""
        for le in (self.txt_chip, self.txt_ubicacion, self.combo_proveedor_exterior.lineEdit(),
                   self.combo_proveedor_local.lineEdit(), self.combo_fabricante.lineEdit()):
            self._a_mayusculas(le)
        self._a_mayusculas_texto(self.txt_descripcion)

    def bloquear_campos_completos(self):
        d = self.datos_componente
        self.mapping_widgets = [
            (self.txt_descripcion, d.get('descripcion')),
            (self.combo_fabricante, d.get('fabricante')),
            (self.txt_chip, d.get('chip_principal')),
            (self.combo_interfaz, d.get('interfaz_comunicacion')),
            (self.combo_pines, d.get('pines_relevantes')),
            (self.combo_encapsulado, d.get('encapsulado')),
            (self.txt_ubicacion, d.get('ubicacion')),
            (self.txt_stock, d.get('stock')),
            (self.txt_precio_exterior, d.get('precio_exterior')),
            (self.combo_proveedor_exterior, d.get('proveedor_exterior')),
            (self.txt_precio_local, d.get('precio_local')),
            (self.combo_proveedor_local, d.get('proveedor_local'))
        ]
        for combo in self._combos_clasificacion():          # la clasificación no se modifica al editar
            self._poner_habilitado(combo, False)
        for widget, valor in self.mapping_widgets:
            self._poner_habilitado(widget, self._valor_pendiente(valor))
        self._sincronizar_monedas()

    @staticmethod
    def _valor_pendiente(valor):
        """True si el dato aún no se ingresó: vacío, 0, o 'N/A' (lo que guardan los combos de fabricante,
        interfaz, pines y encapsulado cuando no se eligió nada)."""
        texto = str(valor).strip() if valor is not None else ""
        return texto.upper() in ("", "0", "0.0", "N/A")

    def _sincronizar_monedas(self):
        """La moneda se puede cambiar cuando se puede cambiar su precio."""
        self._poner_habilitado(self.combo_moneda_ext, self.txt_precio_exterior.isEnabled())
        self._poner_habilitado(self.combo_moneda_local, self.txt_precio_local.isEnabled())

    def toggle_habilitar_edicion(self, state):
        habilitar_todo = (state == Qt.Checked)
        if habilitar_todo:
            for widget, _ in self.mapping_widgets:          # no incluye los 4 combos de clasificación
                self._poner_habilitado(widget, True)
            self._sincronizar_monedas()
        else:
            self.bloquear_campos_completos()

    def toggle_habilitar_quitar(self, state):
        habilitar = (state == Qt.Checked)
        for btn in self.botones_quitar:
            btn.setEnabled(habilitar)

    def agregar_imagen_desde_temporal(self, path_temp, filename):
        item_archivo = {
            'tipo': 'imagen',
            'nombre': filename,
            'ruta': path_temp,
            'en_bd': False
        }
        self.archivos_lista.append(item_archivo)
        self.refrescar_combos_multianexo()
        self.combo_imagenes.setCurrentIndex(self.combo_imagenes.count() - 1)

    def agregar_archivo_multianexo(self, tipo):
        if tipo == "imagen":
            filtro = "Imágenes (*.png *.jpg *.jpeg *.bmp *.webp)"
        elif tipo == "datasheet":
            filtro = "Archivos PDF (*.pdf)"
        elif tipo == "tutorial":
            filtro = "Archivos PDF (*.pdf)"
        elif tipo == "esquema":
            filtro = "Imágenes (*.png *.jpg *.jpeg *.bmp *.webp)"

        path, _ = QFileDialog.getOpenFileName(self, f"Seleccionar {tipo.capitalize()}", "", filtro)
        if path:
            dest_path, base_name = almacenamiento.preparar_adjunto(path)
```

```
            item_archivo = {
                'tipo': tipo,
                'nombre': base_name,
                'ruta': dest_path,
                'en_bd': False
            }
            self.archivos_lista.append(item_archivo)
            self.refrescar_combos_multianexo()

            combo = self.combo_imagenes if tipo == "imagen" else (
                    self.combo_datasheets if tipo == "datasheet" else (
                    self.combo_tutoriales if tipo == "tutorial" else self.combo_esquemas))
            combo.setCurrentIndex(combo.count() - 1)
            QMessageBox.information(self, "Éxito", f"{tipo.capitalize()} agregado correctamente.")

    def ver_archivo_multianexo(self, tipo):
        # Asegúrate de incluir self.combo_esquemas en el selector por tipo
        combo = self.combo_imagenes if tipo == "imagen" else (
                self.combo_datasheets if tipo == "datasheet" else (
                self.combo_tutoriales if tipo == "tutorial" else self.combo_esquemas))

        idx = combo.currentIndex()
        if idx < 0 or combo.itemText(idx).startswith("--"):
            QMessageBox.information(self, "Información", f"No hay ningún {tipo} seleccionado para visualizar.")
            return

        ruta = combo.currentData()
        if ruta and almacenamiento.existe(ruta):
            # <--- AQUÍ ESTÁ LA CLAVE: Agregamos "esquema" junto a "imagen" para que use el visor gráfico
            if tipo in ["imagen", "esquema"]:
                editor = DialogoEditorImagen(ruta, self, titulo=combo.itemText(idx), dialogo_componente=self)
                self.ventanas_hijas_abiertas.append(editor)
                editor.show()
            else:
                visor = DialogoVisorPDF(ruta, parent_dialogo_componente=self, parent=self,
titulo=combo.itemText(idx))
                self.ventanas_hijas_abiertas.append(visor)
                visor.show()
        else:
            QMessageBox.warning(self, "Error", "El archivo físico no se encuentra en la ruta especificada.")

    def quitar_archivo_multianexo(self, tipo):
        # 1. Seleccionar el combo correcto incluyendo "esquema"
        combo = self.combo_imagenes if tipo == "imagen" else (
                self.combo_datasheets if tipo == "datasheet" else (
                self.combo_tutoriales if tipo == "tutorial" else self.combo_esquemas))

        idx = combo.currentIndex()
        if idx < 0 or combo.itemText(idx).startswith("--"):
            return

        ruta_a_quitar = combo.currentData()

        # 2. El archivo físico NO se borra: otro componente puede estar usando el mismo documento.
        #    Los documentos que quedan sin uso se retiran al arrancar (almacenamiento.limpiar_huerfanos).
        # 3. Remover de la lista en memoria
        # Se compara también el tipo: el mismo documento (misma ruta) puede estar, por ejemplo, como datasheet y
como tutorial.
        self.archivos_lista = [f for f in self.archivos_lista if not (f['ruta'] == ruta_a_quitar and f['tipo'] ==
tipo)]

        # 4. Refrescar los elementos visuales de todos los combos
        self.refrescar_combos_multianexo()

    def refrescar_combos_multianexo(self):
        for combo, tipo in [
            (self.combo_imagenes, 'imagen'),
            (self.combo_datasheets, 'datasheet'),
            (self.combo_tutoriales, 'tutorial'),
            (self.combo_esquemas, 'esquema')
        ]:
            seleccion_previa = combo.currentData()      # lo que el usuario tenía elegido
            combo.blockSignals(True)
            combo.clear()

            archivos_tipo = [f for f in self.archivos_lista if f['tipo'] == tipo]
            if archivos_tipo:
                for f in archivos_tipo:
                    combo.addItem(f['nombre'], f['ruta'])
                # Se conserva la elección si ese archivo sigue en la lista; si no, el primero
                indice = combo.findData(seleccion_previa) if seleccion_previa is not None else -1
```

```
                combo.setCurrentIndex(indice if indice >= 0 else 0)
            else:
                # Diccionario limpio para los textos por defecto exactos que necesitas
                textos_default = {
                    'imagen': '-- Sin imagen --',
                    'datasheet': '-- Sin datasheets --',
                    'tutorial': '-- Sin tutoriales --',
                    'esquema': '-- Sin esquemas --'
                }
                texto_defecto = textos_default.get(tipo, f"Sin {tipo}")

                combo.addItem(texto_defecto, None)
                combo.setCurrentIndex(0)

            combo.blockSignals(False)

        self.actualizar_miniatura_superior()

    def on_imagen_seleccionada(self, index):
        self.actualizar_miniatura_superior()

    def actualizar_miniatura_superior(self):
        if self.combo_imagenes.count() > 0 and self.combo_imagenes.currentData() is not None:
            ruta = self.combo_imagenes.currentData()
            if ruta and almacenamiento.existe(ruta):
                pix = QPixmap(ruta)
                pix_escalado = pix.scaledToHeight(110, Qt.SmoothTransformation)
                self.lbl_miniatura.setPixmap(pix_escalado)
                self.lbl_miniatura.setFixedSize(pix_escalado.size())
                return
            if ruta:
                self.lbl_miniatura.setPixmap(QPixmap())          # quita la imagen anterior
                self.lbl_miniatura.setText("Archivo no encontrado")
                self.lbl_miniatura.setFixedSize(110, 110)
                return
        self.lbl_miniatura.setText("Sin Imagen")
        self.lbl_miniatura.setFixedSize(110, 110)

    def cargar_combos_enlazados(self):
        self.combo_tipo.blockSignals(True)
        self.combo_tipo.clear()
        for t in db.listar_tipos():
            self.combo_tipo.addItem(t)
        self.combo_tipo.setCurrentIndex(-1)
        self.combo_tipo.blockSignals(False)

    def on_tipo_changed(self):
        tipo = limpiar_prefijo(self.combo_tipo.currentText())
        self.combo_subcat.blockSignals(True); self.combo_subcat.clear(); self.combo_subcat.addItem("-- Todas las
Subcat --"); self.combo_subcat.blockSignals(False)
        self.combo_item.blockSignals(True); self.combo_item.clear(); self.combo_item.addItem("-- Todos los Ítems
--"); self.combo_item.blockSignals(False)

        categorias = db.listar_categorias(sin_comodin(tipo, "-- Todos los Tipos --"))

        self.combo_cat.blockSignals(True)
        self.combo_cat.clear()
        self.combo_cat.addItem("-- Todas las Categorías --")
        self.combo_cat.addItems(categorias)
        self.combo_cat.blockSignals(False)
        self.on_cat_changed()

    def on_cat_changed(self):
        tipo = limpiar_prefijo(self.combo_tipo.currentText())
        cat = self.combo_cat.currentText()
        self.combo_item.blockSignals(True); self.combo_item.clear(); self.combo_item.addItem("-- Todos los Ítems
--"); self.combo_item.blockSignals(False)

        subcats = db.listar_subcategorias(sin_comodin(tipo, "-- Todos los Tipos --"), sin_comodin(cat, "-- Todas
las Categorías --"))

        self.combo_subcat.blockSignals(True)
        self.combo_subcat.clear()
        self.combo_subcat.addItem("-- Todas las Subcat --")
        self.combo_subcat.addItems(subcats)
        self.combo_subcat.blockSignals(False)
        self.on_subcat_changed()

    def on_subcat_changed(self):
        tipo = limpiar_prefijo(self.combo_tipo.currentText())
        cat = self.combo_cat.currentText()
        subcat = self.combo_subcat.currentText()
```

```
        items = db.listar_items(
            tipo=sin_comodin(tipo, "-- Todos los Tipos --"),
            cat=sin_comodin(cat, "-- Todas las Categorías --"),
            subcat=sin_comodin(subcat, "-- Todas las Subcat --"),
        )

        self.combo_item.blockSignals(True)
        self.combo_item.clear()
        self.combo_item.addItem("-- Todos los Ítems --")
        self.combo_item.addItems(items)
        self.combo_item.blockSignals(False)
        self.actualizar_habilitacion_campos()
        self.actualizar_qr_en_vivo()

    def _poner_texto_combo(self, combo, valor):
        """Muestra `valor` en un combo editable (vacío = «N/A»); si no está en la lista, lo deja escrito."""
        valor = str(valor).strip() if valor is not None else ""
        valor = valor or "N/A"
        indice = combo.findText(valor)
        if indice >= 0:
            combo.setCurrentIndex(indice)
        else:
            combo.setEditText(valor)

    def cargar_datos_existentes(self):
        d = self.datos_componente

        self.combo_tipo.setEditText(limpiar_prefijo(str(d.get('tipo_componente', ''))))
        self.combo_cat.setEditText(str(d.get('categoria', '')))
        self.combo_subcat.setEditText(str(d.get('subcategoria', '')))
        self.combo_item.setEditText(str(d.get('item_especifico', '')))

        self.txt_descripcion.setPlainText(str(d.get('descripcion', '')))

        fab_val = str(d.get('fabricante', ''))
        idx_fab = self.combo_fabricante.findText(fab_val)
        if idx_fab >= 0:
            self.combo_fabricante.setCurrentIndex(idx_fab)
        else:
            self.combo_fabricante.setEditText(fab_val)

        self.txt_chip.setText(str(d.get('chip_principal', '')))

        interfaz_val = str(d.get('interfaz_comunicacion', ''))
        self.combo_interfaz.set_seleccionadas(interfaz_val)

        pines_val = str(d.get('pines_relevantes', ''))
        self.combo_pines.set_seleccionadas(pines_val)

        encap_val = str(d.get('encapsulado', ""))
        self.combo_encapsulado.set_seleccionadas(encap_val)

        self.txt_ubicacion.setText(str(d.get('ubicacion', '')))
        self.txt_stock.setText(str(d.get('stock', 0)))

        self.txt_precio_exterior.setText(str(d.get('precio_exterior', 0.0)))
        self._poner_texto_combo(self.combo_proveedor_exterior, d.get('proveedor_exterior'))
        self.txt_precio_local.setText(str(d.get('precio_local', 0.0)))
        self._poner_texto_combo(self.combo_proveedor_local, d.get('proveedor_local'))

        # Cargar la selección de los radio buttons de obsolescencia
        obsolescencia = str(d.get('obsolescencia', 'No')).strip()
        if obsolescencia in ["Sí", "Si", "1", 1]:
            self.rb_obsolescencia_si.setChecked(True)
        else:
            self.rb_obsolescencia_no.setChecked(True)

        componente_id = d.get('id')
        if componente_id:
            rows_archivos = db.listar_archivos(componente_id)
            self.archivos_lista = [{'tipo': t, 'nombre': n, 'ruta': r, 'en_bd': True} for t, n, r in
almacenamiento.resolver_archivos(rows_archivos)]
            self.refrescar_combos_multianexo()

            self.parametros_electricos_temp = db.listar_parametros(componente_id)

    def guardar_todo_unificado(self):
        chip = self.txt_chip.text().strip() if hasattr(self, 'txt_chip') else ""
        if not chip:
            QMessageBox.warning(self, "Aviso Requerido", "Por favor ingrese el 'Chip Principal' antes de guardar
el registro.")
```

```
            return

        datos = self._leer_campos_formulario(chip)

        try:
            # Primero, asegurate de capturar el texto actual de los QComboBox de moneda antes de armar los datos:
            moneda_ext_val = self.combo_moneda_ext.currentText()
            moneda_loc_val = self.combo_moneda_local.currentText()
            datos.update(moneda_exterior=moneda_ext_val, moneda_local=moneda_loc_val)

            comp_id = None
            if self.is_editing and self.datos_componente and 'id' in self.datos_componente:
                comp_id = self.datos_componente['id']   # editando: UPDATE; si no, INSERT

            archivos = self._archivos_para_guardar()

            parametros = self._parametros_para_guardar()

            comp_id = db.guardar_componente(comp_id, datos, archivos, parametros)
            QMessageBox.information(self, "Éxito", "Componente, archivos y especificaciones guardados
correctamente.")
            self.accept()

        except Exception as e:
            QMessageBox.critical(self, "Error de Guardado", f"Ocurrió un error al persistir la información:
{str(e)}")

    def _texto_clasificacion(self, combo):
        """Texto del combo de clasificación. Al crear, si coincide (sin distinguir mayúsculas) con una
        entrada del catálogo se usa la ortografía del catálogo, para no crear «arduino uno» junto a «Arduino
Uno»."""
        texto = combo.currentText().strip()
        if not self.is_editing and texto:
            indice = combo.findText(texto, Qt.MatchFixedString)   # sin distinguir mayúsculas
            if indice > 0:      # 0 es el texto comodín «-- Todos … --»
                return combo.itemText(indice)
        return texto

    def _leer_campos_formulario(self, chip):
        """Lee los widgets del formulario y devuelve el dict de datos del componente (sin las monedas)."""
        tipo_val = limpiar_prefijo(self._texto_clasificacion(self.combo_tipo)) if hasattr(self, 'combo_tipo') else
""
        cat_val = self._texto_clasificacion(self.combo_cat) if hasattr(self, 'combo_cat') else ""
        subcat_val = self._texto_clasificacion(self.combo_subcat) if hasattr(self, 'combo_subcat') else ""
        item_val = self._texto_clasificacion(self.combo_item) if hasattr(self, 'combo_item') else ""
        nombre_val = item_val or subcat_val or cat_val or "Componente sin nombre"

        desc_val = self.txt_descripcion.toPlainText().strip() if hasattr(self, 'txt_descripcion') else ""

        stock_text = self.txt_stock.text().strip() if hasattr(self, 'txt_stock') else "0"
        stock = int(stock_text) if stock_text.isdigit() else 0

        # Precios y proveedores seguros
        try:
            precio_ext = float(self.txt_precio_exterior.text().strip()) if hasattr(self, 'txt_precio_exterior')
and self.txt_precio_exterior.text().strip() else 0.0
        except ValueError:
            precio_ext = 0.0

        try:
            precio_loc = float(self.txt_precio_local.text().strip()) if hasattr(self, 'txt_precio_local') and
self.txt_precio_local.text().strip() else 0.0
        except ValueError:
            precio_loc = 0.0

        prov_ext = self.combo_proveedor_exterior.currentText().strip() if hasattr(self,
'combo_proveedor_exterior') else ""
        prov_loc = self.combo_proveedor_local.currentText().strip() if hasattr(self, 'combo_proveedor_local') else
""

        try:
            precio_unico = float(self.txt_precio.text().strip()) if hasattr(self, 'txt_precio') and
self.txt_precio.text().strip() else 0.0
        except ValueError:
            precio_unico = 0.0

        # Lecturas seguras de widgets que pueden ser ComboBox o LineEdit
        fabricante_val = self.combo_fabricante.currentText().strip() if hasattr(self, 'combo_fabricante') else
(self.txt_fabricante.text().strip() if hasattr(self, 'txt_fabricante') else "")

        interfaz_val = self.combo_interfaz.currentText().strip() if hasattr(self, 'combo_interfaz') else
(self.txt_interfaz.text().strip() if hasattr(self, 'txt_interfaz') else "N/A")
```

```
        pines_val = self.combo_pines.currentText().strip() if hasattr(self, 'combo_pines') else
(self.txt_pines.text().strip() if hasattr(self, 'txt_pines') else "N/A")

        encapsulado_val = self.combo_encapsulado.currentText().strip() if hasattr(self, 'combo_encapsulado') else
(self.txt_encapsulado.text().strip() if hasattr(self, 'txt_encapsulado') else "")

        ubicacion_val = self.txt_ubicacion.text().strip() if hasattr(self, 'txt_ubicacion') else ""

        # Capturar la obsolescencia ANTES de interactuar con la base de datos
        obsolescencia_val = "Sí" if hasattr(self, 'rb_obsolescencia_si') and self.rb_obsolescencia_si.isChecked()
else "No"

        return {
            'nombre': nombre_val, 'tipo_componente': tipo_val, 'categoria': cat_val,
            'subcategoria': subcat_val, 'item_especifico': item_val, 'descripcion': desc_val,
            'fabricante': fabricante_val, 'chip_principal': chip,
            'interfaz_comunicacion': interfaz_val, 'pines_relevantes': pines_val,
            'encapsulado': encapsulado_val, 'stock': stock, 'precio': precio_unico,
            'ubicacion': ubicacion_val, 'precio_exterior': precio_ext,
            'proveedor_exterior': prov_ext, 'precio_local': precio_loc,
            'proveedor_local': prov_loc, 'obsolescencia': obsolescencia_val,
        }

    def _archivos_para_guardar(self):
        """Archivos múltiples (imágenes, datasheets, tutoriales) como tuplas (tipo, nombre, ruta en el
almacén)."""
        archivos = []
        if hasattr(self, 'archivos_lista') and self.archivos_lista:
            # Cada documento entra al almacén por hash (o se reutiliza si ya existe).
            # Dos entradas con el mismo tipo y el mismo contenido se registran una sola vez.
            vistos = set()
            # La imagen elegida en el combo de imágenes pasa a ser la principal (logo del componente):
            # se guarda primera entre las imágenes y así se vuelve a mostrar al reabrir o consultar.
            principal = self.combo_imagenes.currentData() if hasattr(self, 'combo_imagenes') else None
            ordenados = sorted(self.archivos_lista,
                               key=lambda f: not (f['tipo'] == 'imagen' and principal is not None and f['ruta'] ==
principal))
            for f in ordenados:
                ruta = almacenamiento.para_guardar(f['ruta'])
                if (f['tipo'], ruta) not in vistos:
                    vistos.add((f['tipo'], ruta))
                    archivos.append((f['tipo'], f['nombre'], ruta))
        return archivos

    def _parametros_para_guardar(self):
        """Parámetros eléctricos / especificaciones como lista de (clave, valor)."""
        parametros = []
        if hasattr(self, 'parametros_electricos_temp') and self.parametros_electricos_temp:
            parametros = list(self.parametros_electricos_temp)
        return parametros
```

interfaz\dialogo_parametros.py

```
"""Diálogo de parámetros eléctricos y técnicos de un componente.

A la izquierda, el árbol de la taxonomía con los parámetros disponibles en cada nodo;
a la derecha, la tabla de valores del componente. Un componente hereda los parámetros
de todos los niveles de su clasificación (ver db.parametros_heredados).
"""
from PyQt5.QtCore import Qt
from PyQt5.QtGui import QFont
from PyQt5.QtWidgets import (
    QAbstractItemView,
    QDialog,
    QHBoxLayout,
    QHeaderView,
    QLabel,
    QMessageBox,
    QPushButton,
    QSplitter,
    QTableWidget,
    QTableWidgetItem,
    QVBoxLayout,
    QWidget
)

import db
from interfaz.arbol_parametros import ArbolParametros
from interfaz.widgets import aplicar_franjas

PARAMETROS_GENERICOS = ["Tensión de Operación Vcc", "Corriente de Trabajo"]


# ---------------------------------------------------------------------
# VENTANA AUXILIAR: PARÁMETROS ELÉCTRICOS (No modal / Independiente)
# ---------------------------------------------------------------------
class DialogoParametrosElectricos(QDialog):
    def __init__(self, ruta=(), parametros_existentes=None, solo_lectura=False, parent=None):
        """`ruta`: clasificación del componente (hasta 4 nombres). `solo_lectura`: consulta sin edición."""
        super().__init__(parent)
        self.parent_dialog = parent
        self.ruta = tuple(ruta or ())
        self.solo_lectura = solo_lectura
        self.parametros_existentes = parametros_existentes or []
        self.parametros_resultado = []

        self.setWindowTitle("⚡ Parámetros Eléctricos y Técnicos" + (" (consulta)" if solo_lectura else ""))
        self.setWindowFlags(Qt.Window | Qt.WindowCloseButtonHint | Qt.WindowMinimizeButtonHint |
Qt.WindowMaximizeButtonHint)
        self.setWindowState(Qt.WindowMaximized)   # siempre abre maximizada

        layout = QVBoxLayout(self)

        clasificacion = " › ".join(self.ruta) if self.ruta else "sin clasificación"
        lbl_info = QLabel(f"Clasificación del componente: {clasificacion}")
        lbl_info.setFont(QFont("Arial", 9, QFont.Bold))
        lbl_info.setWordWrap(True)
        layout.addWidget(lbl_info)

        divisor = QSplitter(Qt.Horizontal)
        divisor.addWidget(self._construir_panel_arbol())
        divisor.addWidget(self._construir_panel_tabla())
        divisor.setStretchFactor(0, 4)
        divisor.setStretchFactor(1, 5)
        layout.addWidget(divisor)

        if solo_lectura:
            btn_cerrar = QPushButton("Cerrar")
            btn_cerrar.setAutoDefault(False)
            btn_cerrar.setDefault(False)
            btn_cerrar.clicked.connect(self.close)
            layout.addWidget(btn_cerrar)
        else:
            btn_aceptar = QPushButton("✔ Confirmar y Volcar a Ventana Principal")
            btn_aceptar.setAutoDefault(False)
            btn_aceptar.setDefault(False)
            btn_aceptar.setStyleSheet("background-color: #28a745; color: white; font-weight: bold; padding: 10px;
font-size: 13px;")
            btn_aceptar.clicked.connect(self.aceptar_parametros)
            layout.addWidget(btn_aceptar)

        # Cargar los datos iniciales al arrancar la ventana
```

```
        self.cargar_datos_iniciales()

    # --- construcción -------------------------------------------
    def _construir_panel_arbol(self):
        panel = QWidget()
        capa = QVBoxLayout(panel)
        capa.setContentsMargins(0, 0, 0, 0)
        lbl = QLabel("Parámetros disponibles según la clasificación:")
        lbl.setFont(QFont("Arial", 9, QFont.Bold))
        capa.addWidget(lbl)

        self.arbol = ArbolParametros()
        self.arbol.refrescar(abrir=self.ruta)
        capa.addWidget(self.arbol)

        self.lbl_heredados = QLabel("")
        self.lbl_heredados.setWordWrap(True)
        self.lbl_heredados.setStyleSheet("color: #495057; padding: 2px;")
        capa.addWidget(self.lbl_heredados)
        self.arbol.itemSelectionChanged.connect(self.mostrar_heredados)
        self.mostrar_heredados()

        if not self.solo_lectura:
            btn_cargar = QPushButton("➡ Agregar a la tabla los parámetros del nodo elegido")
            btn_cargar.setAutoDefault(False)
            btn_cargar.setDefault(False)
            btn_cargar.setStyleSheet("background-color: #6f42c1; color: white; font-weight: bold; padding: 6px;")
            btn_cargar.clicked.connect(self.agregar_desde_nodo)
            capa.addWidget(btn_cargar)
        return panel

    def _construir_panel_tabla(self):
        panel = QWidget()
        capa = QVBoxLayout(panel)
        capa.setContentsMargins(0, 0, 0, 0)
        lbl = QLabel("Valores del componente:")
        lbl.setFont(QFont("Arial", 9, QFont.Bold))
        capa.addWidget(lbl)

        self.tabla_params = QTableWidget()
        aplicar_franjas(self.tabla_params)
        self.tabla_params.setColumnCount(2)
        self.tabla_params.setHorizontalHeaderLabels(["Parámetro / Propiedad Técnica", "Valor Especificado"])
        self.tabla_params.horizontalHeader().setSectionResizeMode(0, QHeaderView.Stretch)
        self.tabla_params.horizontalHeader().setSectionResizeMode(1, QHeaderView.Stretch)
        if self.solo_lectura:
            self.tabla_params.setEditTriggers(QAbstractItemView.NoEditTriggers)
        capa.addWidget(self.tabla_params)

        if not self.solo_lectura:
            layout_botones_fila = QHBoxLayout()
            btn_agregar_fila = QPushButton("➕ Agregar Fila Personalizada")
            btn_agregar_fila.setAutoDefault(False)
            btn_agregar_fila.setDefault(False)
            btn_agregar_fila.setStyleSheet("background-color: #17a2b8; color: white; font-weight: bold; padding:
6px;")
            btn_agregar_fila.clicked.connect(lambda: self.agregar_fila())

            btn_eliminar_fila = QPushButton("🗑️ Quitar Fila Seleccionada")
            btn_eliminar_fila.setAutoDefault(False)
            btn_eliminar_fila.setDefault(False)
            btn_eliminar_fila.setStyleSheet("background-color: #dc3545; color: white; font-weight: bold; padding:
6px;")
            btn_eliminar_fila.clicked.connect(self.eliminar_fila)

            layout_botones_fila.addWidget(btn_agregar_fila)
            layout_botones_fila.addWidget(btn_eliminar_fila)
            capa.addLayout(layout_botones_fila)
        return panel

    # --- árbol --------------------------------------------------
    def mostrar_heredados(self):
        """Debajo del árbol: todos los parámetros que corresponden al nodo elegido, con el nodo que los define."""
        nodo = self.arbol.nodo_actual()
        if not nodo:
            self.lbl_heredados.setText("Elija un nodo para ver los parámetros que le corresponden.")
            return
        heredados = db.parametros_heredados(nodo)
        if not heredados:
            self.lbl_heredados.setText(f"«{nodo[-1]}» no tiene parámetros definidos.")
            return
```

```
        lineas = [f"• {parametro}" + ("" if origen == nodo else f"  (de {origen[-1]})") for parametro, origen in
heredados]
        self.lbl_heredados.setText(f"Le corresponden {len(heredados)} parámetros:\n" + "\n".join(lineas))

    def agregar_desde_nodo(self):
        """Suma a la tabla los parámetros del nodo elegido que todavía no están (no toca los que ya tienen
valor)."""
        nodo = self.arbol.nodo_actual()
        if not nodo:
            QMessageBox.information(self, "Parámetros", "Elija primero un nodo del árbol.")
            return
        presentes = {self.tabla_params.item(f, 0).text().strip().lower()
                     for f in range(self.tabla_params.rowCount()) if self.tabla_params.item(f, 0)}
        nuevos = [p for p, _ in db.parametros_heredados(nodo) if p.lower() not in presentes]
        if not nuevos:
            QMessageBox.information(self, "Parámetros", "La tabla ya tiene todos los parámetros de ese nodo.")
            return
        for parametro in nuevos:
            self.agregar_fila(parametro, "")

    # --- tabla -------------------------------------------------------
    def obtener_plantilla_por_contexto(self):
        """Nombres de los parámetros que corresponden a la clasificación del componente."""
        plantilla = [parametro for parametro, _ in db.parametros_heredados(self.ruta)] if self.ruta else []
        return plantilla or list(PARAMETROS_GENERICOS)

    def agregar_fila(self, clave="", valor=""):
        row = self.tabla_params.rowCount()
        self.tabla_params.insertRow(row)
        item_clave = QTableWidgetItem(str(clave))
        item_valor = QTableWidgetItem(str(valor))
        self.tabla_params.setItem(row, 0, item_clave)
        self.tabla_params.setItem(row, 1, item_valor)

    def eliminar_fila(self):
        row = self.tabla_params.currentRow()
        if row >= 0:
            self.tabla_params.removeRow(row)

    def cargar_datos_iniciales(self):
        # Si hay parámetros previos pero NO son los genéricos por defecto, los cargamos
        es_generico_viejo = False
        if self.parametros_existentes and len(self.parametros_existentes) == 2:
            nombres_viejos = [str(p[0]) for p in self.parametros_existentes]
            if all(n in nombres_viejos for n in PARAMETROS_GENERICOS):
                es_generico_viejo = True

        if self.parametros_existentes and not es_generico_viejo:
            self.tabla_params.setRowCount(len(self.parametros_existentes))
            for row, param in enumerate(self.parametros_existentes):
                nombre = param[0] if len(param) > 0 else ""
                valor = param[1] if len(param) > 1 else ""
                self.tabla_params.setItem(row, 0, QTableWidgetItem(str(nombre)))
                self.tabla_params.setItem(row, 1, QTableWidgetItem(str(valor)))
            return

        # Sin parámetros previos: se parte de los que corresponden a la clasificación (con herencia)
        if self.solo_lectura:
            return
        plantilla = self.obtener_plantilla_por_contexto()
        self.tabla_params.setRowCount(len(plantilla))
        for row, nombre_param in enumerate(plantilla):
            self.tabla_params.setItem(row, 0, QTableWidgetItem(str(nombre_param)))
            self.tabla_params.setItem(row, 1, QTableWidgetItem(""))

    def aceptar_parametros(self):
        self.parametros_resultado = []
        for row in range(self.tabla_params.rowCount()):
            item_c = self.tabla_params.item(row, 0)
            item_v = self.tabla_params.item(row, 1)
            if item_c and item_c.text().strip():
                val = item_v.text().strip() if item_v else ""
                self.parametros_resultado.append((item_c.text().strip(), val))

        if self.parent_dialog and hasattr(self.parent_dialog, 'recibir_parametros_avanzados'):
            self.parent_dialog.recibir_parametros_avanzados(self.parametros_resultado)

        self.close()
```

interfaz\dialogo_catalogos.py

```
"""Ventana para administrar el catálogo de clasificación y las listas de los desplegables.

No contiene consultas: todo el acceso a datos está en db.py.
"""
from PyQt5.QtCore import Qt
from PyQt5.QtWidgets import (
    QDialog,
    QHBoxLayout,
    QInputDialog,
    QLabel,
    QListWidget,
    QMessageBox,
    QPushButton,
    QHeaderView,
    QTabWidget,
    QTableWidget,
    QTableWidgetItem,
    QTreeWidget,
    QTreeWidgetItem,
    QVBoxLayout,
    QWidget,
)

import db
from interfaz.arbol_parametros import ArbolParametros
from interfaz.widgets import aplicar_franjas

NOMBRES_NIVEL = ("Tipo", "Categoría", "Subcategoría", "Ítem")

# (clave en db.LISTAS, título de la pestaña, ayuda)
LISTAS = (
    ("fabricante", "🏭 Fabricantes", "Opciones del desplegable «Fabricante». También se puede escribir uno nuevo
directamente en el formulario."),
    ("interfaz", "🔌 Interfaces", "Opciones de «Interfaz de comunicación». No se permiten comas."),
    ("pines", "📍 Pines relevantes", "Opciones de «Pines relevantes». No se permiten comas."),
    ("encapsulado", "📦 Encapsulados", "Opciones de «Encapsulado». No se permiten comas."),
    ("moneda", "💲 Monedas", "Monedas disponibles junto a los precios."),
    ("proveedor_local", "🏪 Proveedores locales", "Proveedores del país: opciones de «Proveedor Local». También se
puede escribir uno nuevo en el formulario."),
    ("proveedor_exterior", "🌎 Proveedores del exterior", "Proveedores del exterior: opciones de «Proveedor
Exterior». También se puede escribir uno nuevo en el formulario."),
)

ESTILO_BOTON = "font-weight: bold; padding: 5px 10px; color: white; background-color: %s;"


def _boton(texto, color, accion):
    b = QPushButton(texto)
    b.setAutoDefault(False)
    b.setStyleSheet(ESTILO_BOTON % color)
    b.clicked.connect(accion)
    return b


def _ayuda(texto):
    lbl = QLabel(texto)
    lbl.setWordWrap(True)
    lbl.setStyleSheet("color: #495057; padding: 2px;")
    return lbl


class _PestanaClasificacion(QWidget):
    """Árbol Tipo > Categoría > Subcategoría > Ítem."""

    def __init__(self, parent=None):
        super().__init__(parent)
        layout = QVBoxLayout(self)
        layout.addWidget(_ayuda(
            "Elija un nodo y pulse «Agregar» para crearle un hijo (sin selección se crea un Tipo nuevo). "
            "Renombrar actualiza también los componentes ya cargados; Eliminar solo quita la opción del catálogo "
            "(los componentes conservan lo que tienen escrito). Un Ítem es el producto concreto (p. ej. Arduino
Uno); "
            "en las familias generales el producto real va en el Chip Principal."))

        self.arbol = QTreeWidget()
        self.arbol.setHeaderLabels(["Clasificación", "Nivel"])
        self.arbol.setColumnWidth(0, 520)
        aplicar_franjas(self.arbol)
        layout.addWidget(self.arbol)
```

```
        fila = QHBoxLayout()
        fila.addWidget(_boton("➕ Agregar", "#28a745", self.agregar))
        fila.addWidget(_boton("✏ Renombrar", "#ffc107; color: #212529", self.renombrar))
        fila.addWidget(_boton("🗑️ Eliminar", "#dc3545", self.eliminar))
        fila.addStretch()
        layout.addLayout(fila)

        self.refrescar()

    # --- árbol ---------------------------------------------------
    def refrescar(self, abrir=None):
        """Reconstruye el árbol conservando los nodos abiertos; `abrir` es una ruta que se despliega y
selecciona."""
        abiertos = set(self._rutas_abiertas())
        if abrir:
            abiertos |= {tuple(abrir[:k]) for k in range(1, len(abrir))}
        self.arbol.clear()
        nodos = {}
        for fila in db.listar_catalogo():
            ruta = ()
            for nivel, nombre in enumerate(fila):
                if not nombre:
                    break
                ruta += (nombre,)
                if ruta not in nodos:
                    padre = nodos.get(ruta[:-1]) or self.arbol.invisibleRootItem()
                    nodo = QTreeWidgetItem(padre, [nombre, NOMBRES_NIVEL[nivel]])
                    nodo.setData(0, Qt.UserRole, ruta)
                    nodos[ruta] = nodo
        for ruta, nodo in nodos.items():
            nodo.setExpanded(ruta in abiertos)
        if abrir and tuple(abrir) in nodos:
            self.arbol.setCurrentItem(nodos[tuple(abrir)])

    def _rutas_abiertas(self):
        pila = [self.arbol.topLevelItem(i) for i in range(self.arbol.topLevelItemCount())]
        while pila:
            nodo = pila.pop()
            if nodo.isExpanded():
                yield nodo.data(0, Qt.UserRole)
            pila.extend(nodo.child(i) for i in range(nodo.childCount()))

    def _ruta_elegida(self):
        nodo = self.arbol.currentItem()
        return tuple(nodo.data(0, Qt.UserRole)) if nodo else None

    # --- acciones ------------------------------------------------
    def agregar(self):
        padre = self._ruta_elegida() or ()
        if len(padre) >= 4:
            QMessageBox.information(self, "Agregar", "Un Ítem es el último nivel: no puede tener hijos.")
            return
        nivel = NOMBRES_NIVEL[len(padre)]
        nombre, ok = QInputDialog.getText(self, f"Nuevo {nivel}", f"Nombre del nuevo {nivel}:")
        if not ok or not nombre.strip():
            return
        try:
            nueva = db.agregar_catalogo(padre + (nombre.strip(),))
        except ValueError as e:
            QMessageBox.warning(self, "No se pudo agregar", str(e))
            return
        self.refrescar(abrir=nueva)

    def renombrar(self):
        ruta = self._ruta_elegida()
        if not ruta:
            QMessageBox.information(self, "Renombrar", "Elija primero un nodo del árbol.")
            return
        nivel = NOMBRES_NIVEL[len(ruta) - 1]
        usados = db.contar_componentes_catalogo(ruta)
        nuevo, ok = QInputDialog.getText(self, f"Renombrar {nivel}",
                                         f"Nuevo nombre para «{ruta[-1]}»" + (f"\n(se actualizarán {usados}
componente(s))" if usados else "") + ":",
                                         text=ruta[-1])
        if not ok or not nuevo.strip() or nuevo.strip() == ruta[-1]:
            return
        try:
            nueva = db.renombrar_catalogo(ruta, nuevo.strip())
        except ValueError as e:
            QMessageBox.warning(self, "No se pudo renombrar", str(e))
            return
```

```
        self.refrescar(abrir=nueva)

    def eliminar(self):
        ruta = self._ruta_elegida()
        if not ruta:
            QMessageBox.information(self, "Eliminar", "Elija primero un nodo del árbol.")
            return
        usados = db.contar_componentes_catalogo(ruta)
        texto = f"¿Quitar «{ruta[-1]}» del catálogo"
        texto += " junto con todo lo que contiene?" if len(ruta) < 4 else "?"
        if usados:
            texto += f"\n\n{usados} componente(s) ya cargados conservan su clasificación; solo dejará de ofrecerse
en los desplegables."
        if QMessageBox.question(self, "Eliminar", texto, QMessageBox.Yes | QMessageBox.No, QMessageBox.No) !=
QMessageBox.Yes:
            return
        db.eliminar_catalogo(ruta)
        self.refrescar(abrir=ruta[:-1] or None)


class _PestanaLista(QWidget):
    """Una lista de valores (fabricantes, interfaces, pines, encapsulados, monedas)."""

    def __init__(self, clave, ayuda, parent=None):
        super().__init__(parent)
        self.clave = clave
        layout = QVBoxLayout(self)
        layout.addWidget(_ayuda(ayuda + (" «N/A» es fija." if clave in db.LISTAS_CON_SIN_DATO else "")))

        self.lista = QListWidget()
        aplicar_franjas(self.lista)
        layout.addWidget(self.lista)

        fila = QHBoxLayout()
        fila.addWidget(_boton("➕ Agregar", "#28a745", self.agregar))
        fila.addWidget(_boton("✏ Renombrar", "#ffc107; color: #212529", self.renombrar))
        fila.addWidget(_boton("🗑️ Eliminar", "#dc3545", self.eliminar))
        fila.addStretch()
        layout.addLayout(fila)

        self.refrescar()

    def refrescar(self, elegir=None):
        self.lista.clear()
        self.lista.addItems(db.listar_valores(self.clave))
        if elegir:
            for fila in range(self.lista.count()):
                if self.lista.item(fila).text() == elegir:
                    self.lista.setCurrentRow(fila)
                    break

    def _valor_elegido(self):
        item = self.lista.currentItem()
        return item.text() if item else None

    def agregar(self):
        valor, ok = QInputDialog.getText(self, "Agregar", "Nuevo valor:")
        if not ok or not valor.strip():
            return
        try:
            guardado = db.agregar_valor(self.clave, valor)
        except ValueError as e:
            QMessageBox.warning(self, "No se pudo agregar", str(e))
            return
        self.refrescar(elegir=guardado)

    def renombrar(self):
        viejo = self._valor_elegido()
        if viejo is None:
            QMessageBox.information(self, "Renombrar", "Elija primero un valor de la lista.")
            return
        usados = db.contar_uso_valor(self.clave, viejo)
        nuevo, ok = QInputDialog.getText(self, "Renombrar",
                                         f"Nuevo nombre para «{viejo}»" + (f"\n(se actualizarán {usados}
componente(s))" if usados else "") + ":",
                                         text=viejo)
        if not ok or not nuevo.strip() or nuevo.strip() == viejo:
            return
        try:
            db.renombrar_valor(self.clave, viejo, nuevo)
        except ValueError as e:
            QMessageBox.warning(self, "No se pudo renombrar", str(e))
```

```
            return
        self.refrescar(elegir=nuevo.strip())

    def eliminar(self):
        valor = self._valor_elegido()
        if valor is None:
            QMessageBox.information(self, "Eliminar", "Elija primero un valor de la lista.")
            return
        if valor.upper() == db.SIN_DATO:
            QMessageBox.information(self, "Eliminar", f"«{db.SIN_DATO}» es fija y no se puede eliminar.")
            return
        usados = db.contar_uso_valor(self.clave, valor)
        texto = f"¿Quitar «{valor}» de la lista?"
        if usados:
            texto += f"\n\n{usados} componente(s) ya cargados conservan este valor; solo dejará de ofrecerse en el
desplegable."
        if QMessageBox.question(self, "Eliminar", texto, QMessageBox.Yes | QMessageBox.No, QMessageBox.No) !=
QMessageBox.Yes:
            return
        db.eliminar_valor(self.clave, valor)
        self.refrescar()


class _PestanaParametros(QWidget):
    """Parámetros disponibles en cada nodo de la taxonomía (se heredan hacia abajo)."""

    def __init__(self, parent=None):
        super().__init__(parent)
        layout = QVBoxLayout(self)
        layout.addWidget(_ayuda(
            "Elija un nodo y pulse «Agregar parámetro». Un componente hereda los parámetros de todos los niveles
de su "
            "clasificación: lo común conviene definirlo en el Tipo o la Categoría, y lo particular en la
Subcategoría o el Ítem. "
            "Renombrar o quitar un parámetro no modifica los valores ya cargados en los componentes."))

        self.arbol = ArbolParametros()
        layout.addWidget(self.arbol)

        self.lbl_heredados = _ayuda("")
        layout.addWidget(self.lbl_heredados)
        self.arbol.itemSelectionChanged.connect(self.mostrar_heredados)

        fila = QHBoxLayout()
        fila.addWidget(_boton("➕ Agregar parámetro", "#28a745", self.agregar))
        fila.addWidget(_boton("✏ Renombrar", "#ffc107; color: #212529", self.renombrar))
        fila.addWidget(_boton("🗑️ Quitar", "#dc3545", self.quitar))
        fila.addStretch()
        layout.addLayout(fila)
        self.mostrar_heredados()

    def refrescar(self):
        """Se vuelve a leer todo (el catálogo pudo cambiar en la otra pestaña)."""
        self.arbol.refrescar(abrir=self.arbol.nodo_actual())
        self.mostrar_heredados()

    def mostrar_heredados(self):
        nodo = self.arbol.nodo_actual()
        if not nodo:
            self.lbl_heredados.setText("Elija un nodo para ver todos los parámetros que le corresponden.")
            return
        heredados = db.parametros_heredados(nodo)
        detalle = "; ".join(p + ("" if o == nodo else f" (de {o[-1]})") for p, o in heredados) or "ninguno"
        self.lbl_heredados.setText(f"Parámetros de «{nodo[-1]}» con los heredados ({len(heredados)}): {detalle}")

    def agregar(self):
        nodo = self.arbol.nodo_actual()
        if not nodo:
            QMessageBox.information(self, "Agregar parámetro", "Elija primero un nodo del árbol.")
            return
        nombre, ok = QInputDialog.getText(self, "Nuevo parámetro", f"Nombre del parámetro para «{nodo[-1]}»:")
        if not ok or not nombre.strip():
            return
        try:
            db.agregar_parametro_plantilla(nodo, nombre)
        except ValueError as e:
            QMessageBox.warning(self, "No se pudo agregar", str(e))
            return
        self.arbol.refrescar(abrir=nodo)
        self.mostrar_heredados()

    def renombrar(self):
```

```
        elegido = self.arbol.parametro_actual()
        if not elegido:
            QMessageBox.information(self, "Renombrar", "Elija primero un parámetro (los que tienen 📏).")
            return
        nodo, viejo = elegido
        nuevo, ok = QInputDialog.getText(self, "Renombrar parámetro", f"Nuevo nombre para «{viejo}»:", text=viejo)
        if not ok or not nuevo.strip() or nuevo.strip() == viejo:
            return
        try:
            db.renombrar_parametro_plantilla(nodo, viejo, nuevo)
        except ValueError as e:
            QMessageBox.warning(self, "No se pudo renombrar", str(e))
            return
        self.arbol.refrescar(abrir=nodo)
        self.mostrar_heredados()

    def quitar(self):
        elegido = self.arbol.parametro_actual()
        if not elegido:
            QMessageBox.information(self, "Quitar", "Elija primero un parámetro (los que tienen 📏).")
            return
        nodo, nombre = elegido
        if QMessageBox.question(self, "Quitar", f"¿Quitar «{nombre}» de «{nodo[-1]}»?\n\nLos valores ya cargados
en componentes se conservan.",
                                QMessageBox.Yes | QMessageBox.No, QMessageBox.No) != QMessageBox.Yes:
            return
        db.eliminar_parametro_plantilla(nodo, nombre)
        self.arbol.refrescar(abrir=nodo)
        self.mostrar_heredados()


class _PestanaCotizaciones(QWidget):
    """Cotizaciones de las monedas respecto del USD (base del cálculo de capital)."""

    def __init__(self, parent=None):
        super().__init__(parent)
        layout = QVBoxLayout(self)
        layout.addWidget(_ayuda(
            "Escriba cuántas unidades de cada moneda equivalen a 1 USD (por ejemplo, UYU 40,50 significa 1 USD =
40,50 UYU). "
            "La fecha se registra sola cuando cambia el valor. Con estas cotizaciones se calcula el capital en
existencia; "
            "una moneda sin cotización no suma. El USD es la moneda base y siempre vale 1. "
            "Las monedas se agregan en la pestaña «Monedas»."))

        self.tabla = QTableWidget()
        aplicar_franjas(self.tabla)
        self.tabla.setColumnCount(3)
        self.tabla.setHorizontalHeaderLabels(["Moneda", "Unidades por 1 USD", "Actualizada"])
        self.tabla.horizontalHeader().setSectionResizeMode(0, QHeaderView.Stretch)
        self.tabla.horizontalHeader().setSectionResizeMode(1, QHeaderView.Stretch)
        self.tabla.horizontalHeader().setSectionResizeMode(2, QHeaderView.Stretch)
        layout.addWidget(self.tabla)

        fila = QHBoxLayout()
        fila.addWidget(_boton("💾 Guardar cotizaciones", "#28a745", self.guardar))
        fila.addStretch()
        layout.addLayout(fila)
        self.refrescar()

    @staticmethod
    def _texto(por_usd):
        return "" if por_usd is None else f"{por_usd:.10g}"

    def refrescar(self, conservar=True):
        """Vuelve a leer las cotizaciones (las monedas pudieron cambiar en otra pestaña); con `conservar`, sin
perder lo que se está escribiendo."""
        pendientes = {}
        for i in range(self.tabla.rowCount() if conservar else 0):
            moneda, celda = self.tabla.item(i, 0).text(), self.tabla.item(i, 1)
            if celda.text() != self._texto(getattr(self, "guardadas", {}).get(moneda)):
                pendientes[moneda] = celda.text()
        filas = db.listar_cotizaciones()
        self.guardadas = {m: p for m, p, _ in filas}
        self.tabla.setRowCount(len(filas))
        for i, (moneda, por_usd, fecha) in enumerate(filas):
            valor = pendientes.get(moneda, self._texto(por_usd))
            celdas = (QTableWidgetItem(moneda), QTableWidgetItem(valor), QTableWidgetItem(fecha or ""))
            for columna, celda in enumerate(celdas):
                if columna != 1 or moneda.upper() == db.MONEDA_BASE:
                    celda.setFlags(celda.flags() & ~Qt.ItemIsEditable)     # solo se edita el valor (y no el del
USD)
```

```
                self.tabla.setItem(i, columna, celda)

    def guardar(self):
        """Valida todo antes de guardar nada; solo escribe (y fecha) las cotizaciones que cambiaron."""
        cambios = []
        for i in range(self.tabla.rowCount()):
            moneda = self.tabla.item(i, 0).text()
            if moneda.upper() == db.MONEDA_BASE:
                continue
            texto = self.tabla.item(i, 1).text().strip().replace(",", ".")
            try:
                valor = float(texto) if texto else None
            except ValueError:
                QMessageBox.warning(self, "Cotización no válida", f"La cotización de {moneda} no es un número:
«{self.tabla.item(i, 1).text()}».")
                return
            if valor is not None and not valor > 0:
                QMessageBox.warning(self, "Cotización no válida", f"La cotización de {moneda} debe ser mayor que
cero.")
                return
            if valor != self.guardadas.get(moneda):
                cambios.append((moneda, valor))
        for moneda, valor in cambios:
            db.guardar_cotizacion(moneda, valor)
        self.refrescar(conservar=False)
        QMessageBox.information(self, "Cotizaciones", f"Se actualizaron {len(cambios)} cotización(es)." if cambios
else "No hubo cambios.")


class DialogoAdministrarCatalogos(QDialog):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.setWindowTitle("📚 Listas y Catálogo")
        self.setWindowState(Qt.WindowMaximized)   # siempre abre maximizada

        layout = QVBoxLayout(self)
        pestanas = QTabWidget()
        pestanas.addTab(_PestanaClasificacion(), "🗂️ Clasificación")
        self.pestana_parametros = _PestanaParametros()
        pestanas.addTab(self.pestana_parametros, "📏 Parámetros")
        for clave, titulo, ayuda in LISTAS:
            pestanas.addTab(_PestanaLista(clave, ayuda), titulo)
        self.pestana_cotizaciones = _PestanaCotizaciones()
        pestanas.addTab(self.pestana_cotizaciones, "💱 Cotizaciones")
        pestanas.currentChanged.connect(lambda _: (self.pestana_parametros.refrescar(),
self.pestana_cotizaciones.refrescar()))
        layout.addWidget(pestanas)

        cerrar = QPushButton("Cerrar")
        cerrar.setAutoDefault(False)
        cerrar.clicked.connect(self.accept)
        layout.addWidget(cerrar, alignment=Qt.AlignRight)
```

interfaz\dialogo_capital.py

```
"""Ventana con el capital en existencia: subtotales por clasificación, detalle y exportación a Excel."""
import os
from datetime import date

from PyQt5.QtCore import Qt
from PyQt5.QtGui import QColor, QFont
from PyQt5.QtWidgets import (
    QAbstractItemView,
    QDialog,
    QFileDialog,
    QHBoxLayout,
    QLabel,
    QMessageBox,
    QPushButton,
    QTableWidget,
    QTableWidgetItem,
    QTabWidget,
    QTreeWidget,
    QTreeWidgetItem,
    QVBoxLayout,
    QWidget
)

import capital
from interfaz.widgets import COLOR_ALERTA, aplicar_franjas

COLUMNAS_DETALLE = ("Tipo", "Categoría", "Subcategoría", "Ítem", "Chip Principal", "Stock", "Precio usado",
"Moneda",
                    "Origen", "Unit. (USD)", "Total (USD)", "Estado")


def _celda(texto, derecha=False, alerta=False):
    celda = QTableWidgetItem(str(texto))
    celda.setFlags(celda.flags() & ~Qt.ItemIsEditable)
    if derecha:
        celda.setTextAlignment(Qt.AlignRight | Qt.AlignVCenter)
    if alerta:
        celda.setBackground(QColor(COLOR_ALERTA))
    return celda


class DialogoCapital(QDialog):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.setWindowTitle("💰 Capital en existencia")
        self.setWindowState(Qt.WindowMaximized)   # siempre abre maximizada
        self.datos = None
        self._carpeta = os.path.expanduser("~")

        layout = QVBoxLayout(self)
        self.lbl_total = QLabel("")
        self.lbl_total.setFont(QFont("Arial", 13, QFont.Bold))
        layout.addWidget(self.lbl_total)
        self.lbl_avisos = QLabel("")
        self.lbl_avisos.setWordWrap(True)
        layout.addWidget(self.lbl_avisos)

        self.pestanas = QTabWidget()
        self.arbol = QTreeWidget()
        self.arbol.setHeaderLabels(["Clasificación", "Componentes", "Unidades", "Capital (USD)", "% del total"])
        self.arbol.setColumnWidth(0, 460)
        aplicar_franjas(self.arbol)
        self.tabla = QTableWidget()
        self.tabla.setColumnCount(len(COLUMNAS_DETALLE))
        self.tabla.setHorizontalHeaderLabels(list(COLUMNAS_DETALLE))
        self.tabla.setEditTriggers(QAbstractItemView.NoEditTriggers)
        self.tabla.setSortingEnabled(False)
        aplicar_franjas(self.tabla)
        self.pestanas.addTab(self.arbol, "Subtotales por clasificación")
        self.pestanas.addTab(self.tabla, "Detalle de componentes")
        layout.addWidget(self.pestanas)

        botones = QHBoxLayout()
        btn_actualizar = QPushButton("🔄 Actualizar")
        btn_actualizar.setAutoDefault(False)
        btn_actualizar.clicked.connect(self.cargar)
        btn_excel = QPushButton("📊 Exportar a Excel")
        btn_excel.setAutoDefault(False)
        btn_excel.setStyleSheet("background-color: #198754; color: white; font-weight: bold; padding: 6px;")
```

```
        btn_excel.clicked.connect(self.exportar)
        btn_cerrar = QPushButton("Cerrar")
        btn_cerrar.setAutoDefault(False)
        btn_cerrar.clicked.connect(self.close)
        botones.addWidget(btn_actualizar)
        botones.addStretch()
        botones.addWidget(btn_excel)
        botones.addWidget(btn_cerrar)
        layout.addLayout(botones)
        self.cargar()

    # --- datos ---------------------------------------------------
    def cargar(self):
        self.datos = datos = capital.calcular()
        total = datos["total_usd"]
        self.lbl_total.setText(f"Capital total: USD {capital.formato_monto(total)}   ·   {datos['unidades']}
unidades en {len(datos['filas'])} componentes")
        avisos = []
        if datos["sin_precio"]:
            avisos.append(f"{datos['sin_precio']} componente(s) sin precio")
        if datos["sin_cotizacion"]:
            avisos.append(f"{datos['sin_cotizacion']} sin cotización de {',
'.join(datos['monedas_sin_cotizacion'])} "
                          "(cárguela en «📚 Listas y Catálogo» › «💱 Cotizaciones»)")
        self.lbl_avisos.setText("⚠ No suman al capital: " + "; ".join(avisos) if avisos else "")
        self.lbl_avisos.setStyleSheet("color: #856404;")
        self._llenar_arbol(datos, total)
        self._llenar_detalle(datos["filas"])

    def _llenar_arbol(self, datos, total):
        self.arbol.clear()
        nodos, camino_de = {}, {}
        for camino, monto, cantidad, unidades in capital.subtotales(datos["filas"]):
            padre = nodos.get(camino[:-1]) or self.arbol.invisibleRootItem()
            porcentaje = f"{monto / total * 100:.1f} %" if total else ""
            nodo = QTreeWidgetItem(padre, [camino[-1], str(cantidad), str(unidades), capital.formato_monto(monto),
porcentaje])
            for col in (1, 2, 3, 4):
                nodo.setTextAlignment(col, Qt.AlignRight | Qt.AlignVCenter)
            nodos[camino] = nodo
            camino_de[id(nodo)] = camino
        for nodo in nodos.values():
            nodo.setExpanded(len(camino_de[id(nodo)]) == 1)

    def _llenar_detalle(self, filas):
        self.tabla.setRowCount(len(filas))
        orden = sorted(filas, key=lambda f: tuple((f[k] or "").lower() for k in ("tipo_componente", "categoria",
"subcategoria", "item_especifico", "chip_principal")))
        for i, f in enumerate(orden):
            v = f["valor"]
            alerta = v["estado"] != capital.ESTADO_OK
            valores = [f["tipo_componente"] or capital.SIN_DEFINIR, f["categoria"] or capital.SIN_DEFINIR,
                       f["subcategoria"] or capital.SIN_DEFINIR, f["item_especifico"] or capital.SIN_DEFINIR,
                       f["chip_principal"] or "", f["stock"] or 0,
                       capital.formato_monto(v["precio"]) if v["precio"] else "", v["moneda"], v["origen"],
                       capital.formato_monto(v["unitario_usd"]) if v["unitario_usd"] is not None else "",
                       capital.formato_monto(v["total_usd"]) if v["unitario_usd"] is not None else "",
v["estado"]]
            for j, valor in enumerate(valores):
                self.tabla.setItem(i, j, _celda(valor, derecha=j in (5, 6, 9, 10), alerta=alerta))

    # --- exportación ---------------------------------------------
    def exportar(self):
        datos = self.datos or capital.calcular()
        if not datos["filas"]:
            QMessageBox.information(self, "Exportar capital", "No hay componentes cargados.")
            return
        sugerido = os.path.join(self._carpeta, f"Capital_Inventario_{date.today():%Y-%m-%d}.xlsx")
        destino, _ = QFileDialog.getSaveFileName(self, "Guardar planilla de capital", sugerido, "Planilla de Excel
(*.xlsx)")
        if not destino:
            return
        if not destino.lower().endswith(".xlsx"):
            destino += ".xlsx"
        try:
            capital.exportar_xlsx(destino, datos)
        except capital.ExportError as e:
            QMessageBox.critical(self, "No se pudo generar la planilla", str(e))
            return
        except OSError as e:
            QMessageBox.critical(self, "No se pudo guardar el archivo",
                                 f"{e}\n\nSi la planilla está abierta en Excel, ciérrela y vuelva a intentarlo.")
```

```
    return
self._carpeta = os.path.dirname(destino)
QMessageBox.information(self, "Capital exportado", f"Planilla guardada en:\n{destino}")
```

interfaz\dialogo_etiquetas.py

```
"""Diálogo de impresión de etiquetas y generación de su PDF."""
import qrcode

from PyQt5.QtCore import Qt
from PyQt5.QtGui import QFont
from PyQt5.QtWidgets import (
    QComboBox,
    QDialog,
    QDoubleSpinBox,
    QGridLayout,
    QHBoxLayout,
    QLabel,
    QMessageBox,
    QPushButton,
    QSpinBox,
    QVBoxLayout
)
from reportlab.lib.pagesizes import A4, letter
from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
from reportlab.lib.units import mm
from reportlab.pdfgen import canvas
from reportlab.platypus import Paragraph

import almacenamiento
import db
from interfaz.visor_pdf import DialogoVisorPDF


# ---------------------------------------------------------------------
# VENTANA: IMPRESIÓN DE ETIQUETAS
# ---------------------------------------------------------------------
class DialogoImpresionEtiquetas(QDialog):
    def __init__(self, componentes, parent=None):
        super().__init__(parent)
        self.setWindowTitle("Impresión de Etiquetas")
        self.componentes = componentes
        self.init_ui()

    def init_ui(self):
        layout = QVBoxLayout()
        layout.setContentsMargins(15, 15, 15, 15)
        layout.setSpacing(10)

        title = QLabel("Cola de Impresión y Dimensiones de Etiqueta")
        title.setFont(QFont("Arial", 10, QFont.Bold))
        layout.addWidget(title)

        # --- Selector de Tamaño de Papel ---
        box_papel = QHBoxLayout()
        box_papel.addWidget(QLabel("Formato de Papel:"))
        self.combo_papel = QComboBox()
        self.combo_papel.addItems(["A4 (210 x 297 mm)", "Letter / Carta (215.9 x 279.4 mm)"])
        box_papel.addWidget(self.combo_papel)
        layout.addLayout(box_papel)

        box_medidas = QGridLayout()
        box_medidas.addWidget(QLabel("Ancho (mm):"), 0, 0)
        self.spin_w = QDoubleSpinBox()
        self.spin_w.setRange(10.0, 300.0)
        self.spin_w.setValue(40.0)
        box_medidas.addWidget(self.spin_w, 0, 1)

        box_medidas.addWidget(QLabel("Alto (mm):"), 0, 2)
        self.spin_h = QDoubleSpinBox()
        self.spin_h.setRange(10.0, 300.0)
        self.spin_h.setValue(25.0)
        box_medidas.addWidget(self.spin_h, 0, 3)

        box_medidas.addWidget(QLabel("Separación H (mm):"), 1, 0)
        self.spin_gap_x = QDoubleSpinBox()
        self.spin_gap_x.setRange(0.0, 50.0)
        self.spin_gap_x.setValue(2.0)
        box_medidas.addWidget(self.spin_gap_x, 1, 1)

        box_medidas.addWidget(QLabel("Separación V (mm):"), 1, 2)
        self.spin_gap_y = QDoubleSpinBox()
        self.spin_gap_y.setRange(0.0, 50.0)
        self.spin_gap_y.setValue(2.0)
        box_medidas.addWidget(self.spin_gap_y, 1, 3)
```

```
        layout.addLayout(box_medidas)

        self.spin_boxes = {}
        for item in self.componentes:
            hbox = QHBoxLayout()
            lbl = QLabel(f"{item['nombre']} ({item['ubicacion']})")

            # SpinBox para la cantidad de etiquetas (copias)
            spin_copias = QSpinBox()
            spin_copias.setRange(1, 100)
            spin_copias.setValue(1)
            spin_copias.setFixedWidth(55)

            # SpinBox para las unidades por bolsa
            spin_unidades = QSpinBox()
            spin_unidades.setRange(1, 10000)
            spin_unidades.setValue(1)
            spin_unidades.setFixedWidth(65)

            hbox.addWidget(lbl)
            hbox.addStretch()
            hbox.addWidget(QLabel("Copias:"))
            hbox.addWidget(spin_copias)
            hbox.addWidget(QLabel("Unid/Bolsa:"))
            hbox.addWidget(spin_unidades)

            layout.addLayout(hbox)
            # Guardamos ambos selectores en la tupla para leerlos en generar_pdf
            self.spin_boxes[item['id']] = (item, spin_copias, spin_unidades)

        hbox_start = QHBoxLayout()
        hbox_start.addWidget(QLabel("Comenzar en la celda N°:"))
        self.spin_start = QSpinBox()
        self.spin_start.setRange(1, 100)
        self.spin_start.setValue(1)
        self.spin_start.setFixedWidth(70)
        hbox_start.addStretch()
        hbox_start.addWidget(self.spin_start)
        layout.addLayout(hbox_start)

        layout.addSpacing(5)
        btn_pdf = QPushButton("📄 Generar PDF de Etiquetas")
        btn_pdf.setAutoDefault(False); btn_pdf.setDefault(False)
        btn_pdf.setStyleSheet("background-color: #28a745; color: white; font-weight: bold; padding: 8px;")
        btn_pdf.clicked.connect(self.generar_pdf)
        layout.addWidget(btn_pdf)

        self.setLayout(layout)
        self.adjustSize()      # esta ventana se ajusta a su contenido (no se maximiza)

    @staticmethod
    def _dibujar_guias_de_corte(c, cols, rows, label_w, label_h, gap_x, gap_y, margin_x, margin_y, page_h):
        """Líneas a rayas por el punto medio de cada separación (y a media separación de los bordes exteriores)
        para indicar por dónde troquelar. Con separación 0 coinciden con el borde de las etiquetas."""
        ancho = cols * label_w + (cols - 1) * gap_x
        alto = rows * label_h + (rows - 1) * gap_y
        izq, der = margin_x - gap_x / 2, margin_x + ancho + gap_x / 2
        sup, inf = page_h - margin_y + gap_y / 2, page_h - margin_y - alto - gap_y / 2
        c.saveState()
        c.setStrokeColorRGB(0.45, 0.45, 0.45)
        c.setLineWidth(0.4)
        c.setDash(3, 2)
        for k in range(cols + 1):
            x = margin_x + k * (label_w + gap_x) - gap_x / 2
            c.line(x, inf, x, sup)
        for k in range(rows + 1):
            y = page_h - margin_y - k * (label_h + gap_y) + gap_y / 2
            c.line(izq, y, der, y)
        c.restoreState()

    def generar_pdf(self):
        output_pdf = almacenamiento.ruta_pdf_etiquetas()

        # 1. Obtener tamaño de hoja según la selección del ComboBox
        if self.combo_papel.currentIndex() == 0:
            page_size = A4
        else:
            page_size = letter

        page_w, page_h = page_size
```

```
        # 2. Leer dimensiones y espaciados
        label_w = self.spin_w.value() * mm
        label_h = self.spin_h.value() * mm
        gap_x = self.spin_gap_x.value() * mm
        gap_y = self.spin_gap_y.value() * mm

        # 3. Calcular automáticamente columnas, filas y celdas por página
        min_margin = 5 * mm
        cols = max(1, int((page_w - (2 * min_margin) + gap_x) // (label_w + gap_x)))
        rows = max(1, int((page_h - (2 * min_margin) + gap_y) // (label_h + gap_y)))
        total_cells_per_page = cols * rows

        # 4. Cálculo dinámico de márgenes para centrar la grilla perfectamente
        ancho_total_grilla = (cols * label_w) + ((cols - 1) * gap_x)
        alto_total_grilla = (rows * label_h) + ((rows - 1) * gap_y)

        margin_x = max(2 * mm, (page_w - ancho_total_grilla) / 2)
        margin_y = max(2 * mm, (page_h - alto_total_grilla) / 2)

        c = canvas.Canvas(output_pdf, pagesize=page_size)
        current_cell = self.spin_start.value() - 1

        styles = getSampleStyleSheet()
        style_chip = ParagraphStyle('ChipStyle', parent=styles['Normal'], fontName='Helvetica-Bold', fontSize=8,
leading=9)
        style_item = ParagraphStyle('ItemStyle', parent=styles['Normal'], fontName='Helvetica', fontSize=6.5,
leading=8)
        style_desc = ParagraphStyle('DescStyle', parent=styles['Normal'], fontName='Helvetica-Oblique',
fontSize=6, leading=7.5)
        style_ubic = ParagraphStyle('UbicStyle', parent=styles['Normal'], fontName='Helvetica-Bold', fontSize=6.5,
leading=8)
        style_unidades = ParagraphStyle('UnidStyle', parent=styles['Normal'], fontName='Helvetica-Bold',
fontSize=6.5, leading=8) # <--- Agrega esta línea aquí


        for item_id, (item, spin_copias, spin_unidades) in self.spin_boxes.items():
            copias = spin_copias.value()
            unidades_por_bolsa = spin_unidades.value()

            if copias <= 0:
                continue

            row_db = db.obtener_datos_etiqueta(item_id)

            chip_prin = row_db[0] if row_db and row_db[0] else item.get('nombre', 'N/A')
            item_esp = row_db[1] if row_db and row_db[1] else 'N/A'
            desc_completa = row_db[2] if row_db and row_db[2] else 'N/A'
            ubicacion = row_db[3] if row_db and row_db[3] else item.get('ubicacion', 'N/A')
            stock = row_db[5] if row_db and row_db[5] is not None else 0

            texto_qr = f"CHIP: {chip_prin} | ITEM: {item_esp} | DESC: {desc_completa[:35]} | STOCK: {stock} |
UBIC: {ubicacion}"

            qr = qrcode.QRCode(version=1, box_size=1, border=1)
            qr.add_data(texto_qr)
            qr.make(fit=True)
            img_qr = qr.make_image(fill_color="black", back_color="white")

            path_qr_tmp = almacenamiento.guardar_temporal(img_qr, f"qr_temp_{item_id}.png")

            for _ in range(copias):
                if current_cell >= total_cells_per_page:
                    self._dibujar_guias_de_corte(c, cols, rows, label_w, label_h, gap_x, gap_y, margin_x,
margin_y, page_h)
                    c.showPage()
                    current_cell = 0

                col = current_cell % cols
                # Calculamos la fila de arriba hacia abajo (0 es la primera fila superior)
                row = current_cell // cols

                x = margin_x + col * (label_w + gap_x)
                # Invertimos el eje Y para que baje desde el margen superior de la hoja
                y = page_h - margin_y - (row + 1) * label_h - row * gap_y

                c.rect(x, y, label_w, label_h)

                m = 1.0 * mm
                qr_size = min(label_h - (2 * m), label_h * 0.45)
                qr_y = y + (label_h - qr_size) / 2
                c.drawImage(path_qr_tmp, x + label_w - qr_size - m, qr_y, width=qr_size, height=qr_size)
```

```
            text_width = label_w - qr_size - (3 * m)
            text_x = x + m

            desc_cortada = desc_completa[:70] + "..." if len(desc_completa) > 70 else desc_completa

            p_chip = Paragraph(f"<b>{chip_prin}</b>", style_chip)
            p_item = Paragraph(f"Ítem: {item_esp}", style_item)
            p_desc = Paragraph(f"{desc_cortada}", style_desc)  # <--- Usar la versión recortada
            p_ubic = Paragraph(f"Ubic: {ubicacion}", style_ubic)
            p_unid = Paragraph(f"Bolsa: <b>{unidades_por_bolsa} unids.</b>", style_unidades)

            w_c, h_c = p_chip.wrap(text_width, label_h)
            w_i, h_i = p_item.wrap(text_width, label_h)
            w_d, h_d = p_desc.wrap(text_width, label_h)
            w_u, h_u = p_ubic.wrap(text_width, label_h)
            w_un, h_un = p_unid.wrap(text_width, label_h)

            curr_y = y + label_h - m

            curr_y -= h_c
            p_chip.drawOn(c, text_x, curr_y)

            curr_y -= (h_i + 0.5 * mm)
            p_item.drawOn(c, text_x, curr_y)

            curr_y -= (h_d + 0.5 * mm)
            p_desc.drawOn(c, text_x, curr_y)

            curr_y -= (h_u + 0.5 * mm)
            p_ubic.drawOn(c, text_x, curr_y)

            curr_y -= (h_un + 0.3 * mm)
            p_unid.drawOn(c, text_x, curr_y)

            current_cell += 1

    self._dibujar_guias_de_corte(c, cols, rows, label_w, label_h, gap_x, gap_y, margin_x, margin_y, page_h)
    c.save()
    QMessageBox.information(self, "PDF Creado", f"Se generó el archivo:\n{output_pdf}")
    visor = DialogoVisorPDF(output_pdf, None, self, es_modo_etiquetas=True)
    visor.show()
```

interfaz\editor_imagen.py

```
"""Editor y visor de imágenes (recorte y guardado)."""
import os

from PyQt5.QtCore import QPoint, QRect, Qt
from PyQt5.QtGui import QFont, QPainter, QPen, QPixmap
from PyQt5.QtWidgets import (
    QDialog,
    QFileDialog,
    QHBoxLayout,
    QLabel,
    QMessageBox,
    QPushButton,
    QScrollArea,
    QVBoxLayout,
    QWidget
)

import almacenamiento


# ---------------------------------------------------------------------
# EDITOR / VISOR AVANZADO DE IMAGEN
# ---------------------------------------------------------------------
_FILTRO_GUARDAR = "Imágenes PNG (*.png);;Imágenes JPG (*.jpg *.jpeg);;Todos los archivos (*.*)"
_ultima_carpeta = None      # última carpeta elegida al guardar: el siguiente diálogo empieza ahí


def pedir_destino_y_guardar(parent, imagen, sufijo, titulo_dialogo, chip="componente"):
    """Pide al usuario la carpeta y el nombre, guarda `imagen` ahí y avisa. Devuelve la ruta, o None si
    canceló o falló. Es el único camino de guardado de recortes e imágenes (visor de PDF y visor de imagen)."""
    global _ultima_carpeta
    nombre_sugerido = almacenamiento.nombre_archivo(chip or "componente", sufijo)
    if _ultima_carpeta and os.path.isdir(_ultima_carpeta):
        ruta_inicial = os.path.join(_ultima_carpeta, nombre_sugerido)
    else:
        ruta_inicial = os.path.join(os.path.expanduser("~"), nombre_sugerido)     # primera vez: carpeta personal

    destino, _ = QFileDialog.getSaveFileName(parent, titulo_dialogo, ruta_inicial, _FILTRO_GUARDAR)
    if not destino:
        return None                       # canceló: no se guarda nada
    if not os.path.splitext(destino)[1]:
        destino += ".png"                 # sin extensión Qt no sabe en qué formato escribir

    try:
        almacenamiento.guardar_imagen(imagen, destino)
    except Exception as e:
        QMessageBox.critical(parent, "Error al guardar", f"No se pudo guardar la imagen en:\n{destino}\n\n{e}")
        return None
    _ultima_carpeta = os.path.dirname(os.path.abspath(destino))
    QMessageBox.information(parent, "Imagen guardada", f"La imagen se guardó en:\n{destino}")
    return destino


class DialogoEditorImagen(QDialog):
    """Visor de imagen con recorte. Guardar (región o entera) hace UNA sola cosa: pide la carpeta y el
    nombre, y escribe el archivo ahí. No registra la imagen en ningún componente: eso lo decide el usuario
después.
    `dialogo_componente` (opcional) solo sirve para proponer un nombre de archivo con el chip."""

    def __init__(self, image_path, parent=None, titulo=None, dialogo_componente=None):
        super().__init__(parent)
        self.image_path = image_path
        self.dialogo_componente = dialogo_componente
        self.setWindowTitle(f"✂ Editor y Visor de Imagen - {titulo or os.path.basename(image_path)}")
        self.setWindowFlags(Qt.Window | Qt.WindowCloseButtonHint | Qt.WindowMinimizeButtonHint |
Qt.WindowMaximizeButtonHint)
        self.setWindowState(Qt.WindowMaximized)   # siempre abre maximizada

        self.pixmap_original = QPixmap(image_path)
        self.zoom_factor = 1.0
        self.begin = QPoint()
        self.end = QPoint()
        self.selected_rect = QRect()
        self.resultado_imagen = None
        self.drawing = False

        layout = QVBoxLayout(self)

        # --- BARRA SUPERIOR UNIFICADA (ZOOM, GUARDAR Y CANCELAR) ---
```

```
        top_bar_layout = QHBoxLayout()

        # 1. Controles de Zoom
        self.lbl_zoom_info = QLabel("🔍 Zoom: 100%")
        self.lbl_zoom_info.setFont(QFont("Arial", 10, QFont.Bold))

        btn_zoom_mas = QPushButton("🔍➕ Ampliar")
        btn_zoom_mas.setAutoDefault(False); btn_zoom_mas.setDefault(False)
        btn_zoom_mas.setStyleSheet("background-color: #007bff; color: white; font-weight: bold;")

        btn_zoom_menos = QPushButton("🔍➖ Reducir")
        btn_zoom_menos.setAutoDefault(False); btn_zoom_menos.setDefault(False)
        btn_zoom_menos.setStyleSheet("background-color: #007bff; color: white; font-weight: bold;")

        btn_zoom_reset = QPushButton("↺ 100%")
        btn_zoom_reset.setAutoDefault(False); btn_zoom_reset.setDefault(False)
        btn_zoom_reset.setStyleSheet("background-color: #6c757d; color: white; font-weight: bold;")

        btn_zoom_mas.clicked.connect(self.aumentar_zoom)
        btn_zoom_menos.clicked.connect(self.reducir_zoom)
        btn_zoom_reset.clicked.connect(self.reset_zoom)

        # 2. Botones de Guardar
        btn_guardar_recorte = QPushButton("✅ Guardar Región")
        btn_guardar_recorte.setAutoDefault(False); btn_guardar_recorte.setDefault(False)
        btn_guardar_recorte.setStyleSheet("background-color: #28a745; color: white; font-weight: bold;")
        btn_guardar_recorte.clicked.connect(self.guardar_recorte)

        btn_guardar_entera = QPushButton("💾 Guardar Entera")
        btn_guardar_entera.setAutoDefault(False); btn_guardar_entera.setDefault(False)
        btn_guardar_entera.setStyleSheet("background-color: #17a2b8; color: white; font-weight: bold;")
        btn_guardar_entera.clicked.connect(self.guardar_completa)

        # 3. Botón Cancelar (quedará al extremo derecho)
        btn_cancelar = QPushButton("❌ Cancelar")
        btn_cancelar.setAutoDefault(False); btn_cancelar.setDefault(False)
        btn_cancelar.setStyleSheet("background-color: #dc3545; color: white; font-weight: bold;")
        btn_cancelar.clicked.connect(self.close)

        # --- APLICAR ALTURA UNIFORME A TODOS LOS BOTONES ---
        for btn in [btn_zoom_mas, btn_zoom_menos, btn_zoom_reset, btn_guardar_recorte, btn_guardar_entera,
btn_cancelar]:
            btn.setFixedHeight(35)

        # Agregar elementos al layout superior en orden horizontal
        top_bar_layout.addWidget(self.lbl_zoom_info)
        top_bar_layout.addSpacing(10)
        top_bar_layout.addWidget(btn_zoom_mas)
        top_bar_layout.addWidget(btn_zoom_menos)
        top_bar_layout.addWidget(btn_zoom_reset)
        top_bar_layout.addSpacing(15)
        top_bar_layout.addWidget(btn_guardar_recorte)
        top_bar_layout.addWidget(btn_guardar_entera)
        top_bar_layout.addStretch()  # Espaciador flexible que empuja el botón cancelar a la derecha
        top_bar_layout.addWidget(btn_cancelar)

        layout.addLayout(top_bar_layout)
        # ------------------------------------------------------------

        lbl_info = QLabel("💡 Arrastre el mouse para seleccionar un área de recorte, o guarde directamente sin
seleccionar para conservar toda la imagen.")
        lbl_info.setStyleSheet("background-color: #e9ecef; color: #333; padding: 8px; font-weight: bold; border-
radius: 4px;")
        layout.addWidget(lbl_info)

        self.scroll_area = QScrollArea()
        self.scroll_area.setWidgetResizable(True)
        self.canvas_widget = QWidget()
        self.actualizar_pixmap_escalado()

        self.canvas_widget.paintEvent = self.canvas_paint_event
        self.canvas_widget.mousePressEvent = self.canvas_mouse_press
        self.canvas_widget.mouseMoveEvent = self.canvas_mouse_move
        self.canvas_widget.mouseReleaseEvent = self.canvas_mouse_release

        self.scroll_area.setWidget(self.canvas_widget)
        layout.addWidget(self.scroll_area)

    def actualizar_pixmap_escalado(self):
        nuevo_ancho = int(self.pixmap_original.width() * self.zoom_factor)
        nuevo_alto = int(self.pixmap_original.height() * self.zoom_factor)
```

```
        self.pixmap_escalado = self.pixmap_original.scaled(nuevo_ancho, nuevo_alto, Qt.KeepAspectRatio,
Qt.SmoothTransformation)
        self.canvas_widget.setMinimumSize(self.pixmap_escalado.size())
        self.canvas_widget.update()

    def aumentar_zoom(self):
        if self.zoom_factor < 4.0:
            self.zoom_factor += 0.25
            self.lbl_zoom_info.setText(f"🔍 Zoom: {int(self.zoom_factor * 100)}%")
            self.actualizar_pixmap_escalado()

    def reducir_zoom(self):
        if self.zoom_factor > 0.5:
            self.zoom_factor -= 0.25
            self.lbl_zoom_info.setText(f"🔍 Zoom: {int(self.zoom_factor * 100)}%")
            self.actualizar_pixmap_escalado()

    def reset_zoom(self):
        self.zoom_factor = 1.0
        self.lbl_zoom_info.setText("🔍 Zoom: 100%")
        self.actualizar_pixmap_escalado()

    def canvas_paint_event(self, event):
        painter = QPainter(self.canvas_widget)
        painter.drawPixmap(0, 0, self.pixmap_escalado)
        if not self.selected_rect.isNull():
            pen = QPen(Qt.red, 2, Qt.DashLine)
            painter.setPen(pen)
            painter.setBrush(Qt.NoBrush)
            painter.drawRect(self.selected_rect)

    def canvas_mouse_press(self, event):
        if event.button() == Qt.LeftButton:
            self.begin = event.pos()
            self.end = self.begin
            self.selected_rect = QRect()
            self.drawing = True
            self.canvas_widget.update()

    def canvas_mouse_move(self, event):
        if self.drawing and (event.buttons() & Qt.LeftButton):
            self.end = event.pos()
            self.selected_rect = QRect(self.begin, self.end).normalized()
            self.canvas_widget.update()

    def canvas_mouse_release(self, event):
        if event.button() == Qt.LeftButton:
            self.drawing = False
            if not self.selected_rect.isNull() and self.selected_rect.width() > 5 and self.selected_rect.height()
> 5:
                self.selected_rect = self.selected_rect.intersected(self.pixmap_escalado.rect())
                self.canvas_widget.update()

    def guardar_recorte(self):
        if not self.selected_rect.isNull() and self.selected_rect.width() > 5:
            x = int(self.selected_rect.x() / self.zoom_factor)
            y = int(self.selected_rect.y() / self.zoom_factor)
            w = int(self.selected_rect.width() / self.zoom_factor)
            h = int(self.selected_rect.height() / self.zoom_factor)
            rect_real = QRect(x, y, w, h).intersected(self.pixmap_original.rect())
            self._guardar(self.pixmap_original.copy(rect_real), "recorte.png", "Guardar Recorte Como...")
        else:
            self.guardar_completa()      # sin región marcada se guarda la imagen entera

    def guardar_completa(self):
        self._guardar(self.pixmap_original, "completa.png", "Guardar Imagen Completa Como...")

    def _guardar(self, imagen, sufijo, titulo_dialogo):
        """Guarda con `pedir_destino_y_guardar`; si se guardó, cierra el visor (si canceló, queda abierto)."""
        comp = self.dialogo_componente
        chip = comp.txt_chip.text().strip() if comp is not None else ""
        if pedir_destino_y_guardar(self, imagen, sufijo, titulo_dialogo, chip):
            self.resultado_imagen = imagen
            self.close()
```

interfaz\visor_pdf.py

```
"""Visor de PDF con selección y recorte interactivo."""
import os
import webbrowser

import pymupdf as fitz

from PyQt5.QtCore import QPoint, QRect, Qt
from PyQt5.QtGui import QBrush, QColor, QFont, QImage, QPainter, QPen, QPixmap
from PyQt5.QtWidgets import (
    QAction,
    QApplication,
    QDialog,
    QHBoxLayout,
    QLabel,
    QMenu,
    QMessageBox,
    QPushButton,
    QScrollArea,
    QVBoxLayout,
    QWidget
)

import almacenamiento
import ocr
from interfaz.editor_imagen import pedir_destino_y_guardar


# ---------------------------------------------------------------------
# UTILIDAD: ENLACES CLICABLES EN PDF
# ---------------------------------------------------------------------
def hacer_links_clicables_en_pdf(doc_pdf):
    url_pattern = r"(https?://[^\s]+|www\.[^\s]+)"
    for page in doc_pdf:
        links_encontrados = page.search_for(url_pattern)
        for rect in links_encontrados:
            texto_url = page.get_text("text", clip=rect).strip()
            if texto_url:
                uri_final = texto_url if texto_url.startswith("http") else "http://" + texto_url
                try:
                    page.insert_link({
                        "kind": fitz.LINK_URI,
                        "from": rect,
                        "uri": uri_final
                    })
                except Exception:
                    pass


# ---------------------------------------------------------------------
# VISOR DE PDF INTERACTIVO PROFESIONAL CON DOBLE MODO (TEXTO / RECORTE)
# ---------------------------------------------------------------------
class WidgetVisorPDFInteractivo(QWidget):
    def __init__(self, page, zoom_level, parent_dialog=None):
        super().__init__()
        self.page = page
        self.zoom_level = zoom_level
        self.parent_dialog = parent_dialog
        self.modo = "texto"  # "texto" o "recorte"
        self.actualizar_renderizado()
        self.dragging = False
        self.start_pos = QPoint()
        self.end_pos = QPoint()
        self.selection_rect = QRect()
        self.texto_seleccionado_actual = ""
        self.texto_por_ocr = False      # True si el texto seleccionado salió del OCR y no del texto del PDF
        self.aviso_ocr = ""             # motivo por el que el OCR no pudo usarse (para mostrarlo en el menú)
        self.setMouseTracking(True)

    def set_modo(self, modo):
        self.modo = modo
        self.selection_rect = QRect()
        self.update()
        if modo == "recorte":
            self.setCursor(Qt.CrossCursor)
        else:
            self.setCursor(Qt.IBeamCursor)

    def actualizar_renderizado(self):
        matrix = fitz.Matrix(self.zoom_level, self.zoom_level)
```

```
        pix = self.page.get_pixmap(matrix=matrix, colorspace=fitz.csRGB, alpha=False)
        qimage = QImage(pix.samples, pix.width, pix.height, pix.stride, QImage.Format_RGB888)
        self.pixmap = QPixmap.fromImage(qimage)
        self.setFixedSize(self.pixmap.size())
        self.update()

    def paintEvent(self, event):
        painter = QPainter(self)
        painter.fillRect(self.rect(), Qt.white)
        painter.drawPixmap(0, 0, self.pixmap)

        if not self.selection_rect.isNull():
            if self.modo == "texto":
                painter.setPen(QPen(QColor(0, 120, 215), 1, Qt.SolidLine))
                painter.setBrush(QBrush(QColor(0, 120, 215, 60)))
            else:  # Modo recorte: Marco rojo solicitado
                painter.setPen(QPen(Qt.red, 2, Qt.DashLine))
                painter.setBrush(QBrush(QColor(255, 0, 0, 30)))
            painter.drawRect(self.selection_rect)

    def mousePressEvent(self, event):
        if event.button() == Qt.LeftButton:
            if self.modo == "texto":
                point_pdf = fitz.Point(event.pos().x() / self.zoom_level, event.pos().y() / self.zoom_level)
                link_encontrado = None
                for link in self.page.get_links():
                    if "from" in link and link["from"].contains(point_pdf):
                        link_encontrado = link
                        break

                if link_encontrado:
                    if link_encontrado.get("kind") == fitz.LINK_URI:
                        uri = link_encontrado.get("uri")
                        if uri:
                            webbrowser.open(uri)
                            return

            self.dragging = True
            self.start_pos = event.pos()
            self.end_pos = self.start_pos
            self.selection_rect = QRect()
            self.texto_seleccionado_actual = ""
            self.texto_por_ocr = False
            self.aviso_ocr = ""
            self.update()

    def mouseMoveEvent(self, event):
        if self.dragging:
            self.end_pos = event.pos()
            self.selection_rect = QRect(self.start_pos, self.end_pos).normalized()
            self.update()

    def mouseReleaseEvent(self, event):
        if event.button() == Qt.LeftButton and self.dragging:
            self.dragging = False
            self.selection_rect = QRect(self.start_pos, self.end_pos).normalized()
            self.update()

            if self.modo == "texto":
                self.extraer_texto_seleccionado()
            # En modo recorte la región solo queda marcada: se guarda con el botón "Guardar Recorte".

    def extraer_texto_seleccionado(self):
        if self.selection_rect.width() < 3 or self.selection_rect.height() < 3:
            return
        x1 = self.selection_rect.left() / self.zoom_level
        y1 = self.selection_rect.top() / self.zoom_level
        x2 = self.selection_rect.right() / self.zoom_level
        y2 = self.selection_rect.bottom() / self.zoom_level
        pdf_rect = fitz.Rect(x1, y1, x2, y2)
        self.aviso_ocr = ""
        texto = self._texto_en_region(pdf_rect)
        self.texto_por_ocr = False
        if not texto:                       # la página no tiene texto en esa zona (es una imagen): se prueba con
OCR
            texto = self._texto_por_ocr(pdf_rect)
            self.texto_por_ocr = bool(texto)
        self.texto_seleccionado_actual = texto
        if self.texto_seleccionado_actual:
            QApplication.clipboard().setText(self.texto_seleccionado_actual)

    def _texto_por_ocr(self, rect_pdf):
```

```
        """Reconoce con Tesseract el texto de una región que es imagen. Cadena vacía si no hay o no hay OCR
        (en ese caso `aviso_ocr` explica por qué)."""
        region = fitz.Rect(rect_pdf).intersect(self.page.rect)
        if region.is_empty:
            return ""
        QApplication.setOverrideCursor(Qt.WaitCursor)
        try:
            imagen = self._renderizar(region, escala=3.0)        # más resolución: el OCR acierta mucho más
            ruta = almacenamiento.guardar_temporal(imagen, "ocr_temp.png", "PNG")
            return ocr.reconocer_texto(ruta)
        except ocr.OcrError as e:
            self.aviso_ocr = str(e)
            return ""
        finally:
            QApplication.restoreOverrideCursor()

    def _texto_en_region(self, rect_pdf):
        """Texto de la región. Primero el texto contenido en el marco; si no hay, las palabras que el marco toca
        (un marco que corta una letra no debe dejar la selección vacía)."""
        texto = self.page.get_text("text", clip=rect_pdf).strip()
        if texto:
            return texto
        palabras = [w for w in self.page.get_text("words") if fitz.Rect(w[:4]).intersects(rect_pdf)]
        palabras.sort(key=lambda w: (w[5], w[6], w[7]))          # bloque, línea, palabra
        lineas, actual = [], None
        for w in palabras:
            if (w[5], w[6]) != actual:
                lineas.append([])
                actual = (w[5], w[6])
            lineas[-1].append(w[4])
        return "\n".join(" ".join(linea) for linea in lineas).strip()

    def _renderizar(self, clip=None, escala=2.0):
        """Imagen de la página (o de una región de ella), por defecto a doble resolución. Copia propia: no depende
del buffer de PyMuPDF."""
        pix = self.page.get_pixmap(matrix=fitz.Matrix(escala, escala), clip=clip, colorspace=fitz.csRGB,
alpha=False)
        return QImage(pix.samples, pix.width, pix.height, pix.stride, QImage.Format_RGB888).copy()

    def imagen_recorte(self):
        """Imagen de la región marcada en modo recorte, o None si no hay una región válida."""
        if self.modo != "recorte" or self.selection_rect.width() <= 5 or self.selection_rect.height() <= 5:
            return None
        clip = fitz.Rect(self.selection_rect.left() / self.zoom_level, self.selection_rect.top() /
self.zoom_level,
                         self.selection_rect.right() / self.zoom_level, self.selection_rect.bottom() /
self.zoom_level)
        clip.intersect(self.page.rect)          # la región no puede salirse de la página
        if clip.is_empty:
            return None
        return self._renderizar(clip)

    def imagen_pagina(self):
        """Imagen de toda la página que muestra el visor."""
        return self._renderizar()

    def contextMenuEvent(self, event):
        if self.modo != "texto":
            return
        menu = QMenu(self)
        if not self.texto_seleccionado_actual:
            # Siempre se muestra un menú: así se ve que el clic derecho funciona y por qué no hay nada que copiar.
            if self.selection_rect.isNull():
                aviso = "Arrastre el mouse sobre el texto para seleccionarlo"
            elif self.aviso_ocr:
                aviso = self.aviso_ocr
            else:
                aviso = "No se encontró texto en la zona marcada (ni con OCR)"
            accion_aviso = QAction(aviso, self)
            accion_aviso.setEnabled(False)
            menu.addAction(accion_aviso)
            menu.exec_(event.globalPos())
            return
        accion_copiar = QAction("📋 Copiar texto seleccionado" + (" (reconocido por OCR)" if self.texto_por_ocr
else ""), self)
        accion_copiar.triggered.connect(lambda: QApplication.clipboard().setText(self.texto_seleccionado_actual))
        menu.addAction(accion_copiar)

        if self.parent_dialog and hasattr(self.parent_dialog, 'parent_dialogo_componente') and
self.parent_dialog.parent_dialogo_componente:
            padre = self.parent_dialog.parent_dialogo_componente
```

```
            menu.addSeparator()
            accion_chip = QAction("⚡ Usar como '7. Chip Principal'", self)
            accion_chip.triggered.connect(lambda: padre.txt_chip.setText(self.texto_seleccionado_actual))
            menu.addAction(accion_chip)

            accion_desc = QAction("📝 Usar como '5. Descripción'", self)
            accion_desc.triggered.connect(lambda:
padre.txt_descripcion.setPlainText(self.texto_seleccionado_actual))
            menu.addAction(accion_desc)

            accion_ubic = QAction("📍 Usar como '11. Ubicación Física'", self)
            accion_ubic.triggered.connect(lambda: padre.txt_ubicacion.setText(self.texto_seleccionado_actual))
            menu.addAction(accion_ubic)

        menu.exec_(event.globalPos())


class DialogoVisorPDF(QDialog):
    def __init__(self, pdf_path, parent_dialogo_componente=None, parent=None, es_modo_etiquetas=False,
titulo=None):
        super().__init__(parent)
        self.parent_dialogo_componente = parent_dialogo_componente
        self.es_modo_etiquetas = es_modo_etiquetas

        titulo_ventana = "🖨️ Previsualización de Etiquetas" if self.es_modo_etiquetas else f"📖 Visor de Documento
PDF - {titulo or os.path.basename(pdf_path)}"
        self.setWindowTitle(titulo_ventana)
        self.setWindowFlags(Qt.Window | Qt.WindowMaximizeButtonHint | Qt.WindowMinimizeButtonHint |
Qt.WindowCloseButtonHint)
        self.setWindowState(Qt.WindowMaximized)   # siempre abre maximizada

        self.pdf_path = pdf_path
        self.doc = fitz.open(self.pdf_path)

        hacer_links_clicables_en_pdf(self.doc)

        self.current_page = 0
        self.total_pages = len(self.doc)
        self.zoom_level = 1.5
        self.init_ui()

    def init_ui(self):
        layout = QVBoxLayout(self)
        controls_layout = QHBoxLayout()
        self.btn_zoom_menos = QPushButton("🔍➖ Reducir")
        self.btn_zoom_menos.setAutoDefault(False); self.btn_zoom_menos.setDefault(False)
        self.btn_zoom_menos.clicked.connect(self.zoom_menos)

        self.btn_zoom_mas = QPushButton("🔍➕ Ampliar")
        self.btn_zoom_mas.setAutoDefault(False); self.btn_zoom_mas.setDefault(False)
        self.btn_zoom_mas.clicked.connect(self.zoom_mas)

        self.btn_anterior = QPushButton("◀ Anterior")
        self.btn_anterior.setAutoDefault(False); self.btn_anterior.setDefault(False)
        self.btn_anterior.clicked.connect(self.pagina_anterior)

        self.lbl_paginacion = QLabel()
        self.lbl_paginacion.setAlignment(Qt.AlignCenter)
        self.lbl_paginacion.setFont(QFont("Arial", 10, QFont.Bold))

        self.btn_siguiente = QPushButton("Siguiente ▶")
        self.btn_siguiente.setAutoDefault(False); self.btn_siguiente.setDefault(False)
        self.btn_siguiente.clicked.connect(self.pagina_siguiente)

        controls_layout.addWidget(self.btn_zoom_menos)
        controls_layout.addWidget(self.btn_zoom_mas)
        controls_layout.addSpacing(15)
        controls_layout.addWidget(self.btn_anterior)
        controls_layout.addWidget(self.lbl_paginacion)
        controls_layout.addWidget(self.btn_siguiente)
        controls_layout.addSpacing(15)

        if not self.es_modo_etiquetas:
            # Botones para alternar modo conmutador (Texto vs Marco Rojo Recorte)
            self.btn_modo_texto = QPushButton("🖊️ Modo Texto")
            self.btn_modo_texto.setCheckable(True)
            self.btn_modo_texto.setChecked(True)
            self.btn_modo_texto.setStyleSheet("background-color: #007bff; color: white; font-weight: bold;
padding: 5px 10px;")
            self.btn_modo_texto.clicked.connect(lambda: self.cambiar_modo_visor("texto"))

            self.btn_modo_recorte = QPushButton("✂ Modo Recorte (Marco Rojo)")
```

```
            self.btn_modo_recorte.setCheckable(True)
            self.btn_modo_recorte.setChecked(False)
            self.btn_modo_recorte.setStyleSheet("background-color: #6c757d; color: white; font-weight: bold;
padding: 5px 10px;")
            self.btn_modo_recorte.clicked.connect(lambda: self.cambiar_modo_visor("recorte"))

            controls_layout.addWidget(self.btn_modo_texto)
            controls_layout.addWidget(self.btn_modo_recorte)

        layout.addLayout(controls_layout)

        if not self.es_modo_etiquetas:
            # Guardado de imágenes: se activa solo en modo recorte. Guarda en la carpeta que elija el usuario.
            estilo = ("QPushButton { color: white; font-weight: bold; padding: 5px 10px; background-color: %s; }"
                      "QPushButton:disabled { background-color: #ced4da; color: #868e96; }")
            recorte_layout = QHBoxLayout()
            self.btn_guardar_recorte = QPushButton("✅ Guardar Recorte")
            self.btn_guardar_recorte.setAutoDefault(False); self.btn_guardar_recorte.setDefault(False)
            self.btn_guardar_recorte.setStyleSheet(estilo % "#28a745")
            self.btn_guardar_recorte.clicked.connect(self.guardar_recorte)
            self.btn_guardar_entera = QPushButton("💾 Guardar Imagen Entera")
            self.btn_guardar_entera.setAutoDefault(False); self.btn_guardar_entera.setDefault(False)
            self.btn_guardar_entera.setStyleSheet(estilo % "#17a2b8")
            self.btn_guardar_entera.clicked.connect(self.guardar_imagen_entera)
            self.btn_guardar_recorte.setEnabled(False)
            self.btn_guardar_entera.setEnabled(False)
            recorte_layout.addWidget(self.btn_guardar_recorte)
            recorte_layout.addWidget(self.btn_guardar_entera)
            recorte_layout.addStretch()
            layout.addLayout(recorte_layout)

        lbl_hint = QLabel("💡 Modo Texto: seleccione texto y use clic derecho (si la página es una imagen se
reconoce con OCR). Modo Recorte: arrastre el mouse para marcar una región y pulse «Guardar Recorte»; «Guardar
Imagen Entera» guarda la página completa.")
        lbl_hint.setWordWrap(True)
        lbl_hint.setStyleSheet("background-color: #e9ecef; color: #333; padding: 6px; border-radius: 4px;")
        layout.addWidget(lbl_hint)

        self.scroll_area = QScrollArea()
        self.scroll_area.setWidgetResizable(True)
        self.scroll_area.setStyleSheet("background-color: #e0e0e0;")
        self.actualizar_visor_pagina()
        layout.addWidget(self.scroll_area)

    def cambiar_modo_visor(self, modo):
        if modo == "texto":
            self.btn_modo_texto.setChecked(True)
            self.btn_modo_texto.setStyleSheet("background-color: #007bff; color: white; font-weight: bold;
padding: 5px 10px;")
            self.btn_modo_recorte.setChecked(False)
            self.btn_modo_recorte.setStyleSheet("background-color: #6c757d; color: white; font-weight: bold;
padding: 5px 10px;")
        else:
            self.btn_modo_recorte.setChecked(True)
            self.btn_modo_recorte.setStyleSheet("background-color: #dc3545; color: white; font-weight: bold;
padding: 5px 10px;")
            self.btn_modo_texto.setChecked(False)
            self.btn_modo_texto.setStyleSheet("background-color: #6c757d; color: white; font-weight: bold;
padding: 5px 10px;")

        en_recorte = (modo == "recorte")
        self.btn_guardar_recorte.setEnabled(en_recorte)
        self.btn_guardar_entera.setEnabled(en_recorte)

        if hasattr(self, 'pdf_widget') and self.pdf_widget:
            self.pdf_widget.set_modo(modo)

    def _chip_actual(self):
        comp = self.parent_dialogo_componente
        return (comp.txt_chip.text().strip() if comp is not None else "") or "componente"

    def guardar_recorte(self):
        imagen = self.pdf_widget.imagen_recorte()
        if imagen is None:
            QMessageBox.information(self, "Sin región", "Marque primero una región con el mouse (arrastrando sobre
la página).")
            return
        pedir_destino_y_guardar(self, imagen, "recorte.png", "Guardar Recorte Como...", self._chip_actual())

    def guardar_imagen_entera(self):
        pedir_destino_y_guardar(self, self.pdf_widget.imagen_pagina(), f"pagina_{self.current_page + 1}.png",
                                "Guardar Imagen Entera Como...", self._chip_actual())
```

```
    def actualizar_visor_pagina(self):
        if 0 <= self.current_page < self.total_pages:
            page = self.doc[self.current_page]
            modo_actual = "recorte" if (not self.es_modo_etiquetas and hasattr(self, 'btn_modo_recorte') and
self.btn_modo_recorte.isChecked()) else "texto"

            self.pdf_widget = WidgetVisorPDFInteractivo(page, self.zoom_level, self)
            self.pdf_widget.set_modo(modo_actual)

            self.scroll_area.setWidget(self.pdf_widget)
            self.lbl_paginacion.setText(f"Página {self.current_page + 1} de {self.total_pages} (Zoom:
{int(self.zoom_level * 100)}%)")
            self.btn_anterior.setEnabled(self.current_page > 0)
            self.btn_siguiente.setEnabled(self.current_page < self.total_pages - 1)

    def zoom_mas(self):
        if self.zoom_level < 3.0:
            self.zoom_level += 0.25
            self.actualizar_visor_pagina()

    def zoom_menos(self):
        if self.zoom_level > 1.0:
            self.zoom_level -= 0.25
            self.actualizar_visor_pagina()

    def pagina_anterior(self):
        if self.current_page > 0:
            self.current_page -= 1
            self.actualizar_visor_pagina()

    def pagina_siguiente(self):
        if self.current_page < self.total_pages - 1:
            self.current_page += 1
            self.actualizar_visor_pagina()
```

Apéndice C. Base de datos: estructura en árbol

La aplicación guarda todo en un único archivo SQLite, Base_Inventario\inventario_electronica.sqlite3. Sus tablas se crean en la función inicializar_base_de_datos de db.py, que además agrega las columnas que falten en bases creadas por versiones anteriores. Cada conexión activa PRAGMA foreign_keys = ON, de modo que SQLite hace cumplir las relaciones entre tablas.

Este apéndice se obtuvo creando una base nueva con el propio código de la aplicación y leyendo su estructura real. Leyenda: PK = clave primaria, FK = clave foránea, UQ = combinación única, NN = no admite vacío (NOT NULL), NOCASE = se compara sin distinguir mayúsculas.

C.1 Vista general

```
inventario_electronica.sqlite3
├── Datos del inventario
│   ├── productos_fabricantes ····· un componente por fila (23 columnas)
│   │   ├── archivos_componentes ····· N archivos por componente      (FK, borrado en cascada)
│   │   └── parametros_electricos ····· N parámetros por componente    (FK, borrado en cascada)
├── Catálogos y listas (alimentan los desplegables)
│   ├── catalogo_clasificacion ····· taxonomía: tipo > categoría > subcategoría > ítem
│   ├── plantillas_parametros ····· parámetros propuestos para cada nodo de la taxonomía
│   └── listas_valores ····· fabricantes, interfaces, pines, monedas…
├── Valorización
│   └── cotizaciones ····· unidades de cada moneda por 1 USD
└── (interna de SQLite) sqlite_sequence ··· último id usado de cada tabla con AUTOINCREMENT
```

C.2 Árbol completo de tablas y columnas

```
inventario_electronica.sqlite3
│
├── productos_fabricantes                      ← Componentes
│   ├── id ····· INTEGER   PK, autoincremental
│   ├── nombre ····· TEXT      nombre del componente (la grilla lo muestra como Nombre)
│   ├── producto_id ····· TEXT      copia de nombre (se completa al guardar)
│   ├── ── Clasificación (texto; ver nota 1) ──
│   ├── tipo_componente ····· TEXT      nivel 1 de la taxonomía
│   ├── categoria ····· TEXT      nivel 2
│   ├── subcategoria ····· TEXT      nivel 3
│   ├── item_especifico ····· TEXT      nivel 4
│   ├── ── Identificación ──
│   ├── chip_principal ····· TEXT      chip o referencia principal
│   ├── descripcion ····· TEXT      descripción libre (en mayúsculas)
│   ├── fabricante ····· TEXT      lista «fabricante»
│   ├── interfaz_comunicacion ····· TEXT      lista «interfaz» (varias opciones separadas por coma)
│   ├── pines_relevantes ····· TEXT      lista «pines» (varias, separadas por coma)
│   ├── encapsulado ····· TEXT      lista «encapsulado» (varias, separadas por coma)
│   ├── obsolescencia ····· TEXT      estado de obsolescencia
│   ├── ── Existencia ──
│   ├── stock ····· INTEGER   unidades en existencia
│   ├── ubicacion ····· TEXT      lugar físico de almacenamiento
│   ├── ── Precios ──
│   ├── precio ····· REAL      precio único heredado (el capital usa los dos siguientes)
│   ├── proveedor_local ····· TEXT      lista «proveedor_local»
│   ├── precio_local ····· REAL      precio por unidad en el mercado local
│   ├── moneda_local ····· TEXT      lista «moneda»
│   ├── proveedor_exterior ····· TEXT      lista «proveedor_exterior»
│   ├── precio_exterior ····· REAL      precio por unidad en el exterior
│   ├── moneda_exterior ····· TEXT      lista «moneda»
│   └── qr_path ····· TEXT      columna heredada: la aplicación ya no la escribe
│
├── archivos_componentes                       ← Archivos adjuntos de cada componente
│   ├── id ····· INTEGER   PK, autoincremental
│   ├── componente_id ····· INTEGER   NN, FK → productos_fabricantes.id  (ON DELETE CASCADE)
│   ├── tipo_archivo ····· TEXT      NN, «imagen», «datasheet» o «tutorial»
│   ├── nombre_amigable ····· TEXT      NN, nombre que se muestra en la lista
│   └── ruta_archivo ····· TEXT      NN, ruta relativa a adjuntos (almacen/xx/hash.ext)
│
├── parametros_electricos                      ← Parámetros técnicos de cada componente
│   ├── id ····· INTEGER   PK, autoincremental
│   ├── componente_id ····· INTEGER   NN, FK → productos_fabricantes.id  (ON DELETE CASCADE)
│   ├── parametro_clave ····· TEXT      NN, nombre del parámetro (por ejemplo, Tensión)
│   └── parametro_valor ····· TEXT      NN, valor ingresado (texto libre con unidad)
│
├── catalogo_clasificacion                     ← Taxonomía (una fila por camino)
```

```
│   ├── id ····························· INTEGER   PK, autoincremental
│   ├── tipo ··························· TEXT      NN, NOCASE, nivel 1
│   ├── categoria ····················· TEXT      NN, NOCASE, '' si el nivel no está definido
│   ├── subcategoria ·················· TEXT      NN, NOCASE, '' si no está definido
│   ├── item ··························· TEXT      NN, NOCASE, '' si no está definido
│   └── UQ (tipo, categoria, subcategoria, item)
│
├── plantillas_parametros                      ← Parámetros propios de cada nodo de la taxonomía
│   ├── id ····························· INTEGER   PK, autoincremental
│   ├── tipo, categoria, subcategoria, item ·· TEXT   camino del nodo (NOCASE; '' = nivel sin definir)
│   ├── parametro ····················· TEXT      NN, NOCASE, nombre del parámetro propuesto
│   ├── orden ·························· INTEGER   NN, orden de aparición (por defecto 0)
│   └── UQ (tipo, categoria, subcategoria, item, parametro)
│
├── listas_valores                             ← Opciones de los desplegables
│   ├── id ····························· INTEGER   PK, autoincremental
│   ├── lista ·························· TEXT      NN, nombre de la lista (ver C.4)
│   ├── valor ·························· TEXT      NN, NOCASE, la opción
│   ├── orden ·························· INTEGER   NN, orden en el desplegable (por defecto 0)
│   └── UQ (lista, valor)
│
└── cotizaciones                               ← Tipo de cambio (moneda base: USD)
    ├── moneda ························· TEXT      PK, NOCASE, código o nombre de la moneda
    ├── por_usd ························ REAL      unidades de esa moneda por 1 USD (USD = 1.0)
    └── fecha ·························· TEXT      fecha de la cotización (puede estar vacía)
```

C.3 Relaciones

```
productos_fabricantes (id)
├──< archivos_componentes.componente_id        1 componente → N archivos     (ON DELETE CASCADE)
└──< parametros_electricos.componente_id       1 componente → N parámetros   (ON DELETE CASCADE)

Relaciones por nombre (sin clave foránea; las mantiene la aplicación):
├── productos_fabricantes.(tipo_componente, categoria, subcategoria, item_especifico)
│       ↔ catalogo_clasificacion.(tipo, categoria, subcategoria, item)
├── catalogo_clasificacion ↔ plantillas_parametros   (mismo camino; parámetros heredados hacia abajo)
├── productos_fabricantes.(fabricante, interfaz_comunicacion, pines_relevantes, encapsulado,
│       proveedor_local, proveedor_exterior, moneda_local, moneda_exterior)
│       ↔ listas_valores.(lista, valor)
└── productos_fabricantes.(moneda_local, moneda_exterior)  ↔  cotizaciones.moneda
```

Al eliminar un componente, SQLite borra automáticamente sus filas de archivos_componentes y parametros_electricos. Los archivos físicos no se borran en ese momento: quedan sin referencia en adjuntos\almacen y la aplicación los mueve a adjuntos\papelera al arrancar si ya no los usa ningún componente.

> **Nota:** (Nota 1) La clasificación de un componente se guarda como texto en sus cuatro columnas y no como clave foránea al catálogo. Por eso, al renombrar un nodo del catálogo la aplicación actualiza también los componentes que lo usan; y al eliminar un nodo, los componentes ya cargados conservan su clasificación escrita (solo deja de ofrecerse en los desplegables).

C.4 Contenido de la tabla listas_valores

La columna lista toma uno de estos siete valores. Cada uno alimenta uno o dos desplegables de la ventana de componente:

```
listas_valores
├── lista = fabricante ··········· alimenta productos_fabricantes.fabricante
├── lista = interfaz ············· alimenta interfaz_comunicacion   (selección múltiple)
├── lista = pines ················· alimenta pines_relevantes        (selección múltiple)
├── lista = encapsulado ··········· alimenta encapsulado            (selección múltiple)
├── lista = moneda ················ alimenta moneda_local y moneda_exterior
├── lista = proveedor_local ····· alimenta proveedor_local
└── lista = proveedor_exterior ··· alimenta proveedor_exterior

En las listas de selección múltiple los valores no pueden llevar comas (la coma separa las opciones elegidas).
Las listas fabricante, interfaz, pines, encapsulado, proveedor_local y proveedor_exterior
empiezan siempre con el valor «N/A» (sin dato), que no se puede renombrar ni eliminar.
```

C.5 Contenido inicial de una base nueva

Al crear una base vacía con la versión actual de la aplicación, las tablas de catálogos quedan sembradas a partir del archivo catalogo.py; las tablas de componentes empiezan vacías. Las cantidades corresponden a

esta versión del programa y crecen al usar la aplicación (lo que se escribe a mano en la ventana de componente se agrega al catálogo y a las listas).

Tabla	Filas iniciales	Origen
productos_fabricantes	0	Se cargan al usar la aplicación.
archivos_componentes	0	Se cargan al adjuntar archivos.
parametros_electricos	0	Se cargan al ingresar parámetros.
catalogo_clasificacion	74	Taxonomía inicial de catalogo.py.
plantillas_parametros	148	Parámetros por clasificación, de catalogo.py.
listas_valores	139	Listas de desplegables, de catalogo.py (incluye «N/A»).
cotizaciones	1	Moneda base USD con cotización 1.0.

Para ver la estructura con una herramienta externa, abrir el archivo .sqlite3 con DB Browser for SQLite (Manual de instalación, capítulo 3), con la aplicación cerrada, solo para consultar. Desde la línea de comandos de Python, el esquema completo se obtiene con:

```
python -c "import sqlite3; c =
sqlite3.connect('Base_Inventario/inventario_electronica.sqlite3'); print(*[r[0] for r
in c.execute('select sql from sqlite_master where sql is not null')], sep=chr(10)*2)"
```